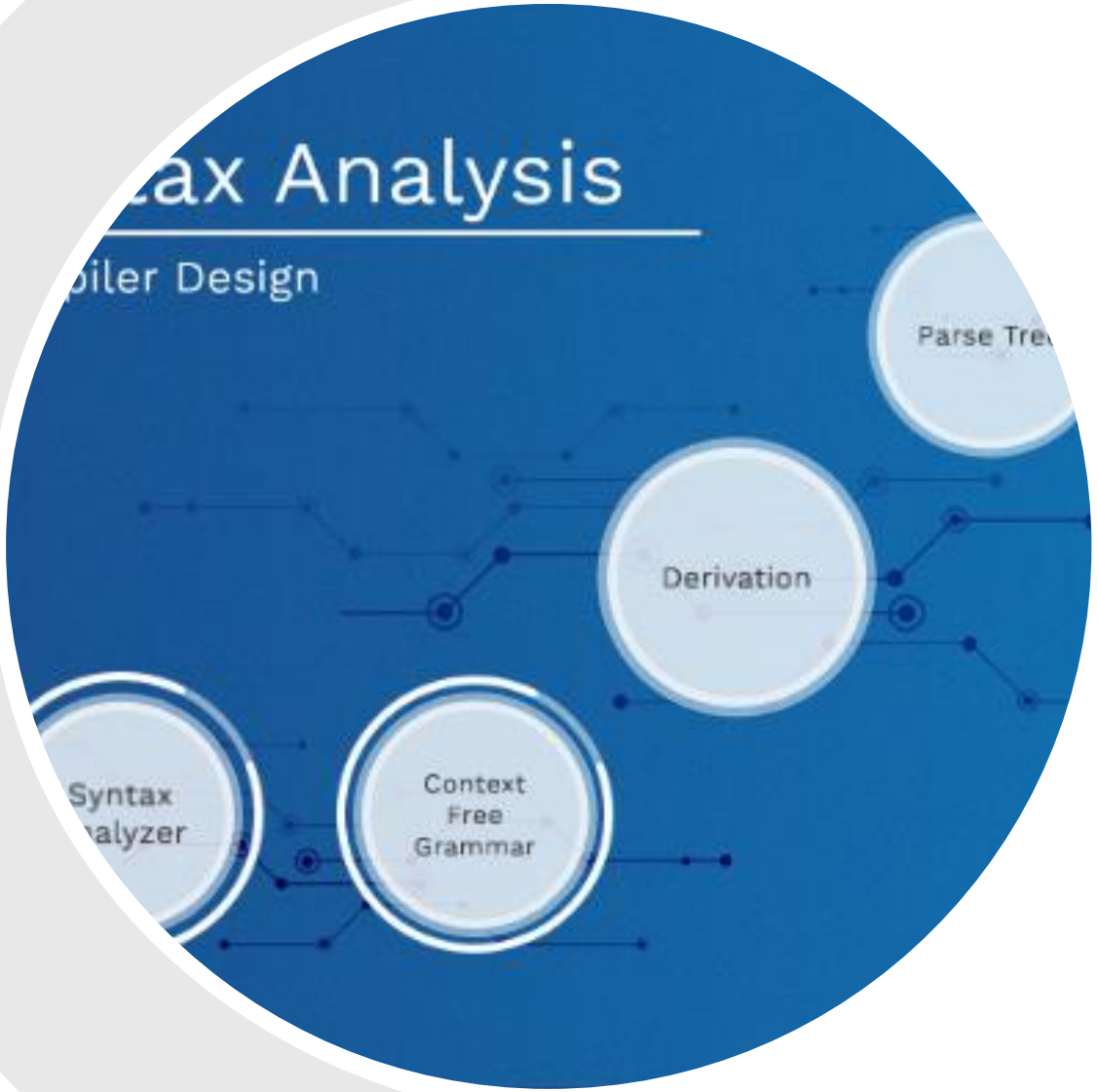


SYNTAX ANALYSIS

MODULE NO 3

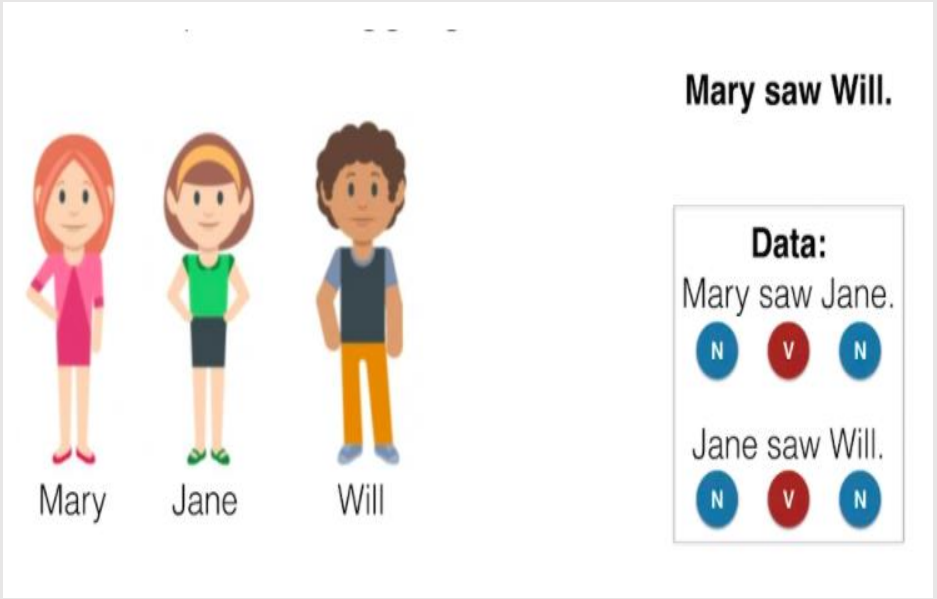
Compiled By : Mrs. Angelin Florence A

- POS Tagging
- Types of POS Tagging
- Parsers



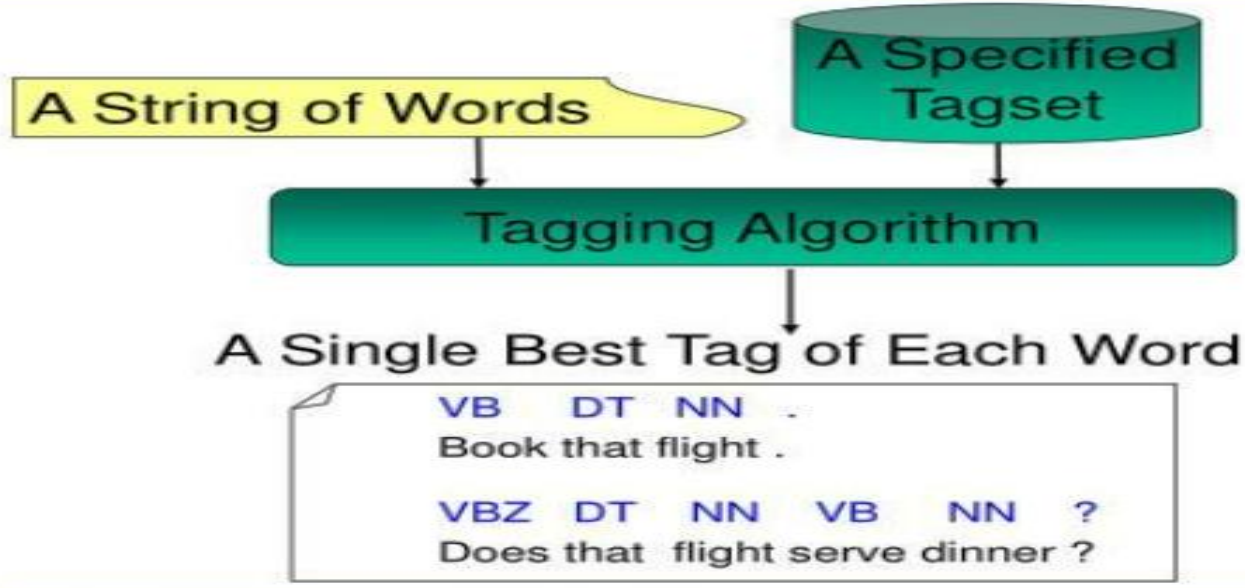
SYNTACTIC ANALYSIS

- Parts of Speech Tagging (POS)
- Tagset for English (Upenn Treebank)
- Difficulties and Challenges in POS tagging
- Rule Based, Stochastic and Transformation based
- Generative Model : Hidden Markov Model (HMM Veterbi) for POS Tagging
- Issues in HMM POS Tagging
- Discriminative Model : Maximum Entropy Model
- Conditional Random Field (CRF)
- Parsers: Top Down and Bottom up
- Modelling Constituency
- Bottom-up parser : CYK, PCFG, SR Parser
- Top-down Parser : Early Parser, Predictive Parser



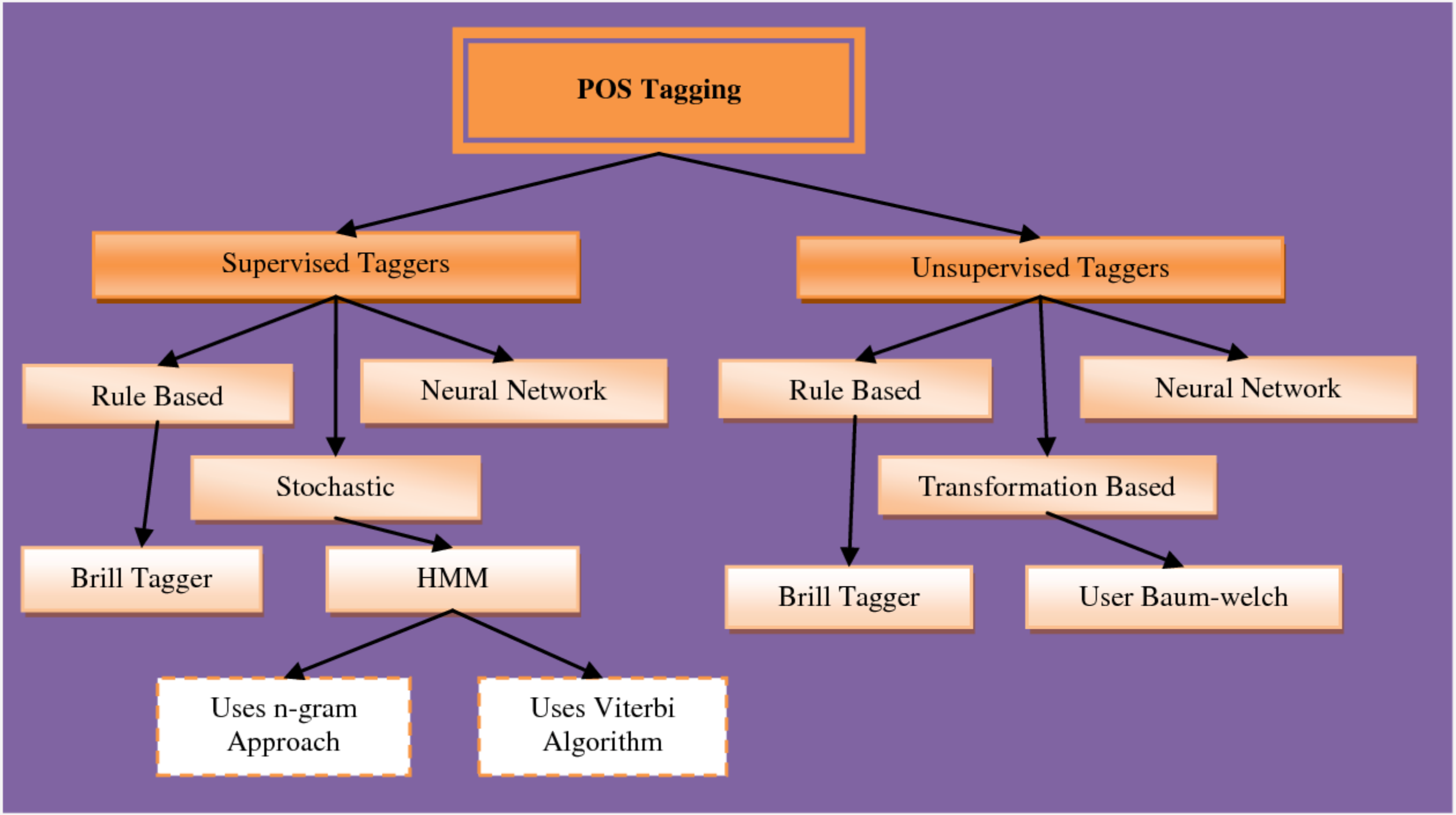
TYPES OF POS TAGGING

Process of POS Tagging

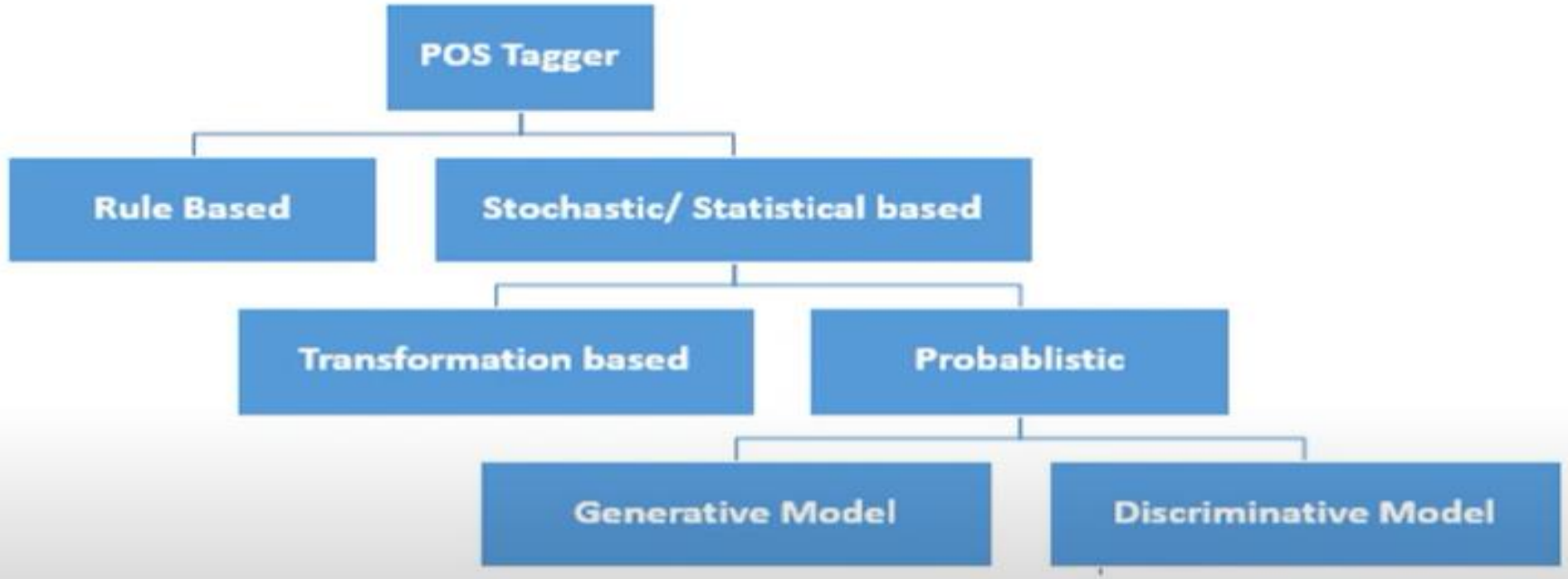


Two information sources used:

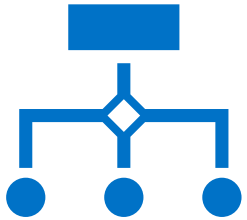
- Syntagmatic information (looking at information about tag sequences)
- Lexical information (predicting a tag based on the word concerned)


POS Tagging
Supervised Taggers
Unsupervised Taggers
Rule Based
Neural Network
Stochastic
Brill Tagger
HMM
Uses n-gram Approach
Uses Viterbi Algorithm
Rule Based
Neural Network
Transformation Based
Brill Tagger
User Baum-welch

Classification of Taggers



PART-OF-SPEECH TAGGING



•Rule-based taggers

Involve a large database of hand-written disambiguation rule specifying, for example, that *an ambiguous word is a noun rather than a verb if it follows a determiner.*



•Stochastic taggers

Resolve tagging ambiguities by using a training corpus to *count the probability of a given word having a given tag in each context.*

The Brill tagger, called the transformation-based tagger, shares features of both tagging architecture.

RULE BASED POS TAGGING (POS)

The earliest algorithms for automatically assigning POS were based on a two-stage architecture.

First, use a dictionary to assign each word a list of potential POS.

Second, use large lists of hand-written disambiguation rules to win now down this list to a single POS for each word.

The ENGTWOL tagger (1995) is based on the same two stage architecture, with much more sophisticated lexicon and disambiguation rules than before.

- Lexicon:
- 56000 entries
- A word with multiple POS is counted as separate entries

RULE BASED POS TAGGING (POS)

Word	POS	Additional POS features
smaller	ADJ	COMPARATIVE
entire	ADJ	ABSOLUTE ATTRIBUTIVE
fast	ADV	SUPERLATIVE
that	DET	CENTRAL DEMONSTRATIVE SG
all	DET	PREDETERMINER SG/PL QUANTIFIER
dog's	N	GENITIVE SG
furniture	N	NOMINATIVE SG NOINDEFDETERMINER
one-third	NUM	SG
she	PRON	PERSONAL FEMININE NOMINATIVE SG3
show	V	IMPERATIVE VFIN
show	V	PRESENT -SG3 VFIN
show	N	NOMINATIVE SG
shown	PCP2	SVOO SVO SV
occurred	PCP2	SV
occurred	V	PAST VFIN SV

Sample lexical entries from the ENGTWOL lexicon.

ADVERBIAL-THAT RULE

Given input: "that"

if

(+1 A/ADV/QUANT); /* *if next word is adj, adverb, or quantifier* */

(+2 SENT-LIM); /* *and following which is a sentence boundary,* */

(NOT -1 SVOC/A); /* *and the previous word is not a verb like* */

/* *'consider' which allows adjs as object complements* */

then eliminate non-ADV tags

else eliminate ADV tag

Example:

It isn't **that** odd!
ADV A

I consider **that** odd.
Compliment NUM

STOCHASTIC POS TAGGING

- Based on probability of certain tag occurring, given various possibilities
  - Necessitates a *training corpus*
- A collection of sentences that have already been tagged
  - Several such corpora exist
- Standard Corpus of Present-Day American English (or just the **Brown Corpus)**
- about 1,000,000 words from a wide variety of sources
  - POS tags assigned to each

Limitations of Rule-based Approach

- Hard coded rule are required.
- Rules has to be updated based on language change.
- It needs to check lexicon every time which is not efficient approach
- Strong language experts team is required.

2) Stochastic/ Statistical based Approach

It is based on concept of probability and machine learning.

Training and Test corpus are used.

Two approaches are there

a) Transformation based Tagger b) Probabilistic based Tagger

a) Transformation based Tagger

- This tagger is based on concept of Transformation-Based Learning (TBL) approach.
- TBL uses supervised learning.
- There is assumption of pre-tagged training corpus.
- It combines idea of the rule-based and stochastic taggers.
- Like the rule based taggers, TBL is based on rules that specify what tags should be assigned to what words.
- But like the stochastic taggers, TBL is a machine learning technique, in which rules are automatically induced from the data.

Label the training set with most frequent tags

e.g. **The can was rusted.**

The/DT can/MD was/VBD rusted/VBD

Add transformation rules to reduce training mistakes.

The/DT can/NN was/VBD rusted/VBN

Explanation: 'can' can be noun as well as modal verb also.

It uses machine learning as well as grammar rules.

1) Model verb is never preceded by determiner.

2) Also in corpus (training data) possibility of 'determiner model verb' pair is zero.

Therefore can is replaced as Noun tag

Explanation: 'rusted' can be VBD as well as VBN also.

1) In grammar rule, VBN is preceded by VBD

2) Also in corpus, possibility of "VBD-VBD' pair is negligible.

Statement:

Race is **incorrectly** tagged in the following statement:

is/VBZ expected/VBN to/TO race/NN tomorrow/NN

In the second case this race is **correctly** tagged as an NN:

the/DT race/NN for/IN outer/JJ space/NN

For example, in the Brown corpus, race is most likely to be a noun:

P(NN|race) = .98 **P(VB|race) = .02**

Therefore race is labelled as NN because its occurrence (probability) is more.

Brill's tagger learned a rule that applies exactly to this mistagging of race:

Change NN to VB when the previous tag is TO.

This rule would change race/NN to race/VB in exactly the following situation, since it is preceded by to/TO:

/VBN to/TO race/NN expected/VBN to/TO **race/VB**

b) Probabilistic:

Approach is to "pick the most-likely tag for this word".

Two approaches are there **whether you generate the data from the class or the class from the data.**

Problem statement: Data available of the form [d, c]

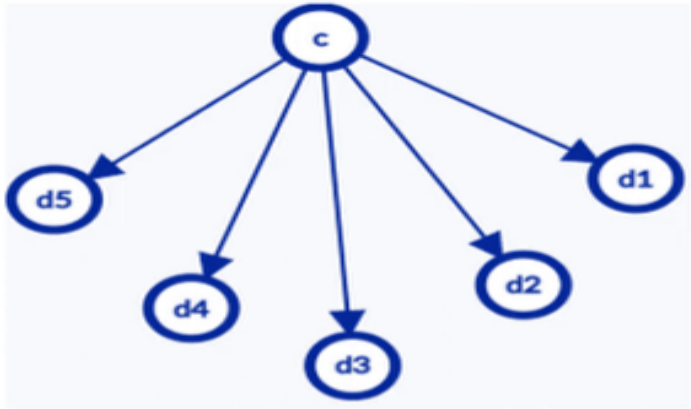
where d is observations and c is hidden classes.

Two types of Model are there

i) Generative Model

ii) Discriminative Model

1) Generative Model



Example are **Hidden Markov Model, Naive Bayes Classifier**

- In generative model, we assume that class is there and it generates data.
- For e.g. class is given (subjects), and all words are generated from subject.
- For e.g. Part- of- Speech Tagging: POS tags are there and words are generated from that.
- Here flow is downward (Classes generates data)

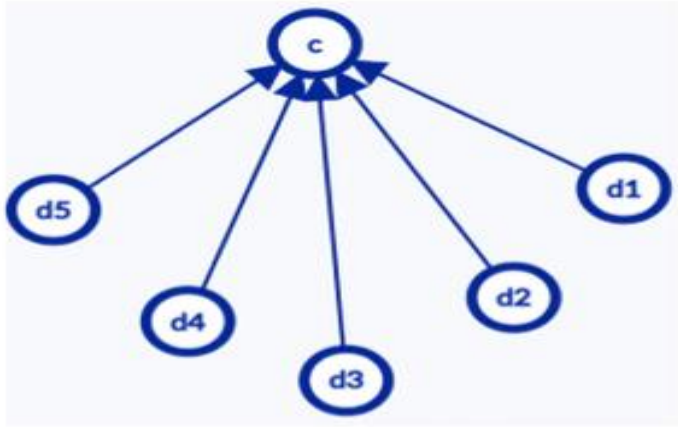
Generative Model



For e.g. POS tag noun(class) generates data like girl, boy, man, woman, water etc.

Statement is The girl is intelligent. In this case, determiner class generates the, NN class generates girl, VBZ generates is and JJ generates adjective.

2)Discriminative Model:

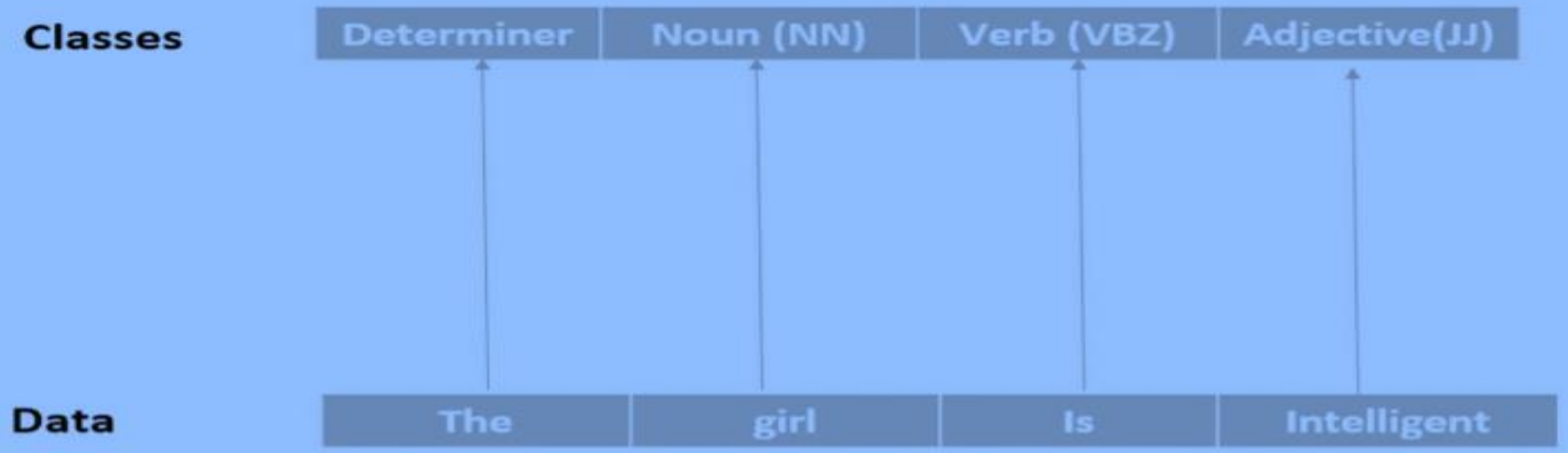


In the discriminative model, data is there and you assume that hidden state is a generated from the data. Here flow is upward. (that data identifies class)

For e.g. keywords of the subjects are given(data) and from the keywords we determine subject. Keywords are data and subject is class. Class is hidden. By seeing nature of data, we can assign probability of the particular class.

Example are **Maximum entropy model, Conditional Random field.**

Discriminative Model



For e.g. For every word, various features are given with respect to previous words and future words. From that features, we can decide POS tag for that word. That is flow is upward. For a given word, using features of neighboring word, we determine class or tag.

- **Stochastic (Probabilistic) tagging**: A stochastic approach includes frequency, probability or statistics.
- The simplest stochastic approach finds out the most frequently used tag for a specific word in the annotated training data and uses this information to tag that word in the unannotated text.
- But sometimes this approach comes up with sequences of tags for sentences that are not acceptable according to the grammar rules of a language.
- One such approach is to calculate the probabilities of various tag sequences that are possible for a sentence and assign the POS tags from the sequence with the highest probability.
- Hidden Markov Models (HMMs) are probabilistic approaches to assign a POS Tag.

WHY HIDDEN, MARKOV MODEL?

- The reason it is called a Hidden Markov Model is because an inference model is constructed based on the assumptions of a Markov process.
- The Markov process assumption is simply that the “future is independent of the past given the present”.
- To make this point clear, let us consider the scenario below where the weather, the hidden variable, can be hot, mild or cold and the observed variables are the type of clothing worn.
- The arrows represent transitions from a hidden state to another hidden state or from a hidden state to an observed variable.
- The Markov assumption, each state only depends on the previous state and not on any other prior states.

State Transition Probability: Based on Markov Model

N-gram Model: An **n-gram** is a contiguous sequence of **n** items from a given sample of text or speech. The items can be phonemes, syllables, letters, words or base pairs according to the application. The **n-grams** typically are collected from a text or speech corpus.

Conditional Probability: $P(B \mid A) = \frac{P(A,B)}{P(A)}$ $\qquad P(A,B) = P(A)P(B \mid A)$

More variables: $P(A,B,C,D) = P(A)P(B \mid A)P(C \mid A,B)P(D \mid A,B,C)$

Chain Rule:

$$P(x_1, x_2, \ldots x_n) = P(x_1)P(x_2 \mid x_1)P(x_3 \mid x_1, x_2) \ldots\ldots P(x_n \mid x_1, \ldots\ldots, x_{n-1})$$

Probability of words in sentences:

$$P(w_1, w_2, \ldots w_n) = \prod_i P(w_i \mid w_1, w_2, w_3 \ldots\ldots, w_{i-1})$$

Unigram(1-gram): **No history is used.**

Bi-gram(2-gram): **One word history**

Tri-gram(3-gram): **Two words history**

Four-gram(4-gram): **Three words history**

Five-gram(5-gram):**Four words history**

Bi-gram(2-gram): **One word history**

$$P(w_1, w_2) = \prod_{i=2} P(w_2 \mid w_1)$$

$$P(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

Bi-gram(2-gram): One word history

$$P(w_1, w_2) = \prod_{i=2} P(w_2 \mid w_1)$$

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

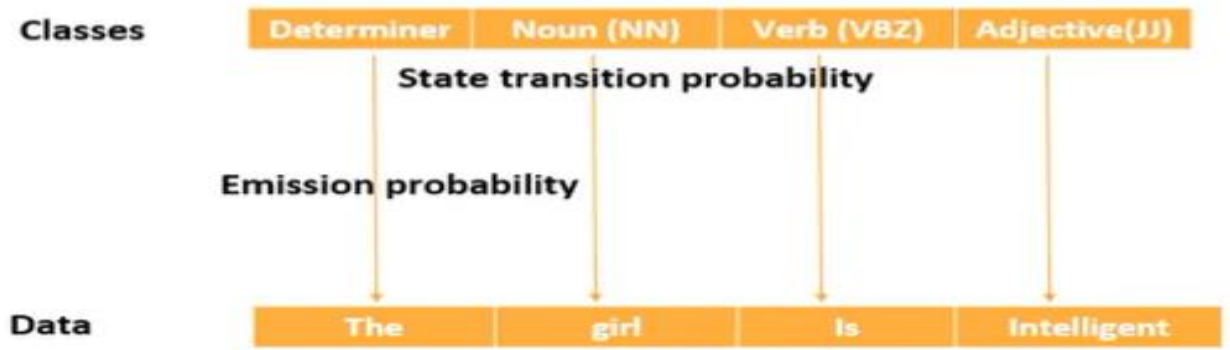
First order Markov assumption:

$p(q_n \mid q_{n-1})$

Second order Markov assumption:

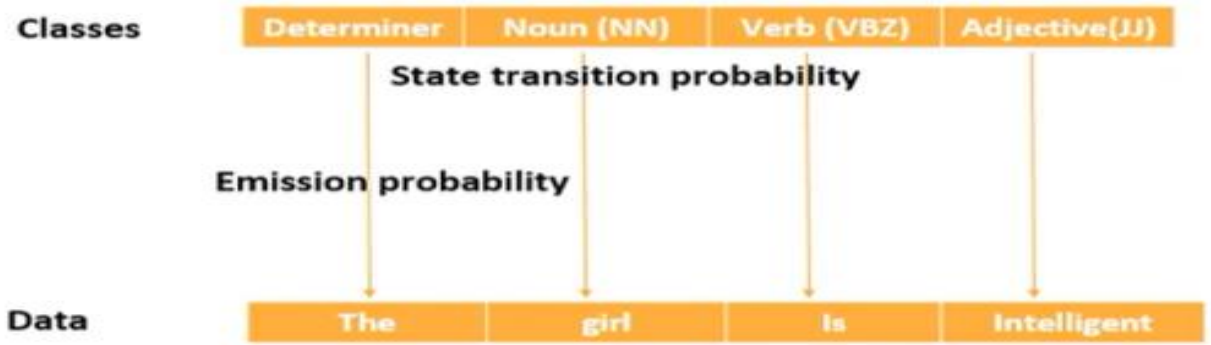
$p(q_n \mid q_{n-1}, q_{n-2})$

Emission Probability matrix gives this Bi-gram value.
For e.g. Appearances of pair:
Determiner-NN



Emission Probability: Based on Bayes rule of conditional probability

Conditional Probability: $P(B \mid A) = \frac{P(A,B)}{P(A)}$ $\qquad P(A,B) = P(A)P(B \mid A)$



$$P(Word|Class) = \frac{P(Class, Word)}{P(Class)} \qquad P(Girl|NN)$$

This value, we calculate from Emission Probability Matrix

Generative Model: Hidden Markov Model (HMM)

$$P(q_1, ..., q_n) = \prod_{i=1}^{n} P(q_i|q_{i-1})$$

Markov Chain Rule: Bigram

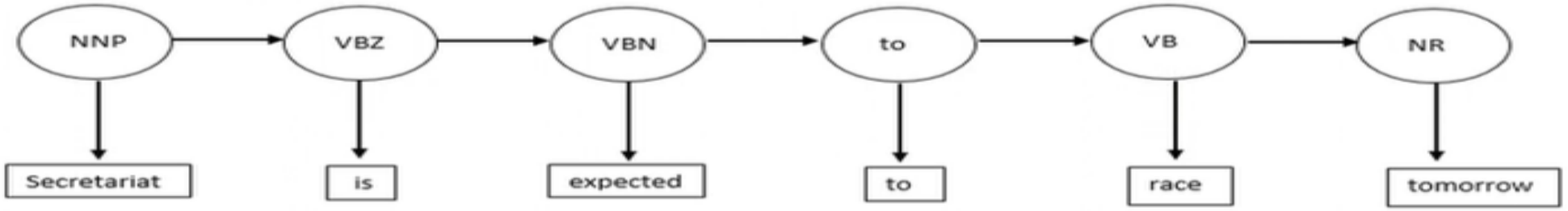
$$P(q_i|x_i) = \frac{P(x_i|q_i)P(q_i)}{P(x_i)}$$

Bayes rule of conditional probability

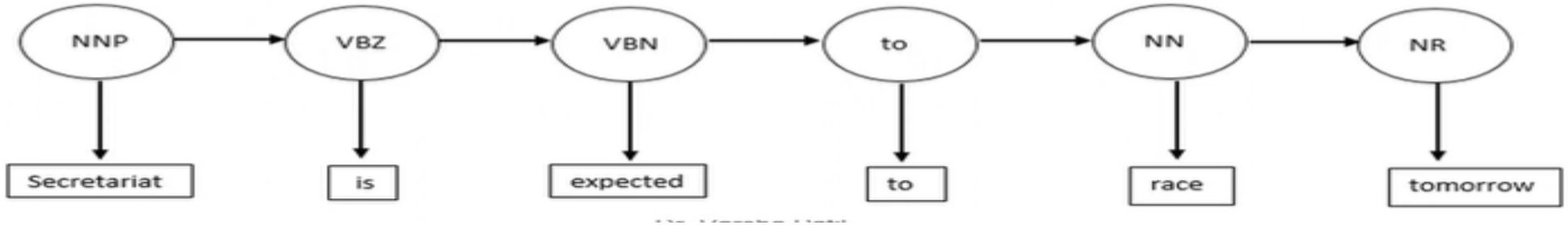
Hidden Markov Model (HMM)

HMM Model for POS Tagging

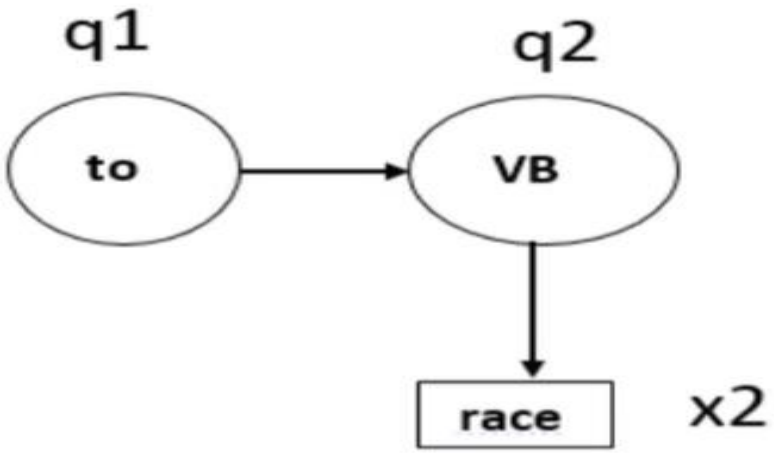
Race: VB



Race: NN



Decide with HMM, word 'Race' is noun or verb



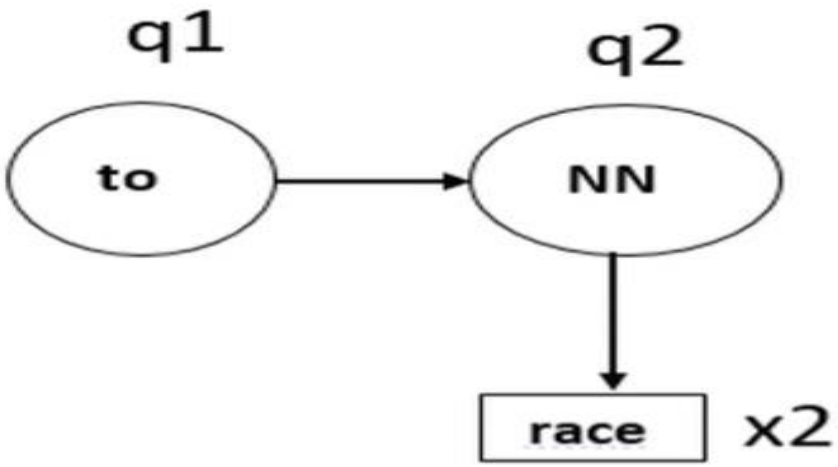
q1 and q2: States (class)

x2: word emitted from q2 state

Bayes rule **Markov Chain Rule: Bigram**

$$P(q2|x2,q1) = P(x2|q2) * P(q2|q1)$$

$$P(VB|race,to) = P(race|VB) * P(VB|to)$$



q1 and q2: States (class)

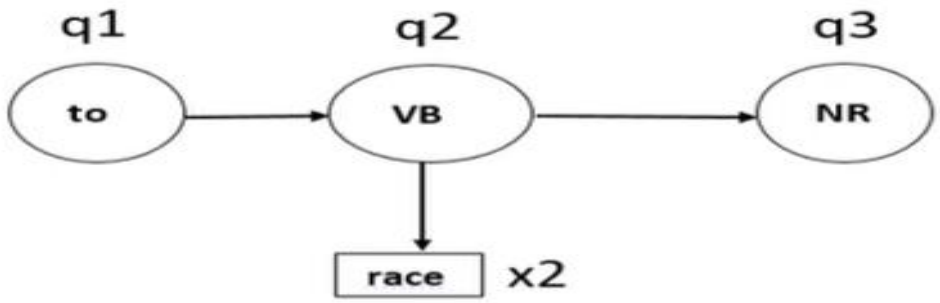
x2: word emitted from q2 state

Bayes rule **Markov Chain Rule: Bigram**

$$P(q2|x2,q1) = P(x2|q2) * P(q2|q1)$$

$$P(NN|race,to) = P(race|NN) * P(NN|to)$$

For better result, previous as well as next state is considered i.e. q1 and q3)

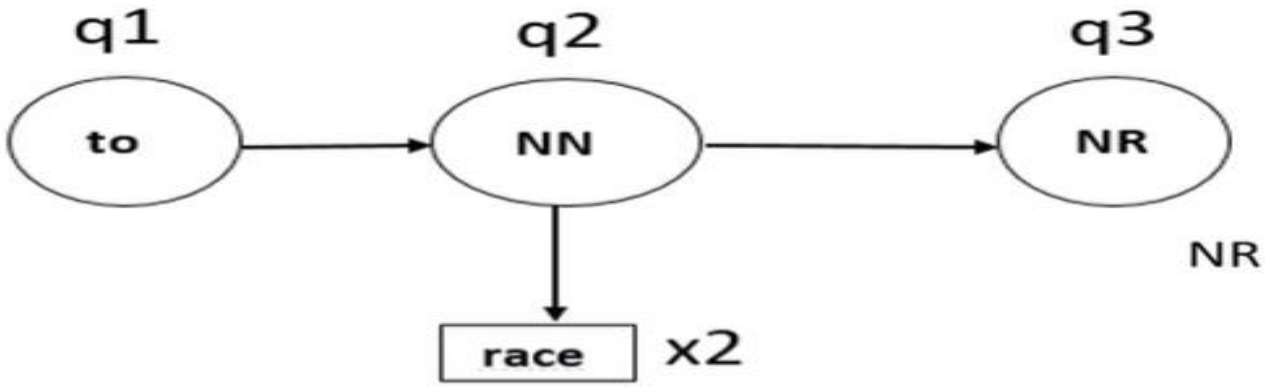


NR: Adverbial Noun

$$P(q2|x2, q1, q3) = P(x2|q2) * P(q2|q1) * P(q3|q2)$$

$$P(VB|race, to) = P(race|VB) * P(NR|VB)$$

$$P(VB|race, to) = 0.82 * 0.00012 * 0.0027 = 9.72 * 10^{-9}$$



NR: Adverbial Noun

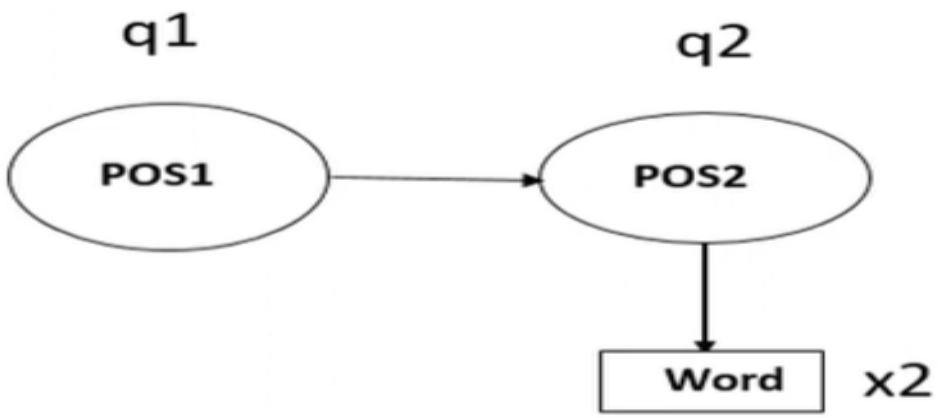
$$P(q2|x2, q1, q3) = P(x2|q2) * P(q2|q1) * P(q3|q2)$$

$$P(NN|race, to) = P(race|NN) * P(NR|NN)$$

$$P(NN|race, to) = 0.0012 * 0.0047 * 0.00057 = 3.21 * 10^{-9}$$

Probability of Verb is more than Noun. Therefore disambiguation process happens and race s assigned verb tag

Generative Model: Hidden Markov Model (HMM)



q1 and q2: States (class)

x2: word emitted from q2 state

$$P(POS2|word, POS1) = P(word|POS2) * P(POS2|POS1)$$

Bayes Rule **Bigram**

Bayes Rule data obtained from Emission Probability matrix while Bigram data is obtained from State Transition Matrix

Emission Probability Matrix:

- From a given corpus, identify unique tokens. Also identify total part of speech tags present in corpus.
- Assign correct Part of Speech (POS) tag for every token.
- Arrange tokens in horizontal and Part of Speech (POS) tag in vertical fashion.
- We can change this order also.
- This will provide data for Bayes rule emission probability.
- For N token, M part of speech tags, size of the emission matrix will be N×M.

State Transition Probability:

- Use part of speech tags found from emission probability matrix.
- Create matrix size of M×M.
- Use Bigram concept and fill the entries in a table.
- This will provide data for state transition probability.

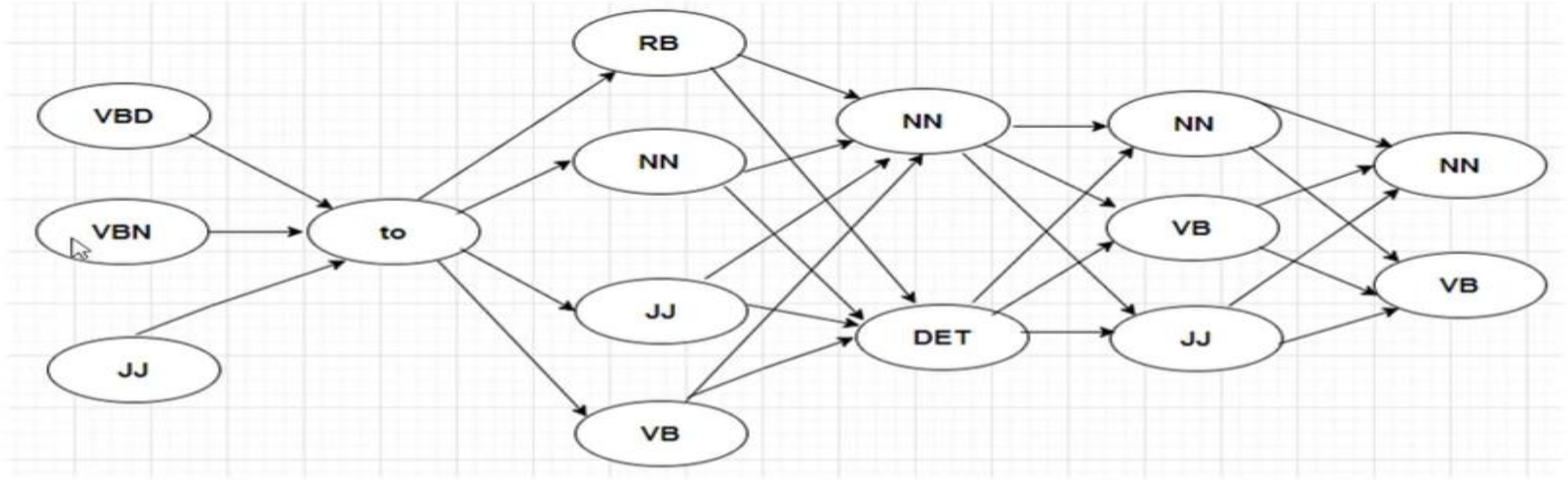
3

Need of Viterbi Decoding Algorithm:

- It is based on dynamic programming.
- Dynamic programming breakdowns the problems into sub-problems.
- It saves the result for future purposes therefore no not need to compute the result again.

Statement is : **Assured to back the silver coin**

assured	to	back	the	silver	coin
W1	W2	W3	W4	W5	W6
VBD: Past Tense	To: to	RB: Adverb	NN: Noun	NN: Noun	NN: Noun
VBN: Participle		NN: Noun	Determiner	JJ: Adjective	VB: Verb
JJ: Adjective		JJ: Adjective		VB: Verb	
		VB: Verb			
(3)	(1)	(4)	(2)	(3)	(2)
Total number of ways are: 3×1×4×2×3×2= 144 possibilities are there.					



Total number of ways are: 3×1×4×2×3×2= 144 possibilities are there.

Viterbi algorithm is used to find efficient path.

Training Corpus:

<S>	Martin	Justin	can	watch	Will	<E>
<S>	Spot	will	watch	Martin	<E>	
<S>	Will	Justin	spot	Martin	<E>	
<S>	Martin	will	pat	Spot	<E>	

<S> and <E> start and end of the statement respectively.

Step 1: Assign correct POS Tag to training corpus

Training Corpus with Correct POS Tag:

<S>	Martin	Justin	can	watch	Will	<E>
	N	N	M	V	N	
<S>	Spot	will	watch	Martin	<E>	
	N	M	V	N		
<S>	Will	Justin	spot	Martin	<E>	
	M	N	V	N		
<S>	Martin	will	pat	Spot	<E>	
	N	M	V	N		

N: Noun
M: Modal verb
V:Verb

Step 2: Creation of Emission Probability Matrix

	N	M	V
Martin	4/9	0	0
Justin			
Will			
Spot			
Can			
Watch			
Pat			
Σ			

<S>	Martin	Justin	can	watch	Will	<E>
	N	N	M	V	N	
<S>	Spot	will	watch	Martin	<E>	
	N	M	V	N		
<S>	Will	Justin	spot	Martin	<E>	
	M	N	V	N		
<S>	Martin	will	pat	Spot	<E>	
	N	M	V	N		

$$P(Word|Class) = \frac{P(Class, Word)}{P(Class)}$$

Step 2: Creation of Emission Probability Matrix

	N	M	V
Martin	4/9	0	0
Justin	2/9	0	0
Will	1/9	3/4	0
Spot	2/9	0	1/4
Can	0	1/4	0
Watch	0	0	2/4
Pat	0	0	1/4
Σ	9	4	4

<S>	Martin	Justin	can	watch	Will	<E>
	N	N	M	V	N	
<S>	Spot	will	watch	Martin	<E>	
	N	M	V	N		
<S>	Will	Justin	spot	Martin	<E>	
	M	N	V	N		
<S>	Martin	will	pat	Spot	<E>	
	N	M	V	N		

$$P(Word|Class) = \frac{P(Class, Word)}{P(Class)}$$

Step 3: Creation of State Transition Probability Matrix

	N	M	V	<E>
<S>	3/4	1/4	0	0
N				
M				
V				

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

<S>	Martin	Justin	can	watch	Will	<E>
	N	N	M	V	N	
<S>	Spot	will	watch	Martin	<E>	
	N	M	V	N		
<S>	Will	Justin	spot	Martin	<E>	
	M	N	V	N		
<S>	Martin	will	pat	Spot	<E>	
	N	M	V	N		

Dr. Varsha Patil

Step 3: Creation of State Transition Probability Matrix

	N	M	V	<E>
<S>	3/4	1/4	0	0
N	1/9	1/3	1/9	4/9
M	1/4	0	3/4	0
V	1	0	0	0

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

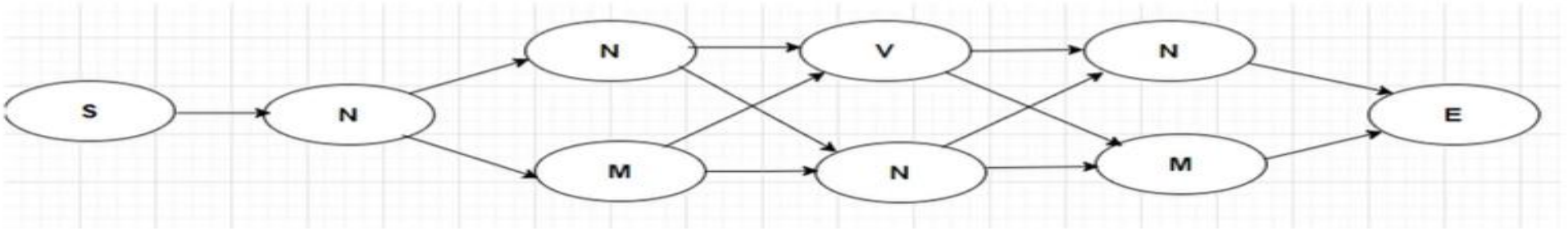
<S>	Martin	Justin	can	watch	Will	<E>
	N	N	M	V	N	
<S>	Spot	will	watch	Martin	<E>	
	N	M	V	N		
<S>	Will	Justin	spot	Martin	<E>	
	M	N	V	N		
<S>	Martin	will	pat	Spot	<E>	
	N	M	V	N		

Dr. Varsha Patil

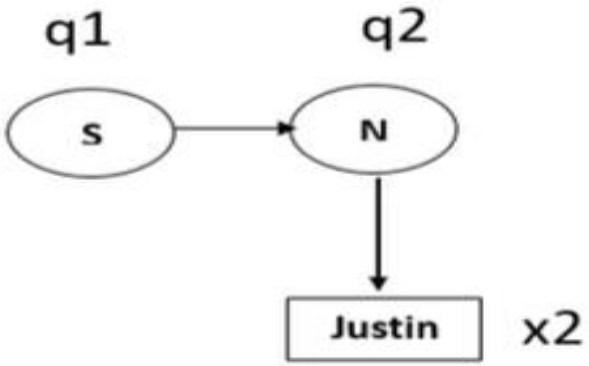
Test Data

<S> Justin will spot Will <E>

Justin	will	Spot	Will
N	N	V	N
	M	N	M
Total ways : 1×2×2×2=8			



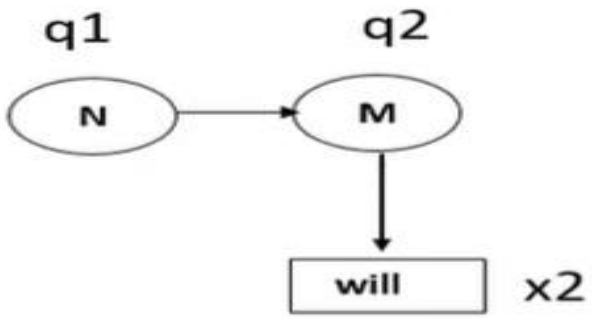
Justin



$$P(N|Justin, < S >) = P(Justin|N) * P(Noun| < S >)$$

$$P(N|Justin, < S >) = \frac{2}{9} \times \frac{3}{4} = \frac{1}{6}$$

Will as Modal Verb

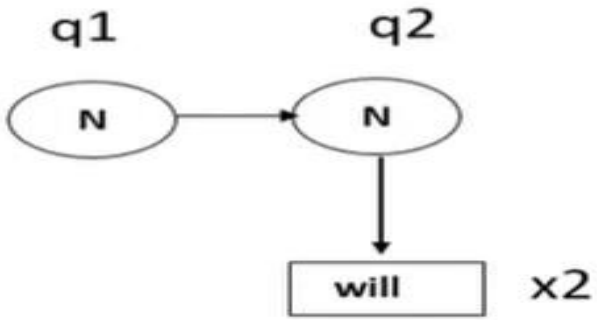


$$P(M|will, N) = P(will|M) * P(M|N)$$

$$P(M|will, N) = \frac{3}{4} \times \frac{1}{3} = \frac{1}{4}$$

$$P(M|will, N) = \frac{1}{4} \times \frac{1}{6} = \frac{1}{24}$$

Will as Noun



$$P(N|will, N) = P(will|N) * P(N|N)$$

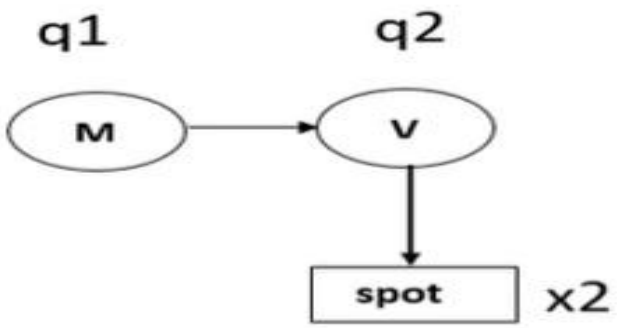
$$P(N|will, N) = \frac{1}{9} \times \frac{1}{9} = \frac{1}{81}$$

$$P(N|will, N) = \frac{1}{81} \times \frac{1}{6} = \frac{1}{486}$$

$$\frac{1}{24} > \frac{1}{486} \quad P(M) > P(N)$$

Result: will as Modal Verb

spot as Verb

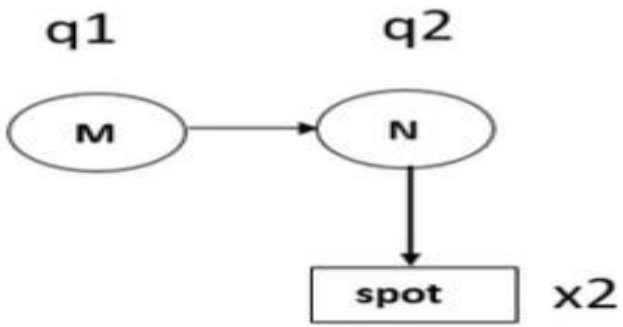


$$P(v|spot, M) = P(spot|v) * P(v|M)$$

$$P(v|spot, M) = \frac{1}{4} \times \frac{3}{4} = \frac{3}{16}$$

$$P(v|spot, M) = \frac{3}{16} \times \frac{1}{24} = \frac{1}{128}$$

spot as Noun



$$P(N|spot, M) = P(spot|N) * P(N|M)$$

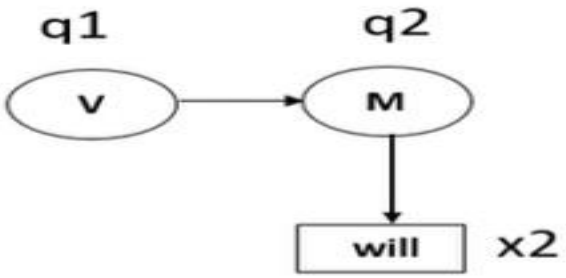
$$P(N|spot, M) = \frac{2}{9} \times \frac{1}{4} = \frac{1}{18}$$

$$P(N|spot, M) = \frac{1}{18} \times \frac{1}{24} = \frac{1}{432}$$

$$\frac{1}{128} > \frac{1}{432} \quad P(V) > P(N)$$

Result: spot as verb

will as modal Verb

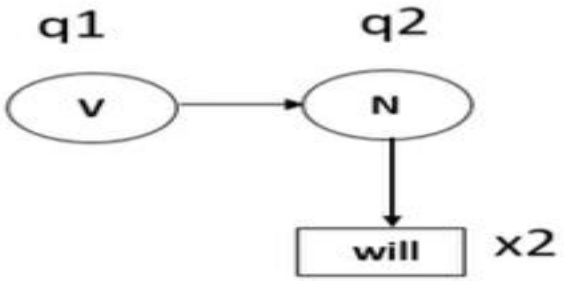


$$P(M|will, V) = P(will|M) * P(M|V)$$

$$P(M|will, V) = \frac{3}{4} \times \frac{1}{1000} = \frac{3}{4000}$$

$$P(M|will, V) = \frac{3}{4000} \times \frac{1}{128} = \frac{1}{512000}$$

will as Noun



$$P(N|will, V) = P(will|N) * P(N|V)$$

$$P(N|will, V) = \frac{1}{9} \times \frac{1}{1} = \frac{1}{9}$$

$$P(N|will, V) = \frac{1}{9} \times \frac{1}{128} = \frac{1}{1152}$$

Result: will as noun

Probability of will as Noun is more.

Result of POS tagging after applying HMM model

<S>	Justin	will	spot	Will	<E>
	N	M	V	N	

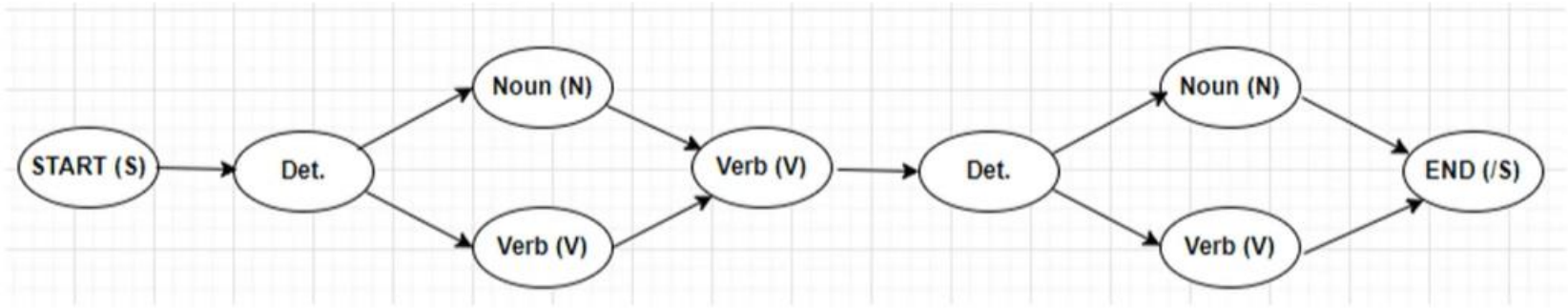
TRANSITION PROBABILITY & EMISSION PROBABILITY

- The transition probability is the likelihood of a particular sequence for example, Eg: “ Ted will spot Will ” how likely is that a noun is followed by a model and a model by a verb and a verb by a noun.
- This probability is known as Transition probability.
- It should be high for a particular sequence to be correct.
- what is the probability that the word Ted is a noun, will is a model, spot is a verb and Will is a noun.
- These sets of probabilities are Emission probabilities and should be high.

Training Corpus with Correct POS Tag

<S>	Book	a	car	</S>			
<S>	Park	the	car	</S>			
<S>	The	book	is	in	the	car	</S>
<S>	The	car	is	in	a	park	</S>

E.g. The park is a book



The park is a book

Total number of ways are: 1×2×1×1×2= 4 possibilities are there.

Viterbi algorithm is used to find efficient path.

Step 1: Assign correct POS Tag to training corpus

<S>	Book	a	car	</S>			
	Verb	Det.	Noun				
<S>	Park	the	car	</S>			
	Verb	Det.	Noun				
<S>	The	book	is	in	the	car	</S>
	Det.	Noun	Verb	Prep.	Det.	Noun	
<S>	The	car	is	in	a	park	</S>
	Det.	Noun	Verb	Prep.	Det.	Noun	

POS Tags

Verb

Det

Noun

Preposition

Step 2: Creation of Emission Probability Matrix

	Verb (4)	Det. (6)	Noun (6)	Prep. (2)
Book	1/4	0	1/6	0
a	0	2/6	0	0
car	0	0	4/6	0
the	0	4/6	0	0
park	1/4	0	1/6	0
is	2/4	0	0	0
in	0	0	0	2/2
Σ	4	6	6	2

<S>	Book	a	car	</S>			
	Verb	Det.	Noun				
<S>	Park	the	car	</S>			
	Verb	Det.	Noun				
<S>	The	book	is	in	the	car	</S>
	Det.	Noun	Verb	Prep.	Det.	Noun	
<S>	The	car	is	in	a	park	</S>
	Det.	Noun	Verb	Prep.	Det.	Noun	

$$P(Word|Class) = \frac{P(Class, Word)}{P(Class)}$$

Step 3: Creation of State Transition Probability Matrix

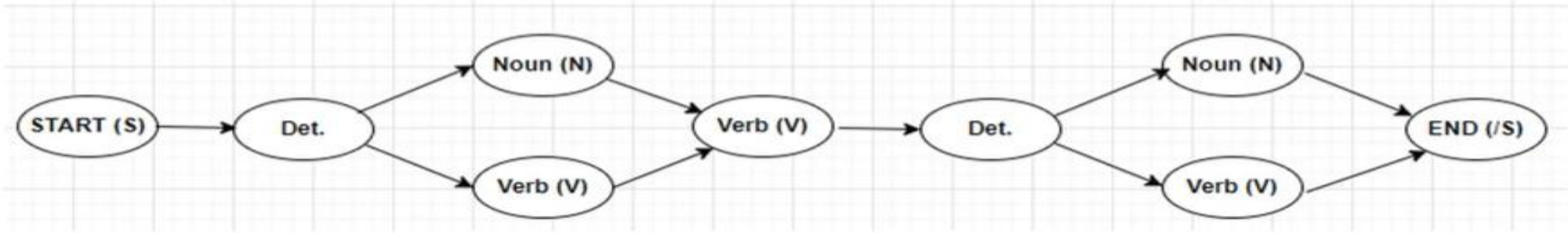
	Noun	Verb	Det.	Prep.	</S>
<S>	0	2/4	2/4	0	0
Noun	0	2/6	0	0	4/6
Verb	0	0	2/4	2/4	0
Det.	6/6	0	0	0	0
Prep.	0	0	2/2	0	0

<S>	Book	a	car	</S>			
	Verb	Det.	Noun				
<S>	Park	the	car	</S>			
	Verb	Det.	Noun				
<S>	The	book	is	in	the	car	</S>
	Det.	Noun	Verb	Prep.	Det.	Noun	
<S>	The	car	is	in	a	park	</S>
	Det.	Noun	Verb	Prep.	Det.	Noun	

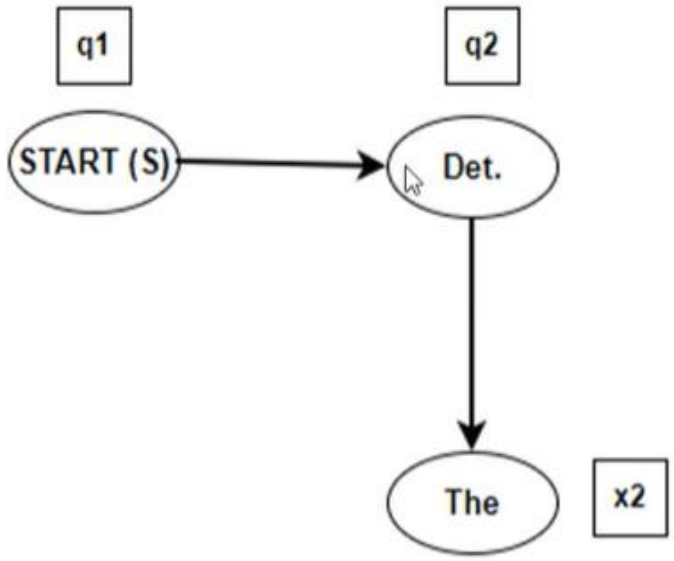
$$P(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

Test Data <S>The park is a book </S>

The	park	is	a	book
Det.	Noun	Verb	Det.	Noun
	Verb			Verb
1×2×1×1×2=4 ways are there				



The

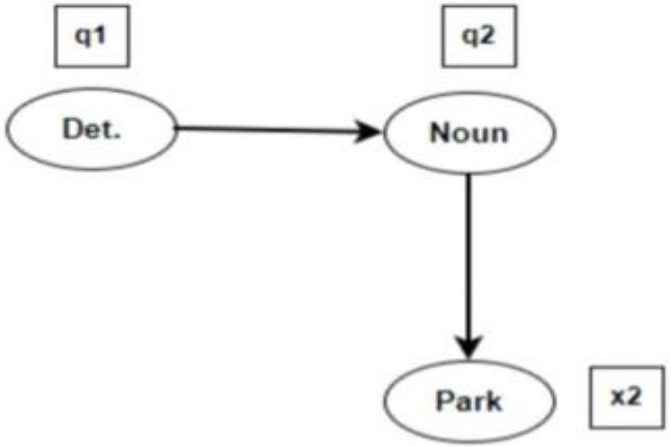


$$P(q2|x2, q1) = P(x2|q2) * P(q2|q1)$$

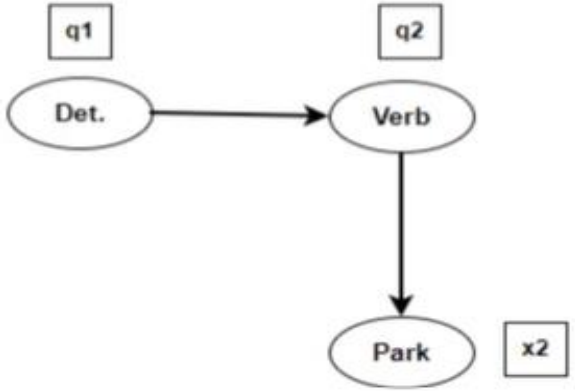
$$P(Det|The, <S>) = P(The|Det) * P(Det|<S>)$$

$$P(Det|The, <S>) = \frac{4}{6} \times \frac{2}{4} = \frac{1}{3}$$

Park as Noun



Park as verb



Previous Probability is 1/3

$$P(q2|x2, q1) = P(x2|q2) \times P(q2|q1)$$

$$P(Noun|Park, Det) = P(Park|Noun) \times P(Noun|Det.)$$

$$P(Noun|Park, Det.) = \frac{1}{6} \times \frac{6}{6} = \frac{1}{6}$$

$$P(Noun|Park, Det.) = \frac{1}{6} \times \frac{1}{3} = \frac{1}{18}$$

$$P(Veb|Park, Det) = P(Park|Verb) \times P(Verb|Det.)$$

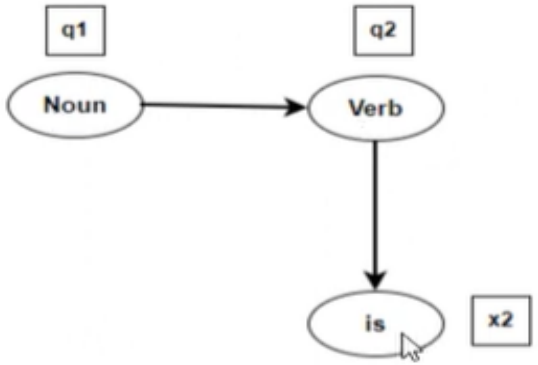
$$P(Verb|Park, Det.) = \frac{1}{4} \times \frac{0}{1} = 0$$ smoothing

$$P(Verb|Park, Det.) = \frac{1}{4} \times \frac{1}{100} = \frac{1}{400}$$

$$P(Verb|Park, Det.) = \frac{1}{400} \times \frac{1}{3} = \frac{1}{1200}$$

$$\frac{1}{18} > \frac{1}{1200}$$ Result: park as noun

is as Verb



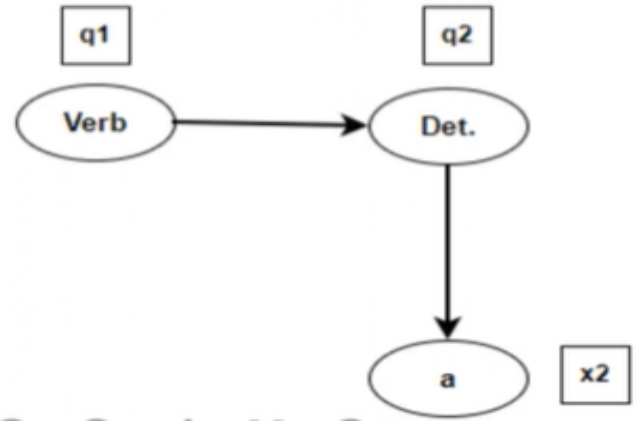
Previous Probability is 1/18

$$P(\text{Verb}|\text{is},\text{Noun}) = p(\text{is}|\text{Verb}) \times P(\text{Verb}|\text{Noun})$$

$$P(\text{Verb}|\text{is},\text{Noun}) = \frac{2}{4} \times \frac{2}{6} = \frac{1}{6}$$

$$= \frac{1}{6} \times \frac{1}{18} = \frac{1}{108}$$

a as Det



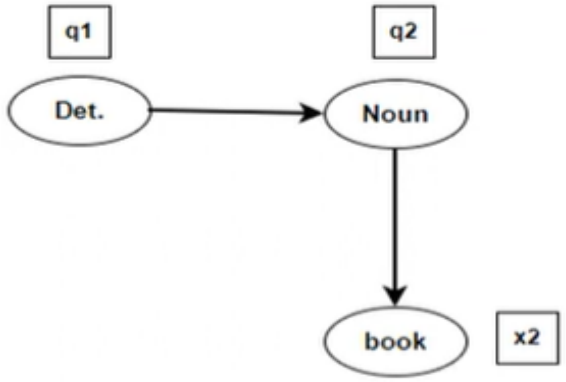
Previous Probability is 1/108

$$P(\text{Det.}|\text{a},\text{Verb}) = P(\text{a}|\text{Det.}) \times P(\text{Det.}|\text{Verb})$$

$$P(\text{Det}|\text{a},\text{Verb}) = \frac{2}{6} \times \frac{2}{4} = \frac{1}{6}$$

$$P(\text{Det.}|\text{a},\text{Verb}) = \frac{1}{6} \times \frac{1}{108} = \frac{1}{648}$$

book as Noun



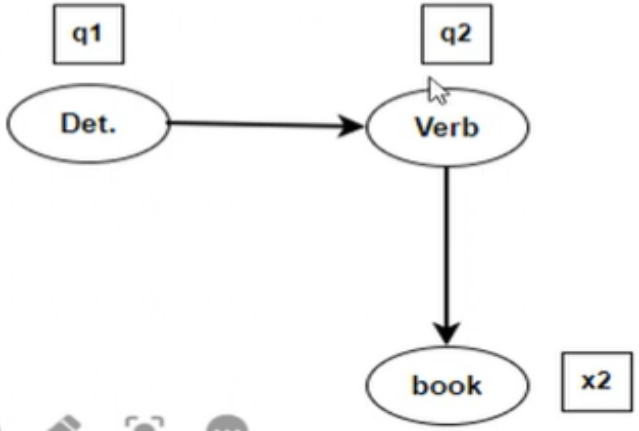
Previous Probability is 1/648

$$P(\text{Noun}|\text{book, Det.}) = P(\text{book}|\text{Noun}) \times P(\text{Noun}|\text{Det})$$

$$P(\text{Noun}|\text{book, Det.}) = \frac{1}{6} \times \frac{6}{6} = \frac{1}{6}$$

$$P(\text{Noun}|\text{book, Det.}) = \frac{1}{6} \times \frac{1}{648} = \frac{1}{3888}$$

book as Verb



$$P(\text{Noun}|\text{Verb, Det.}) = P(\text{book}|\text{Verb}) \times P(\text{Verb}|\text{Det})$$

$$P(\text{Verb}|\text{book, Det.}) = \frac{1}{4} \times 0 = 0$$

$$P(\text{Noun}|\text{book, Det.}) = \frac{1}{4} \times \frac{1}{100} = \frac{1}{400}$$

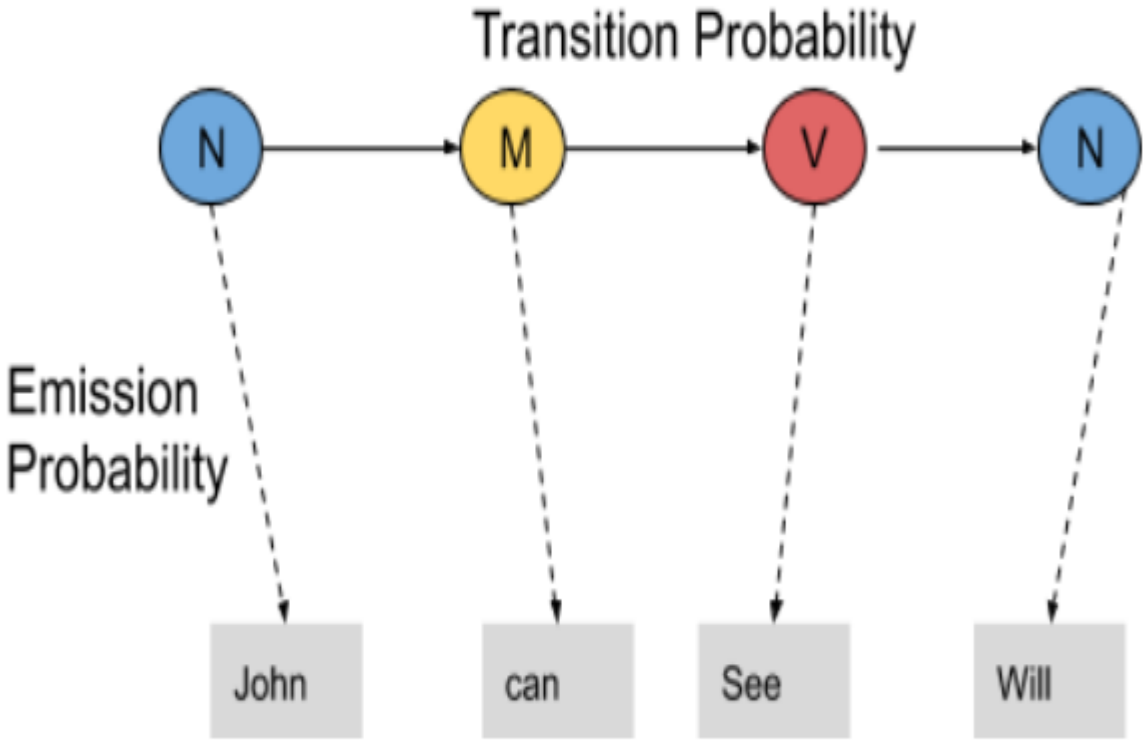
$$P(\text{Noun}|\text{book, Det.}) = \frac{1}{400} \times \frac{1}{648} = \frac{1}{259200}$$

$$\frac{1}{3888} > \frac{1}{259200}$$ **Result: book as Noun**

Result of POS tagging after applying HMM model

<S>	The	park	is	a	book	</S>
	Det.	Noun	Verb	Det.	Noun	

SEQUENCE LABELING USING HMM



Calculate the above two probabilities for the set of sentences below

- Mary Jane can see Will
- Spot will see Mary
- Will Jane spot Mary?
- Mary will pat Spot

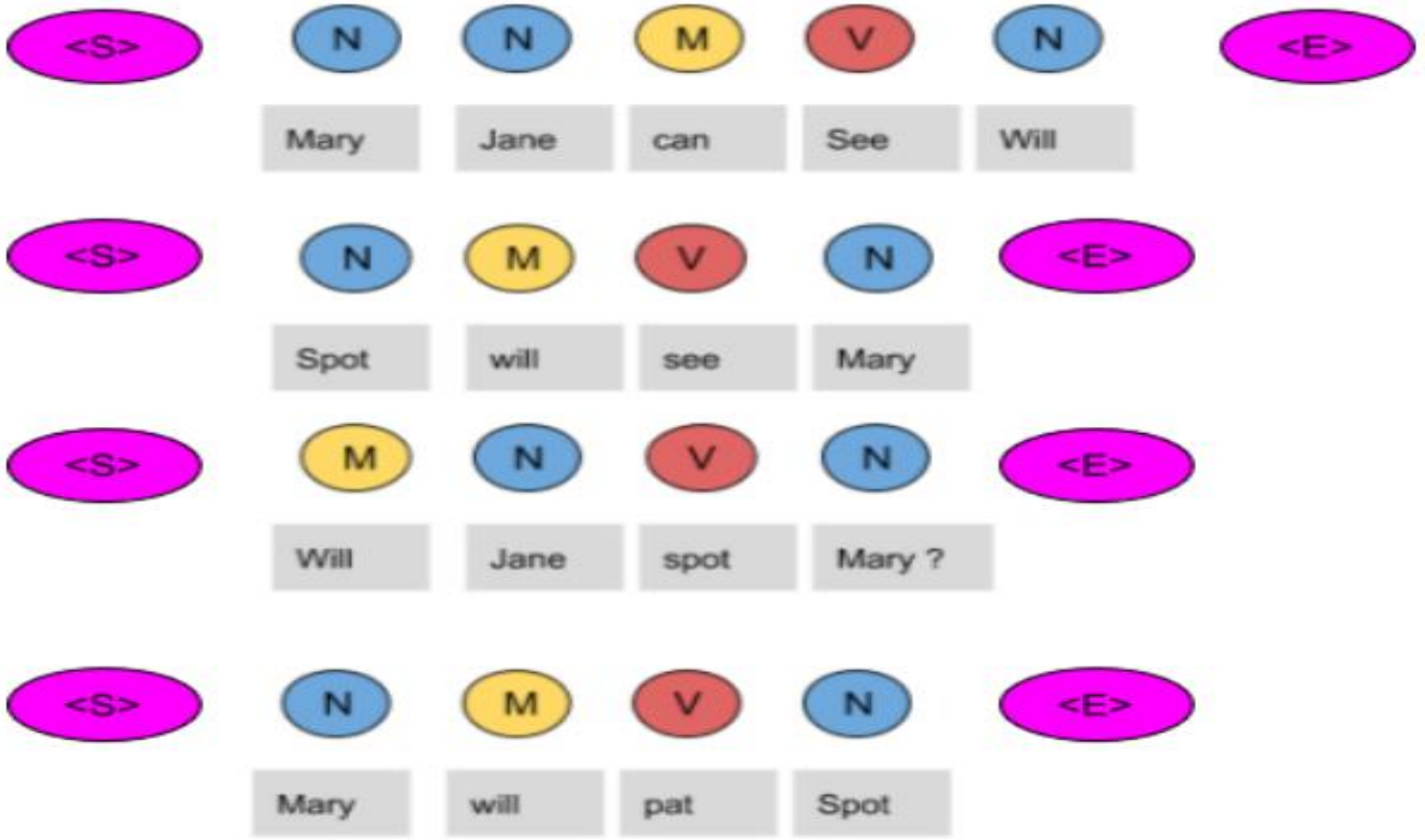


EMISSION PROBABILITY MATRIX

Words	Noun	Model	Verb
Mary	4	0	0
Jane	2	0	0
Will	1	3	0
Spot	2	0	1
Can	0	1	0
See	0	0	2
pat	0	0	1

EMISSION PROBABILITY MATRIX

Words	Noun	Model	Verb
Mary	4/9	0	0
Jane	2/9	0	0
Will	1/9	3/4	0
Spot	2/9	0	1/4
Can	0	1/4	0
See	0	0	2/4
pat	0	0	1


<S>
N
N
M
V
N
<E>
Mary
Jane
can
See
Will
<S>
N
M
V
N
<E>
Spot
will
see
Mary
<S>
M
N
V
N
<E>
Will
Jane
spot
Mary ?
<S>
N
M
V
N
<E>
Mary
will
pat
Spot

TRANSITION PROBABILITY MATRIX

	N	M	V	<E>
<S>	3	1	0	0
N	1	3	1	4
M	1	0	3	0
V	4	0	0	0

TRANSITION PROBABILITY MATRIX

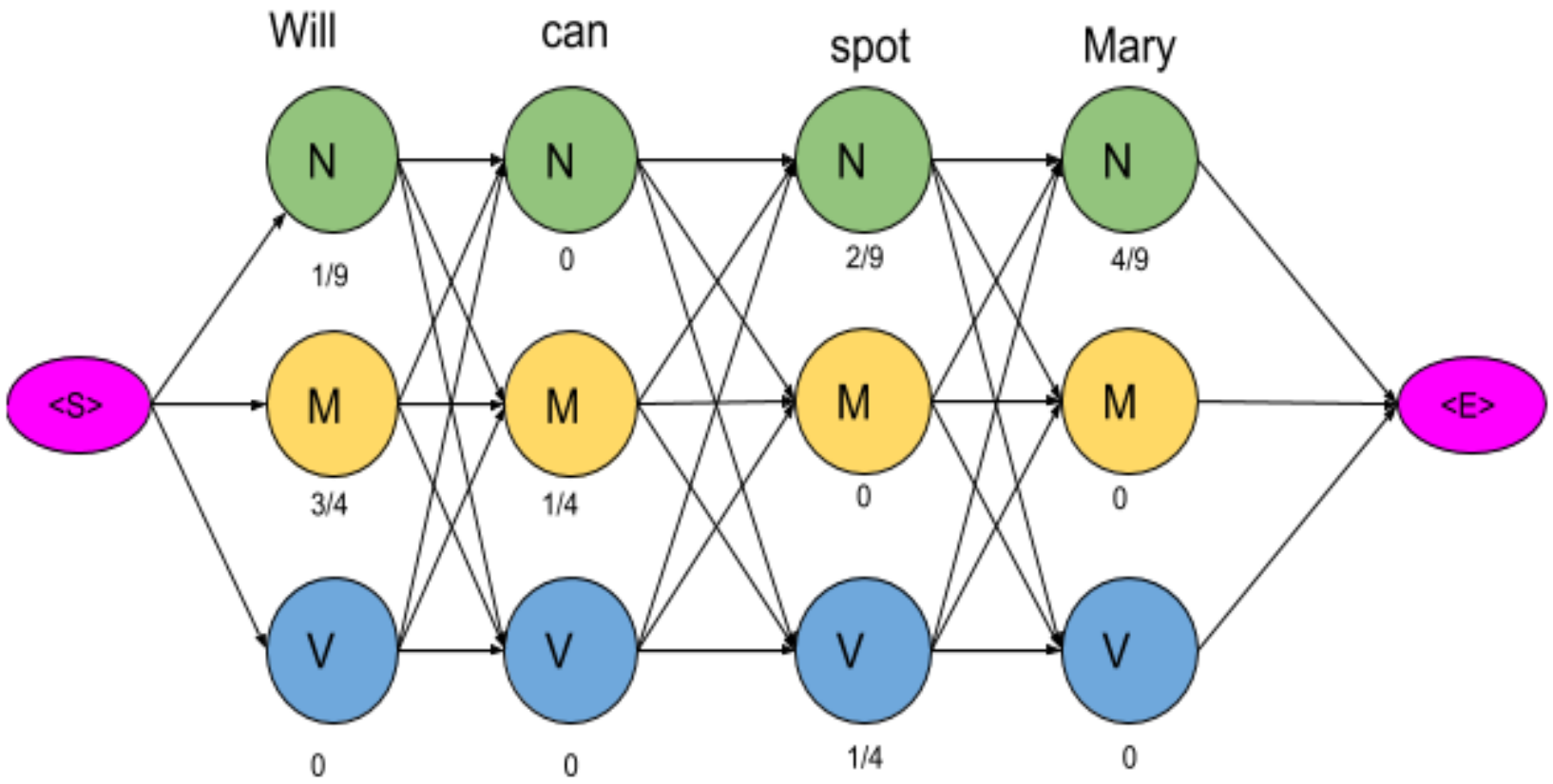
	N	M	V	‹E›
‹S›	3/4	1/4	0	0
N	1/9	3/9	1/9	4/9
M	1/4	0	3/4	0
V	4/4	0	0	0

- How does the HMM determine the appropriate sequence of tags for a particular sentence from the above tables?

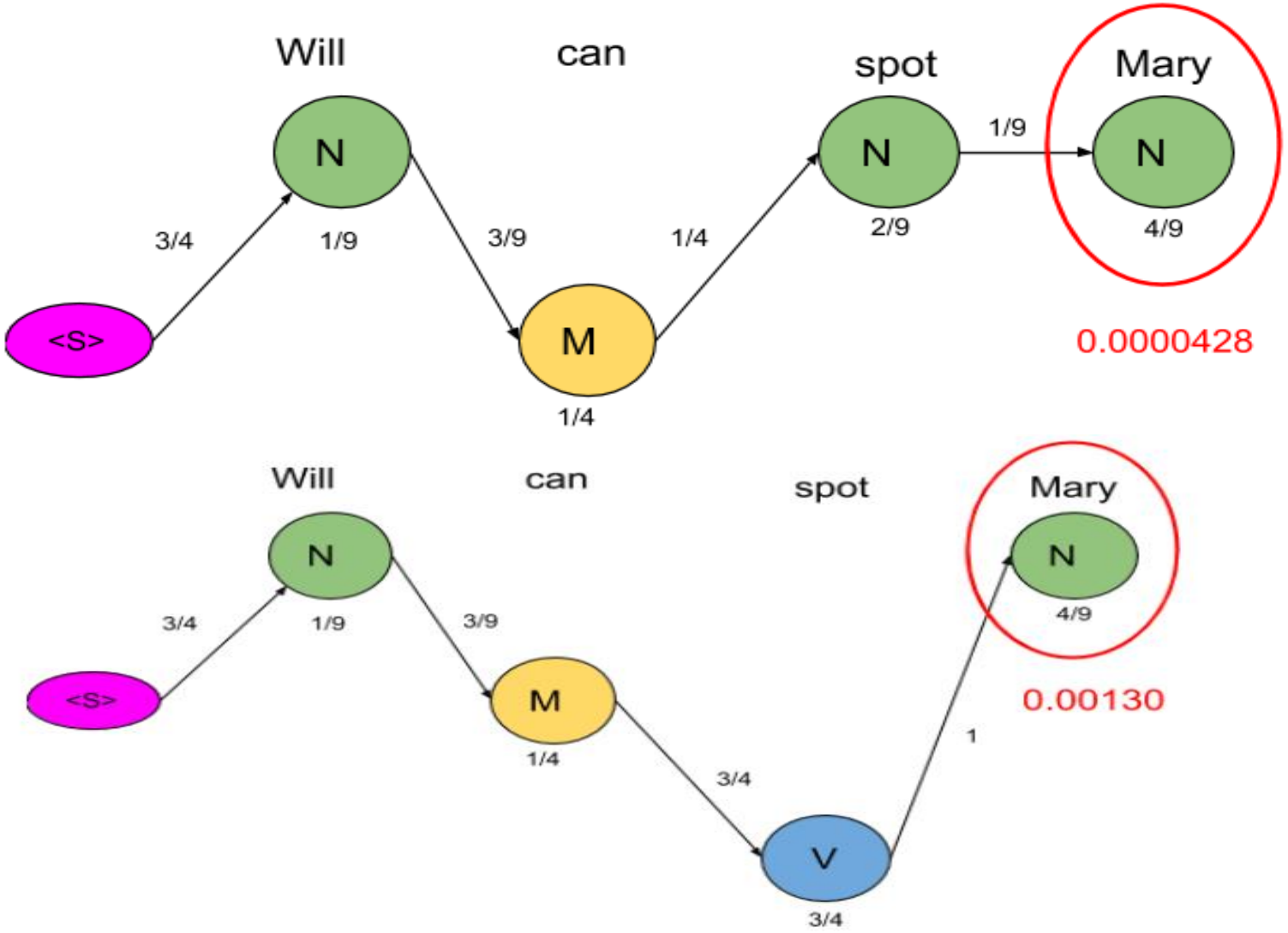
-Take a new sentence and tag them with wrong tags.

Let the sentence, ‘ Will can spot Mary’ be tagged as-

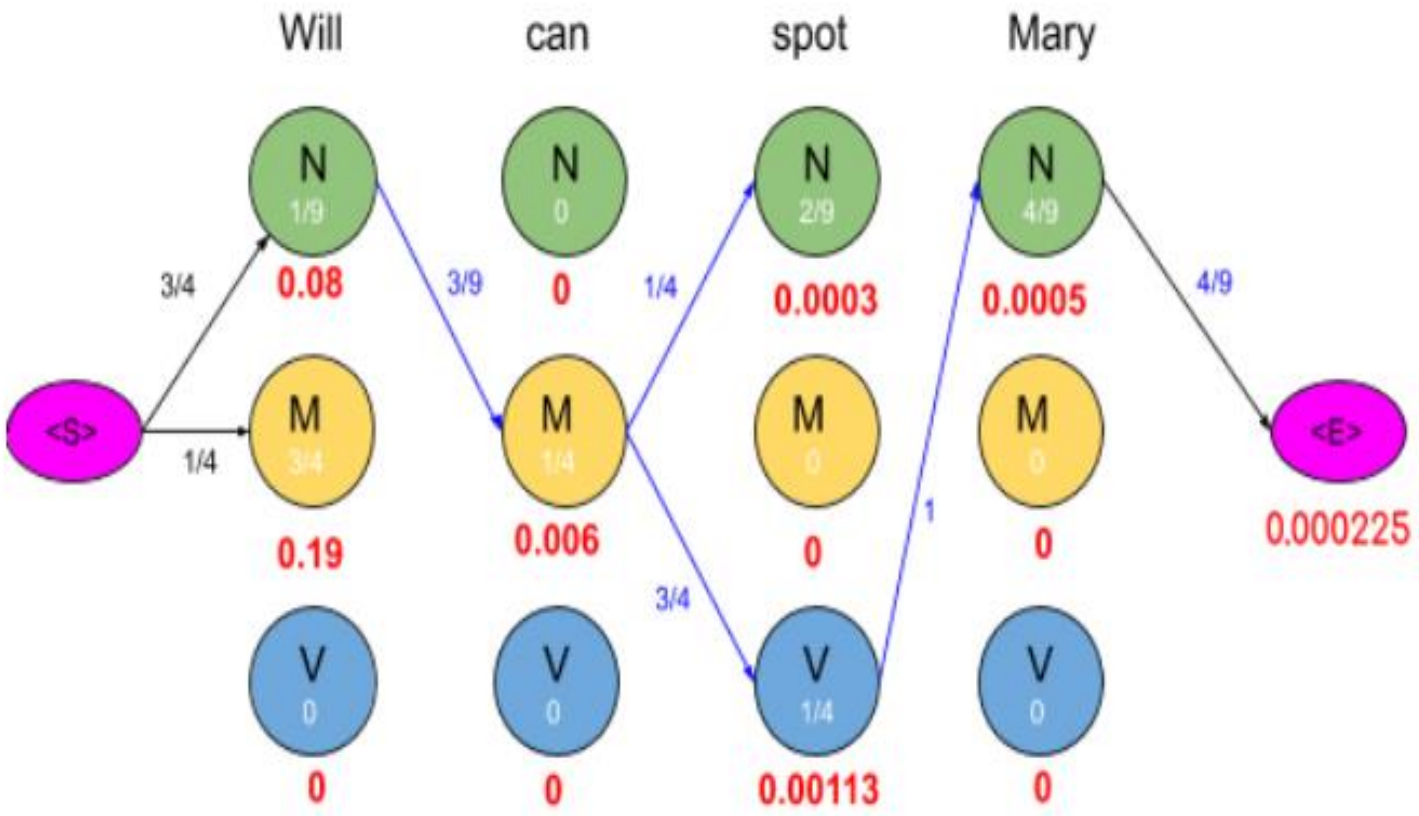
- Will as a model
- Can as a verb
- Spot as a noun
- Mary as a noun


Will
can
spot
Mary
N
N
N
N
1/9
0
2/9
4/9
<S>
M
M
M
M
<E>
3/4
1/4
0
0
V
V
V
V
0
0
1/4
0

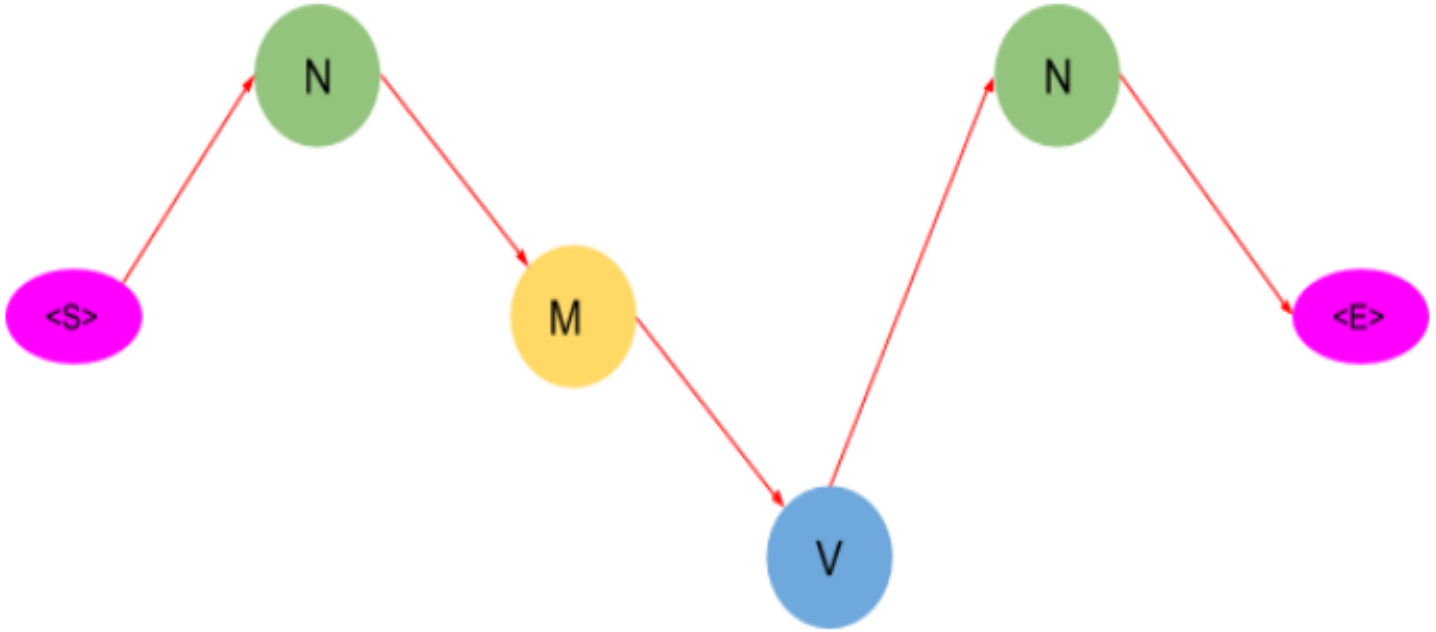
- There are two paths leading to this vertex as shown below along with the probabilities of the two mini-paths.



After Computing probabilities of all paths



- To get an optimal path, start from the end and trace backward, since each state has only one incoming edge, This gives a path as shown below



PROBLEM IN WHICH MATRIX IS GIVEN/ NOT TRAINING DATA

Step 1: Assign correct POS Tag to training corpus

Step 2: Creation of Emission Probability Matrix

	DT	NN	VB
That	0.40	0.00	0.00
Girl	0.00	0.015	0.0031
Smiles	0.00	0.0004	0.20

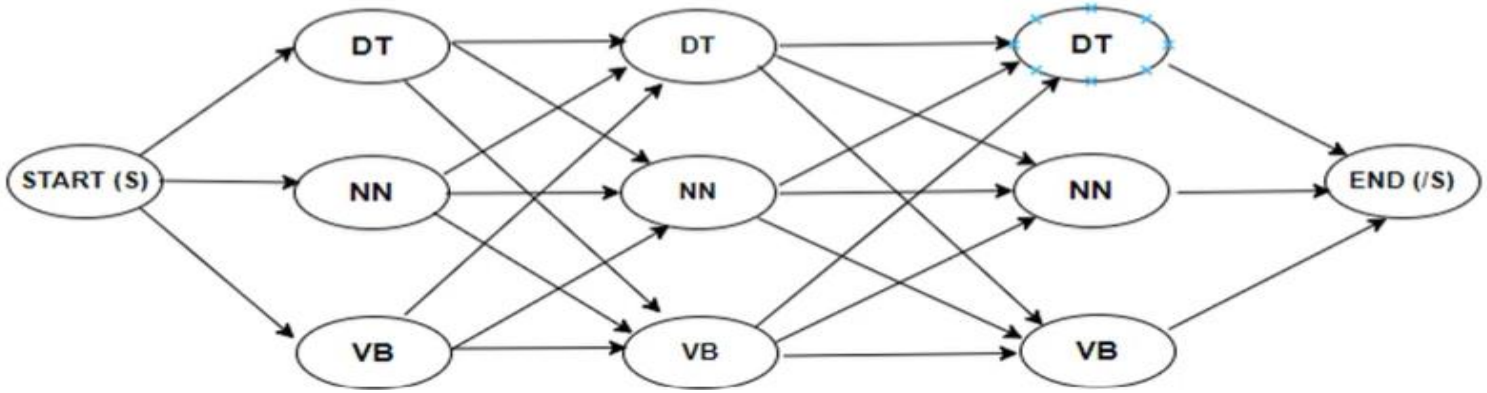
Step 3: Creation of State Transition Probability Matrix

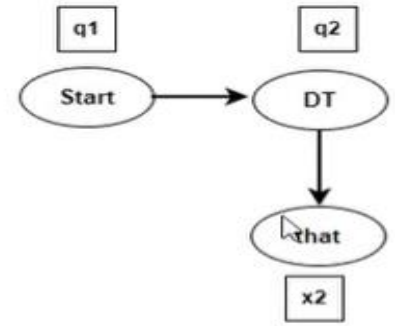
	DT	NN	VB
<S>	0.50	0.40	0.1
DT	0.01	0.99	0.00
NN	0.30	0.30	0.40
VB	0.40	0.40	0.20

$$P(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

Test Data <S> That girl smiles </S>

That	Girl	Smiles
DT	DT	DT
NN	NN	NN
VB	VB	VB
3×3×3=27 ways		

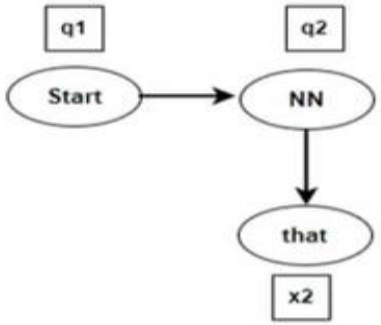




$$P(q2|x2, q1) = P(x2|q2) \times P(q2|q1)$$

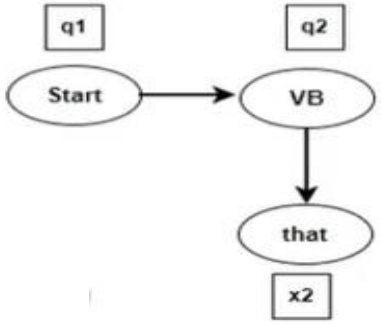
$$P(DT|that, <S>) = P(that|DT) \times P(DT|<S>)$$

$$P(DT|that, <S>) = 0.40 \times 0.50 = 0.2$$



$$P(NN|that, <S>) = P(that|NN) \times P(NN|<S>)$$

$$P(NN|that, <S>) = 0.00 \times 0.40 = 0$$

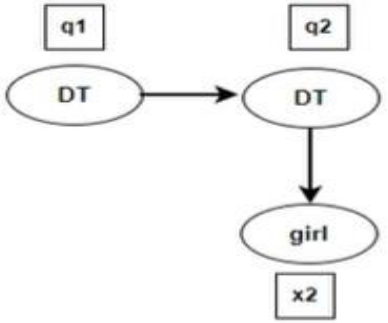


$$P(VB|that, <S>) = P(that|VB) \times P(VB|<S>)$$

$$P(VB|that, <S>) = 0.00 \times 0.1 = 0$$

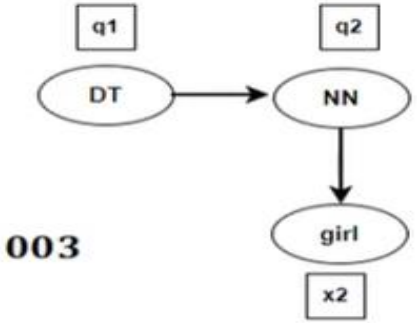
Probability of "that " word is large for DT Part of Speech Tag

Previous Probability is 0.2



$$P(DT|girl, DT) = P(girl|DT \times P(DT|DT)$$

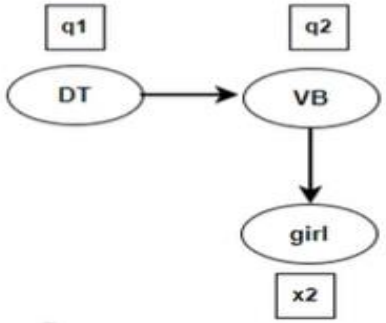
$$P(DT|girl, DT) = 0 \times 0.01 = 0$$



$$P(NN|girl, DT) = P(girl|NN) \times P(NN|DT)$$

$$P(NN|girl, DT) = 0.015 \times 0.99 = 0.01485$$

$$= 0.01458 \times 0.2 = 0.00297 \approx 0.003$$

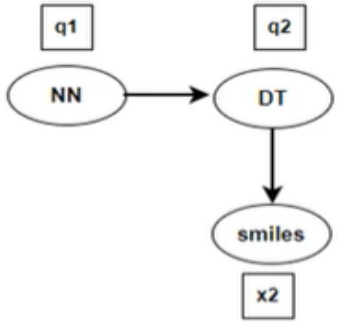


$$P(VB|girl, DT) = P(girl|VB) \times P(VB|DT)$$

$$P(VB|girl, DT) = 0.0031 \times 0.00 = 0$$

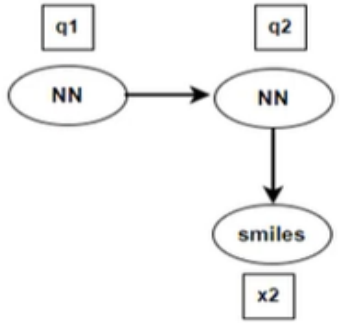
Result: Girl is noun

Previous Probability is 0.003



$$P(DT|smiles, NN) = P(smiles|DT) \times P(DT|NN)$$

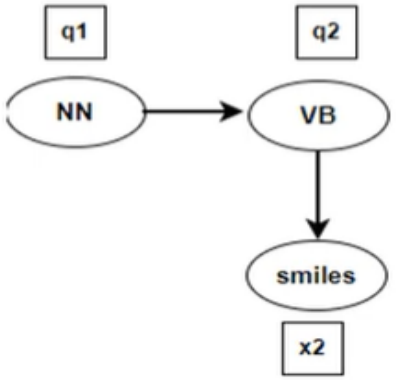
$$P(DT|smiles, NN) = 0 \times 0.30 = 0$$



$$P(NN|smiles, NN) = P(smiles|NN) \times P(NN|NN)$$

$$P(NN|smiles, NN) = 0.0004 \times 0.30 = 0.00012$$

$$= 0.00012 \times 0.003 = 0.00000036$$



$$P(VB|smiles, NN) = P(smiles|VB) \times P(VB|NN)$$

$$P(VB|smiles, NN) = 0.2 \times 0.4 = 0.08$$

$$= 0.08 \times 0.003 = 0.00024$$

Result: Smiles is verb

Result of POS tagging after applying HMM model

<S>	The	girl	smiles	</S>
	Det.	Noun	Verb	

Step 1: Assign correct POS Tag to training corpus

Step 2: Creation of Emission Probability Matrix

	Time	Flies	like	an	arrow
VB	0.1	0.2	0.2	0	0
NN	0.1	0.1	0	0	0.1
IN	0	0	0.25	0	0
DT	0	0	0	0.5	0

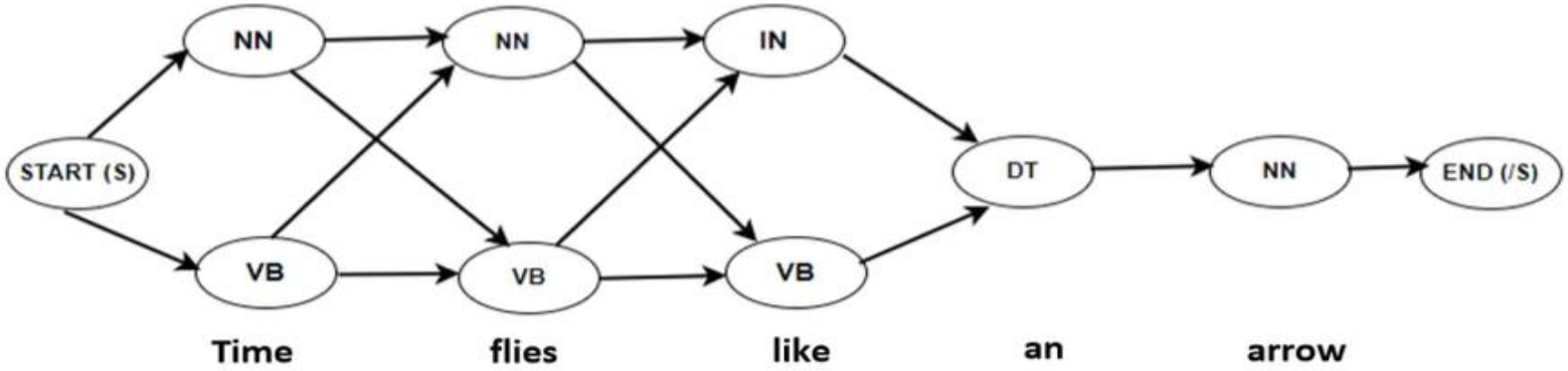
Step 3: Creation of State Transition Probability Matrix

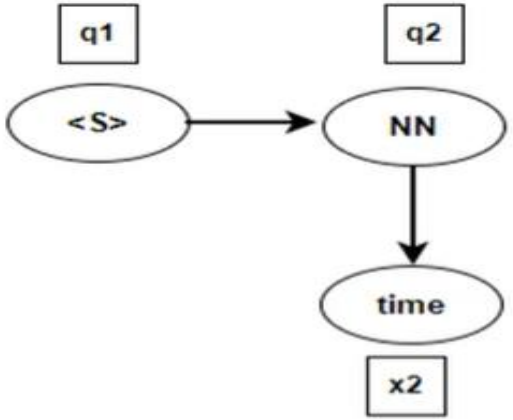
	VB	NN	IN	DT	<E>
<S>	0.2	0.8	0	0	0
VB	0	0.3	0.2	0.5	0
NN	0.4	0.5	0.1	0	0
IN	0	0.75	0	0.25	0
DT	0	1	0	0	0

$$P(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

Test Data <S> Time flies like an arraow </S>

Time	flies	like	an	arrow
NN	NN	IN	DT	NN
VB	VB	NN		
2×2×2×1×1=8 ways				

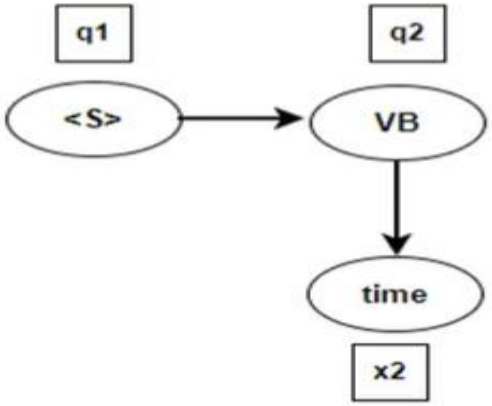




$$P(q2|x2, q1) = P(x2|q2) \times P(q2|q1)$$

$$P(NN|time, <S>) = P(time|NN) \times P(NN|<S>)$$

$$P(NN|time, <S>) = 0.1 \times 0.8 = 0.08$$



$$P(VB|time, <S>) = P(time|VB) \times P(VB|<S>)$$

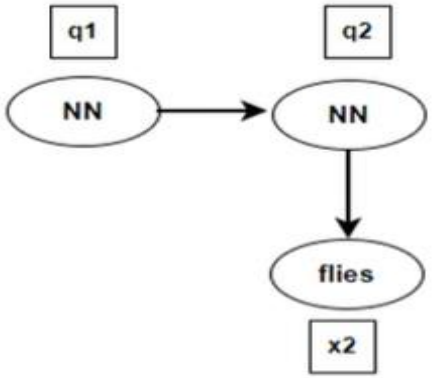
$$P(VB|time, <S>) = 0.1 \times 0.2 = 0.02$$

$$0.08 > 0.02$$

- Probability of "time " word is large for NN Part of Speech Tag

Dr. Varsha Patil

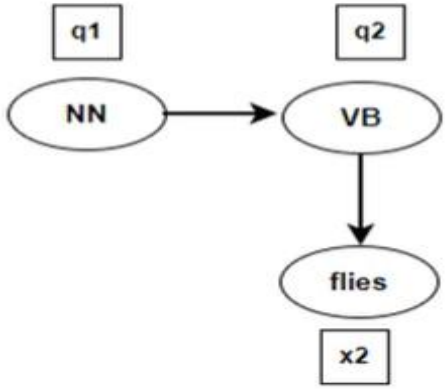
Previous Probability is 0.08



$$P(NN|flies, NN) = P(flies|NN) \times P(NN|NN)$$

$$P(NN|flies, NN) = 0.1 \times 0.5 = 0.05$$

$$= 0.05 \times 0.08 = 0.0040$$



$$P(VB|flies, NN) = P(flies|VB) \times P(VB|NN)$$

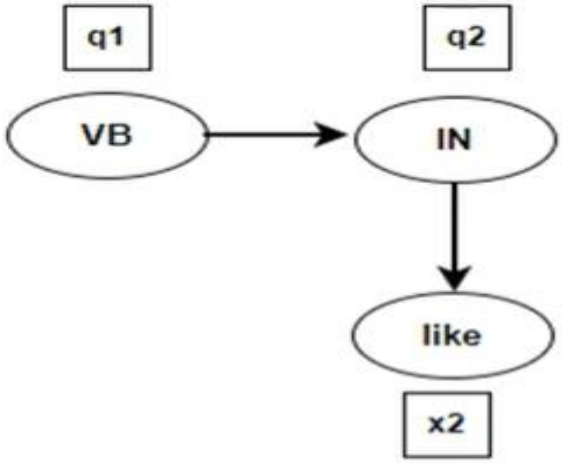
$$P(VB|flies, NN) = 0.2 \times 0.4 = 0.08$$

$$= 0.08 \times 0.08 = 0.0064$$

$$0.0064 > 0.0040$$

Result: flies is verb

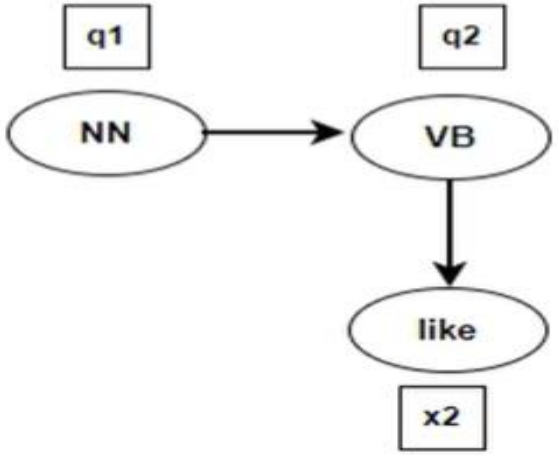
Previous Probability is 0.0064



$$P(In|like, VB) = P(like|In) \times P(In|VB)$$

$$P(IN|like, VB) = 0.25 \times 0.2 = 0.05$$

$$= 0.05 \times 0.0064 = 0.00032$$



$$P(VB|like, VB) = P(like|VB) \times P(VB|VB)$$

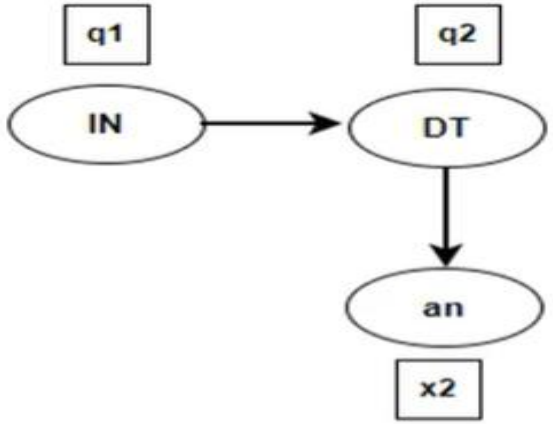
$$P(IN|like, VB) = 0.2 \times 0 = 0$$

$$= 0.2 \times 0.01 = 0.002$$

$$= 0.002 \times 0.0064 = 0.0000128$$

Result: like is preposition

Previous Probability is 0.00032



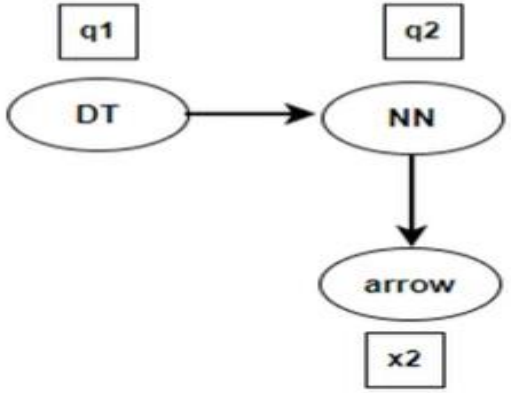
$$P(DT|a, IN) = P(a|DT) \times P(DT|IN)$$

$$P(DT|a, IN) = 0.5 \times 0.25 = 0.125$$

$$= 0.125 \times 0.00032 = 0.00004$$

Result: an is DT

Previous Probability is 0.00004



$$P(NN|arrow, DT) = P(arrow|NN) \times P(NN|DT)$$

$$= 0.1 \times 1 = 0.1$$

$$= 0.1 \times 0.00004 = 0.000004$$

Result:arrow is noun

Test Data <S> Time flies like an arraow </S>

<S>	Time	flies	like	an	arrow	</S>
	NN	VB	IN	DT	NN	

HIDDEN MARKOV MODEL

$Q = q_1 q_2 \ldots q_N$	a set of N **states**
$A = a_{11} a_{12} \ldots a_{n1} \ldots a_{nn}$	a **transition probability matrix** A, each a_{ij} representing the probability of moving from state i to state j, s.t. $\sum_{j=1}^{n} a_{ij} = 1 \quad \forall i$
$O = o_1 o_2 \ldots o_T$	a sequence of T **observations**, each one drawn from a vocabulary $V = v_1, v_2, \ldots, v_V$.
$B = b_i(o_t)$	a sequence of **observation likelihoods:**, also called **emission probabilities**, each expressing the probability of an observation o_t being generated from a state i.
q_0, q_F	a special **start state** and **end (final) state** which are not associated with observations, together with transition probabilities $a_{01} a_{02} .. a_{0n}$ out of the start state and $a_{1F} a_{2F} \ldots a_{nF}$ into the end state.

$\pi = \pi_1, \pi_2, \ldots, \pi_N$	an **initial probability distribution** over states. π_i is the probability that the Markov chain will start in state i. Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^{n} \pi_i = 1$
$QA = \{q_x, q_y \ldots\}$	a set $QA \subset Q$ of legal **accepting states**

A first-order Hidden Markov Model instantiates two simplifying assumptions. First, as with a first-orderMarkov chain, the probability of a particular state is dependent only on the previous state:

Markov Assumption: $P(q_i \mid q_1 .. q_{i-1}) = P(q_i \mid q_{i-1})$

Second, the probability of an output observation o_i is dependent only on the state that produced the observation q_i, and not on any other states or any other observations:

Output Independence: $P(o_i \mid q_1 \ldots q_i, \ldots, q_T, o_1, \ldots, o_i, \ldots, o_T) = P(o_i \mid q_i)$

Fig. 6.3 shows a sample HMM for the ice cream task. The two hidden states (H and C) correspond to hot and cold weather, while the observations (drawn from the alphabet $O = \{1, 2, 3\}$) correspond to the number of ice creams eaten by Jason on a given day.

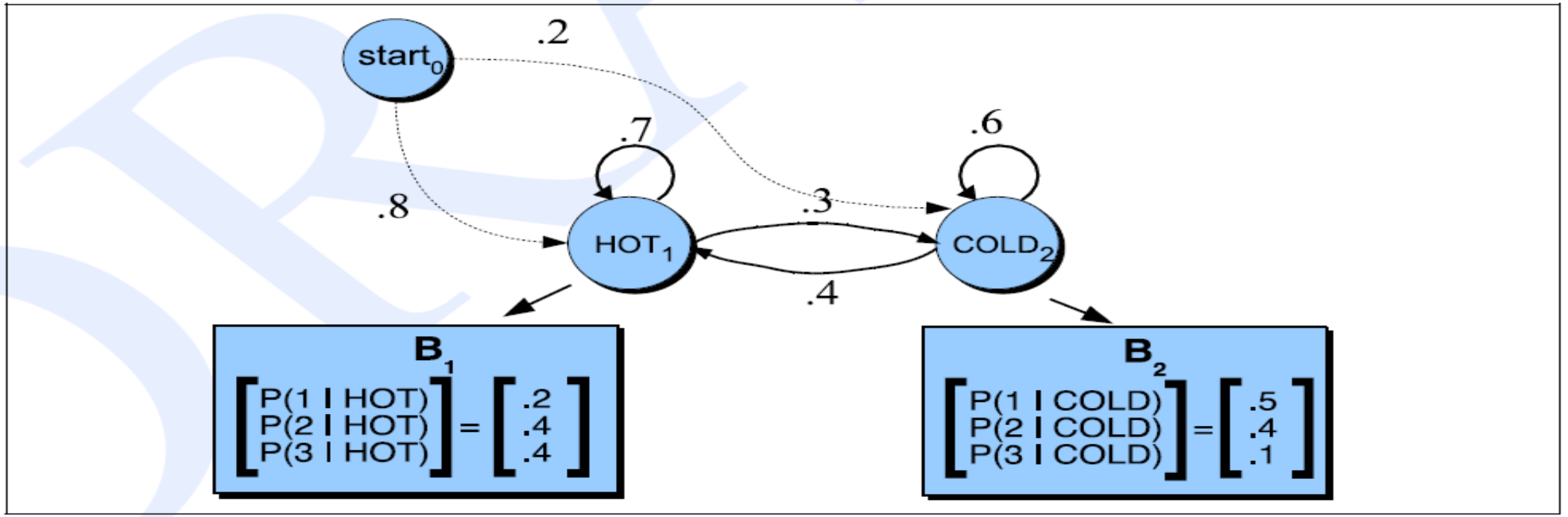


Figure 6.3 A Hidden Markov Model for relating numbers of ice creams eaten by Jason (the observations) to the weather (H or C, the hidden variables). For this example we are not using an end-state, instead allowing both states 1 and 2 to be a final (accepting) state.

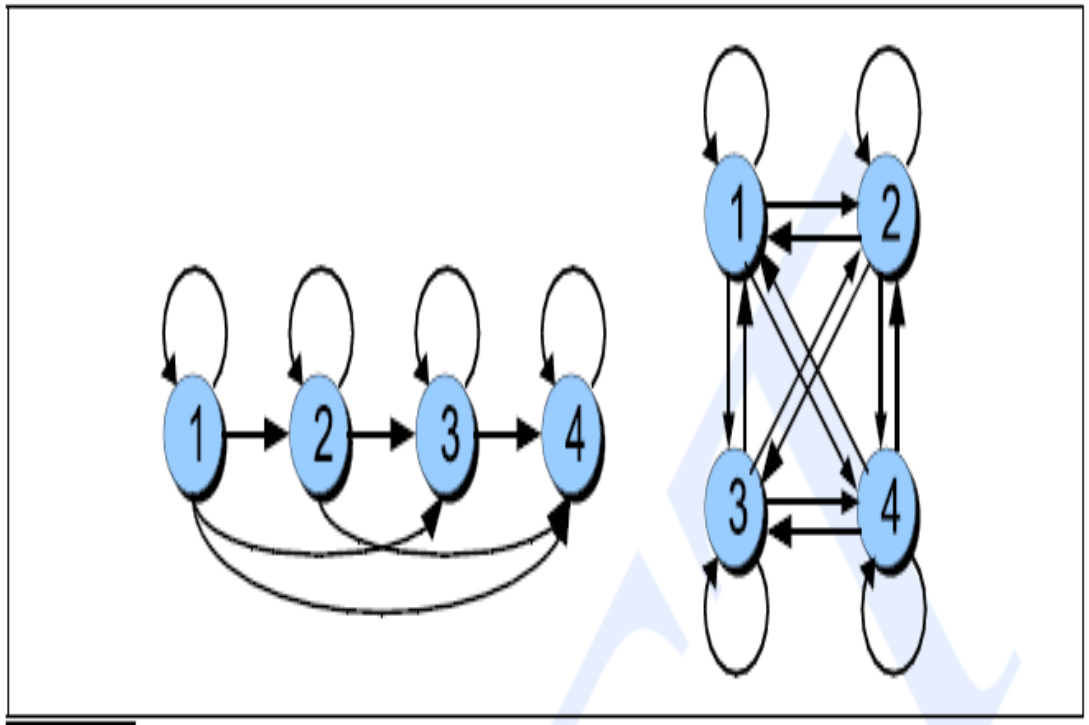


Figure 6.4 Two 4-state Hidden Markov Models; a left-to-right (Bakis) HMM on the left, and a fully-connected (ergodic) HMM on the right. In the Bakis model, all transitions not shown have zero probability.

Now that we have seen the structure of an HMM, we turn to algorithms for computing things with them. An influential tutorial by Rabiner (1989), based on tutorials by Jack Ferguson in the 1960s, introduced the idea that Hidden Markov Models should be characterized by **three fundamental problems**:

Problem 1 (Likelihood):	Given an HMM $\lambda = (A,B)$ and an observation sequence O, determine the likelihood $P(O\|\lambda)$.
Problem 2 (Decoding):	Given an observation sequence O and an HMM $\lambda = (A,B)$, discover the best hidden state sequence Q.
Problem 3 (Learning):	Given an observation sequence O and the set of states in the HMM, learn the HMM parameters A and B.

(6.9) $$P(3\ 1\ 3|\text{hot hot cold}) = P(3|\text{hot}) \times P(1|\text{hot}) \times P(3|\text{cold})$$

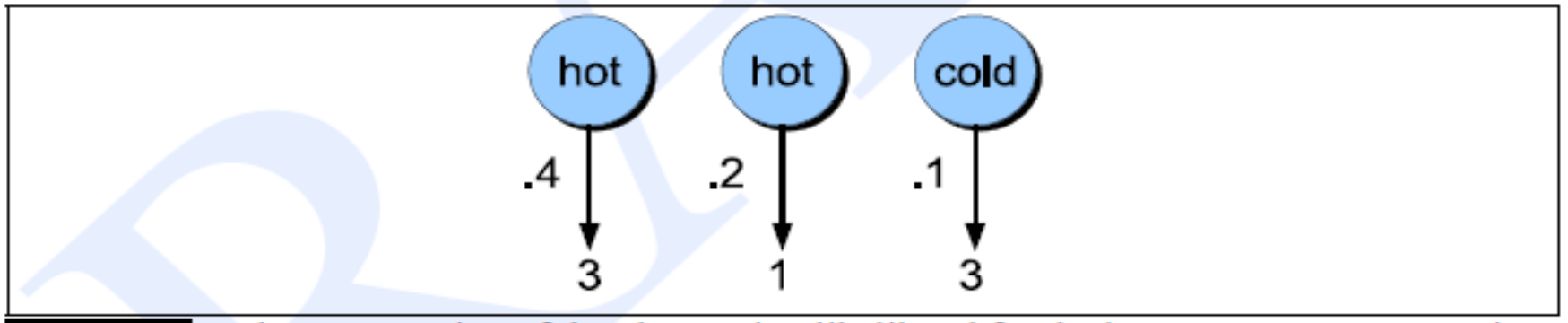


Figure 6.5 The computation of the observation likelihood for the ice-cream events *3 1 3* given the hidden state sequence *hot hot cold*.

(6.11) $$\begin{aligned} P(3\ 1\ 3, \text{hot hot cold}) = {} & P(\text{hot}|\text{start}) \times P(\text{hot}|\text{hot}) \times P(\text{cold}|\text{hot}) \\ & \times P(3|\text{hot}) \times P(1|\text{hot}) \times P(3|\text{cold}) \end{aligned}$$

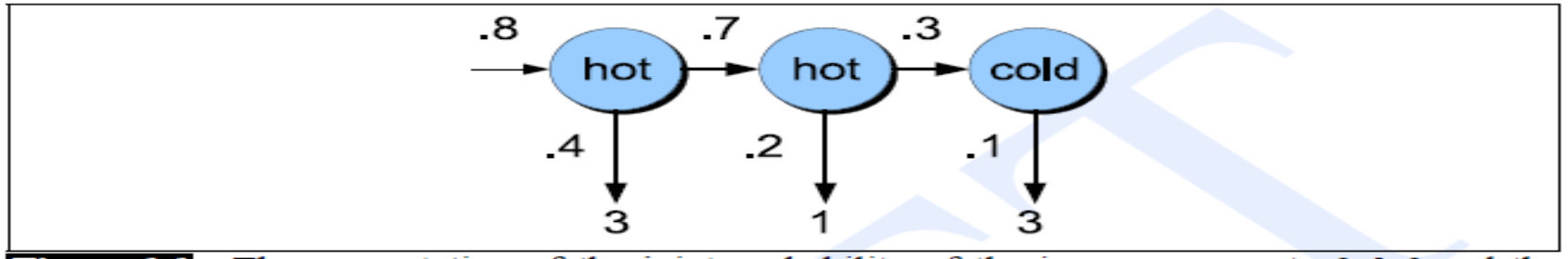


Figure 6.6 The computation of the joint probability of the ice-cream events *3 1 3* and the hidden state sequence *hot hot cold*.

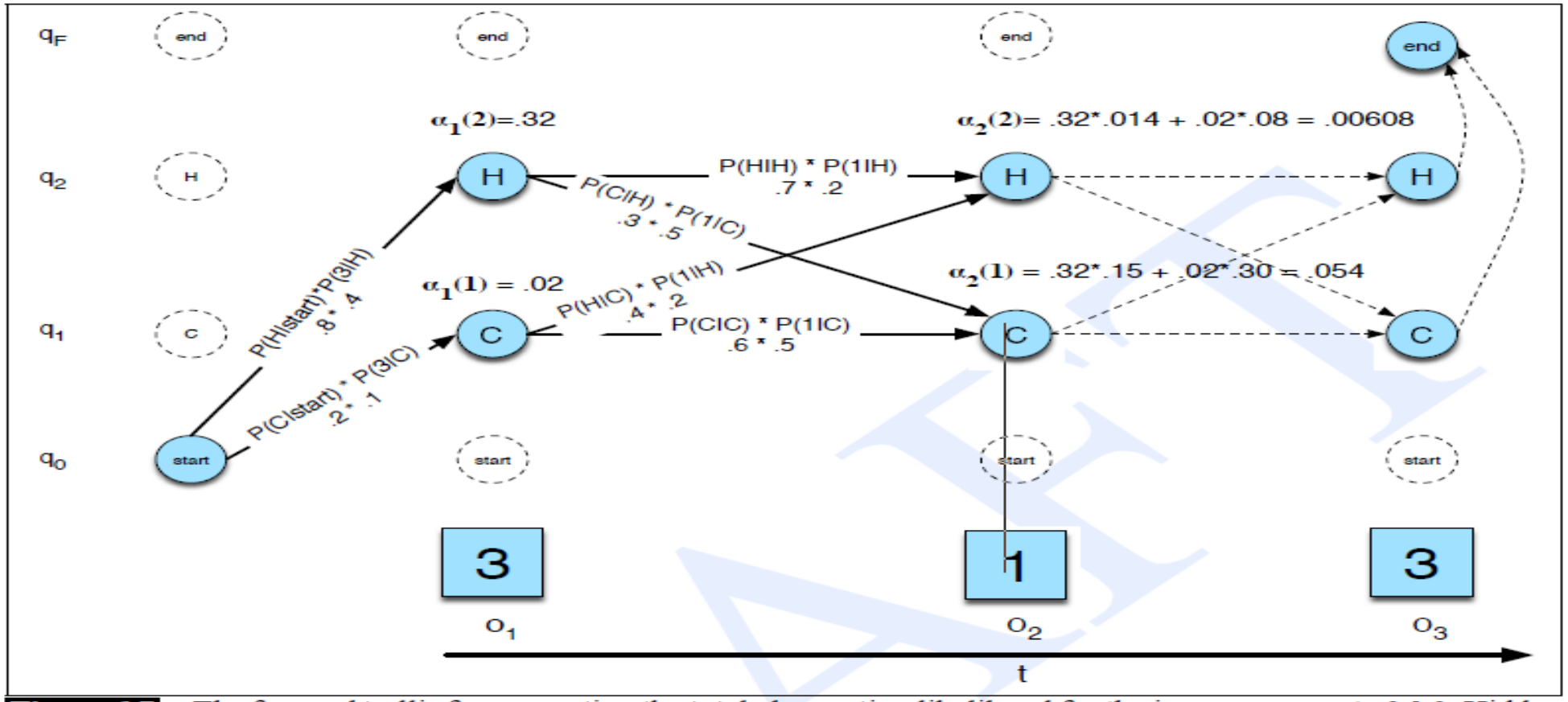


Figure 6.7 The forward trellis for computing the total observation likelihood for the ice-cream events *3 1 3*. Hidden states are in circles, observations in squares. White (unfilled) circles indicate illegal transitions. The figure shows the computation of $\alpha_t(j)$ for two states at two time steps. The computation in each cell follows Eq. 6.15: $\alpha_t(j) = \sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} b_j(o_t)$. The resulting probability expressed in each cell is Eq. 6.14: $\alpha_t(j) = P(o_1, o_2 \ldots o_t, q_t = j|\lambda)$.

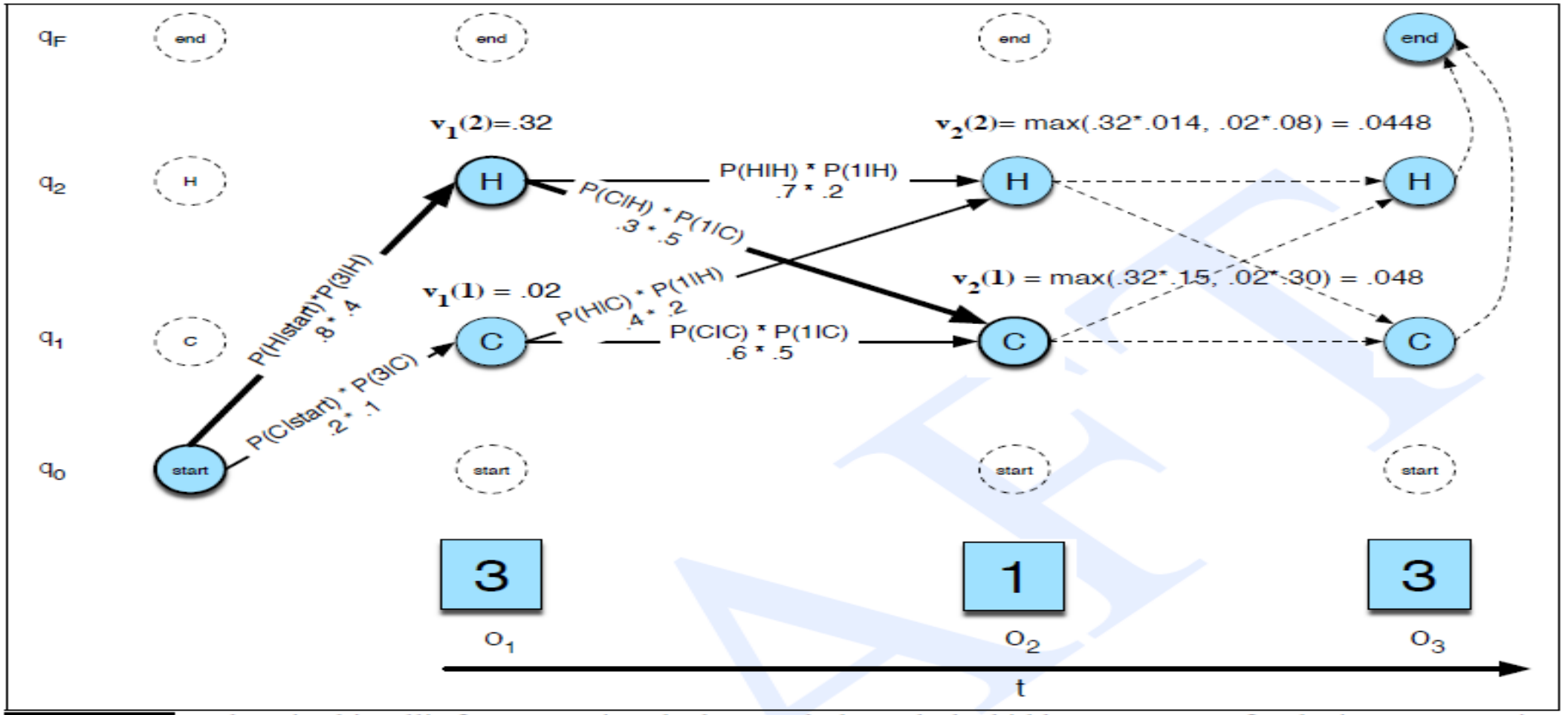


Figure 6.10 The Viterbi trellis for computing the best path through the hidden state space for the ice-cream eating events *3 1 3*. Hidden states are in circles, observations in squares. White (unfilled) circles indicate illegal transitions. The figure shows the computation of $v_t(j)$ for two states at two time steps. The computation in each cell follows Eq. 6.20: $v_t(j) = \max_{1 \le i \le N-1} v_{t-1}(i)\, a_{ij}\, b_j(o_t)$ The resulting probability expressed in each cell is Eq. 6.19: $v_t(j) = P(q_0, q_1, \ldots, q_{t-1}, o_1, o_2, \ldots, o_t, q_t = j|\lambda)$.

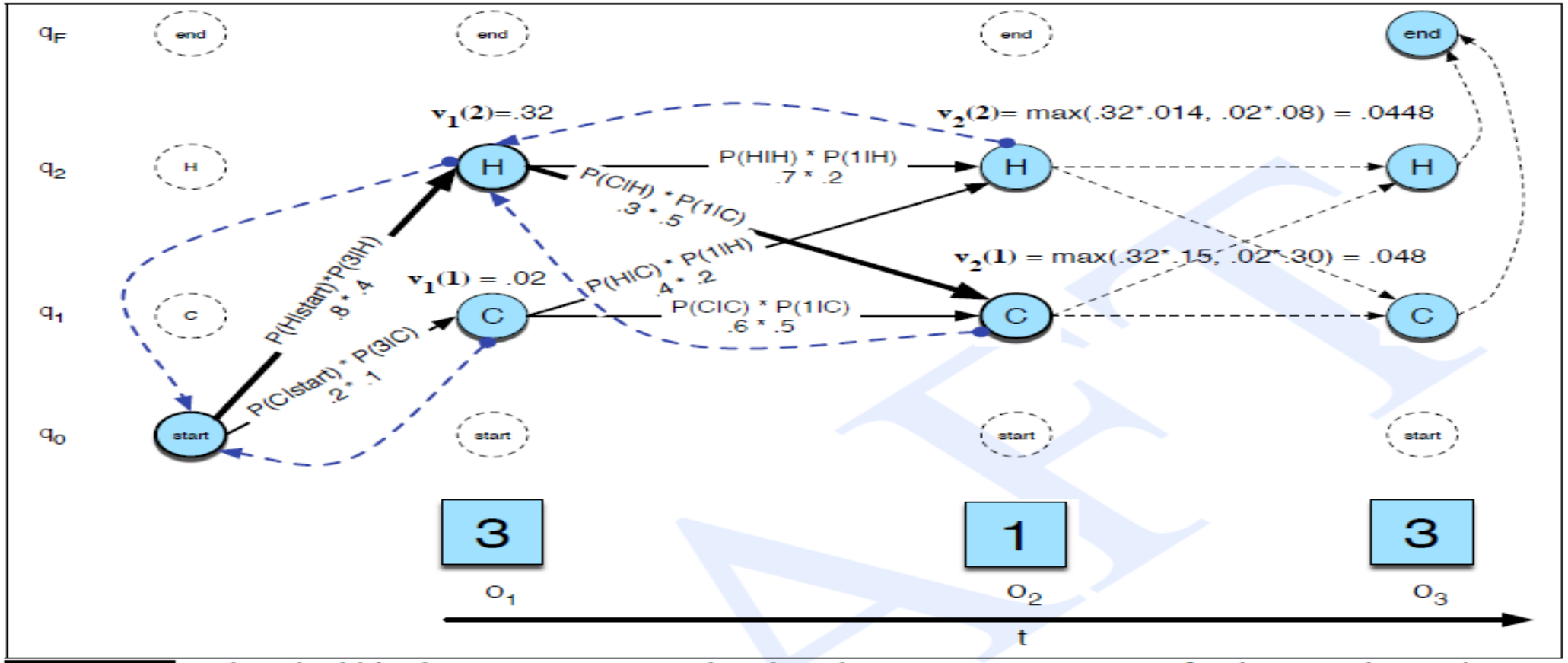


Figure 6.12 The Viterbi backtrace. As we extend each path to a new state account for the next observation, we keep a backpointer (shown with broken blue lines) to the best path that led us to this state.

function VITERBI(*observations* of len T, *state-graph* of len N) **returns** *best-path*

create a path probability matrix $viterbi[N+2,T]$
for each state s **from** 1 **to** N **do** ;initialization step
$viterbi[s,1] \leftarrow a_{0,s} * b_s(o_1)$
$backpointer[s,1] \leftarrow 0$
for each time step t **from** 2 **to** T **do** ;recursion step
for each state s **from** 1 **to** N **do**
$viterbi[s,t] \leftarrow \max_{s'=1}^{N} viterbi[s',t-1] * a_{s',s} * b_s(o_t)$
$backpointer[s,t] \leftarrow \operatorname{argmax}_{s'=1}^{N} viterbi[s',t-1] * a_{s',s}$
$viterbi[q_F,T] \leftarrow \max_{s=1}^{N} viterbi[s,T] * a_{s,q_F}$; termination step
$backpointer[q_F,T] \leftarrow \operatorname{argmax}_{s=1}^{N} viterbi[s,T] * a_{s,q_F}$; termination step
return the backtrace path by following backpointers to states back in time from $backpointer[q_F,T]$

Figure 6.11 Viterbi algorithm for finding optimal sequence of hidden states. Given an observation sequence and an HMM $\lambda = (A,B)$, the algorithm returns the state-path through the HMM which assigns maximum likelihood to the observation sequence. Note that states 0 and q_F are non-emitting.

Exercise:

Emission Probabilities:

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

Transition Probabilities:

	NNP	MD	VB	JJ	NN	RB	DT
<s>	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

Test Sample: `Janet will back the bill`

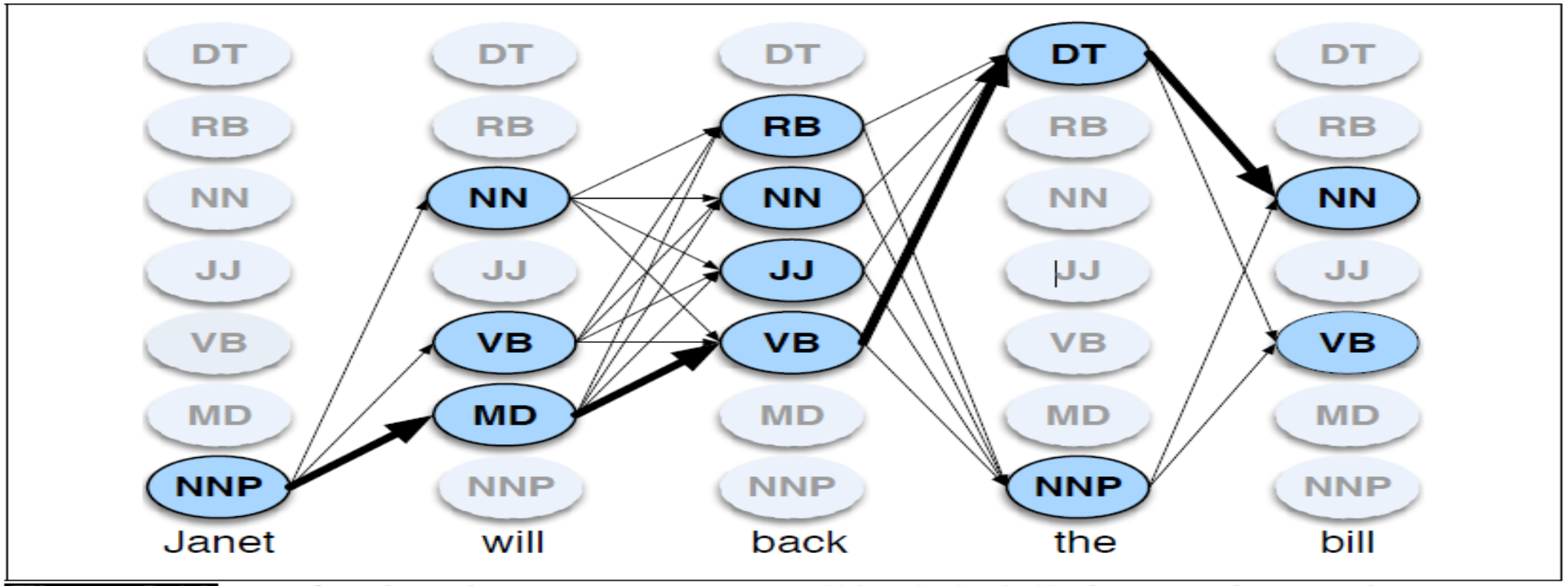


Figure 8.11 A sketch of the lattice for *Janet will back the bill*, showing the possible tags (q_i) for each word and highlighting the path corresponding to the correct tag sequence through the hidden states. States (parts of speech) which have a zero probability of generating a particular word according to the B matrix (such as the probability that a determiner DT will be realized as *Janet*) are greyed out.

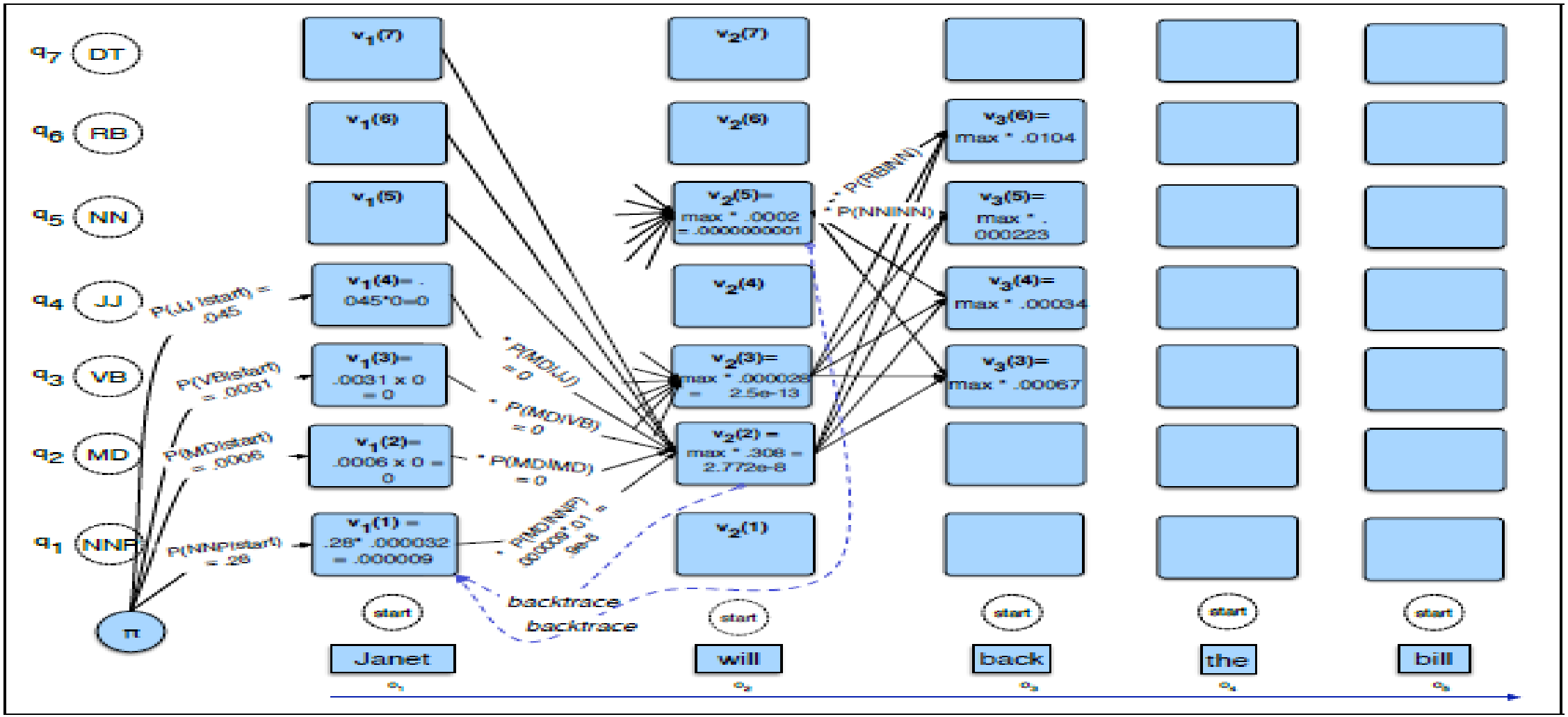


Figure 8.14 The first few entries in the individual state columns for the Viterbi algorithm. Each cell keeps the probability of the best path so far and a pointer to the previous cell along that path. We have only filled out columns 1 and 2; to avoid clutter most cells with value 0 are left empty. The rest is left as an exercise for the reader. After the cells are filled in, backtracing from the *end* state, we should be able to reconstruct the correct state sequence NNP MD VB DT NN.

Apply Viterbi Algorithm to assign POS in effiecient way

Janet	will	back	the	bill
NNP	NN	RB	DT	NN
	VB	NN	NNP	VB
	MD	JJ		
		VB		
1×3×4×2×2=48 ways				

Limitations of HMM POS Tagging

- The emission probabilities and state transition probabilities of the corpus are explicitly used in the HMM model.
- That is, it worked with a dataset that was labelled (i.e. whose emission and state transition probabilities known)
- Baum Welch Algorithm is one way to tackle the challenge of working HMM POS tagging for unlabeled data sets.
- The EM algorithm, which uses a forward and backward approach to get the maximum likelihood estimate of the parameters of a hidden markov model.

Issues of HMM

1) Unknown Words:

- We might get some unknown words at runtime that aren't in the corpus.
- That word has no emission probability
- As a result, Viterbi decoding is not possible.
- We can use morphological signals to determine whether a word is a proper noun if the first character is capital.
- If the word has the suffix 'ed,' it could be past tense or past participle.

2. Limited Context:

- We use a first-order Markov model in Viterbi decoding, which acts as a bigram in state transitions.
- However, as seen in the example below, sometimes preceding background is insufficient.

e.g. correctly idetified by HMM (First order Markov]

is passed [Verb, Past Participle]

she passed [Verb, Past Tense]

is **absolutely** passed

she **absolutely** passed

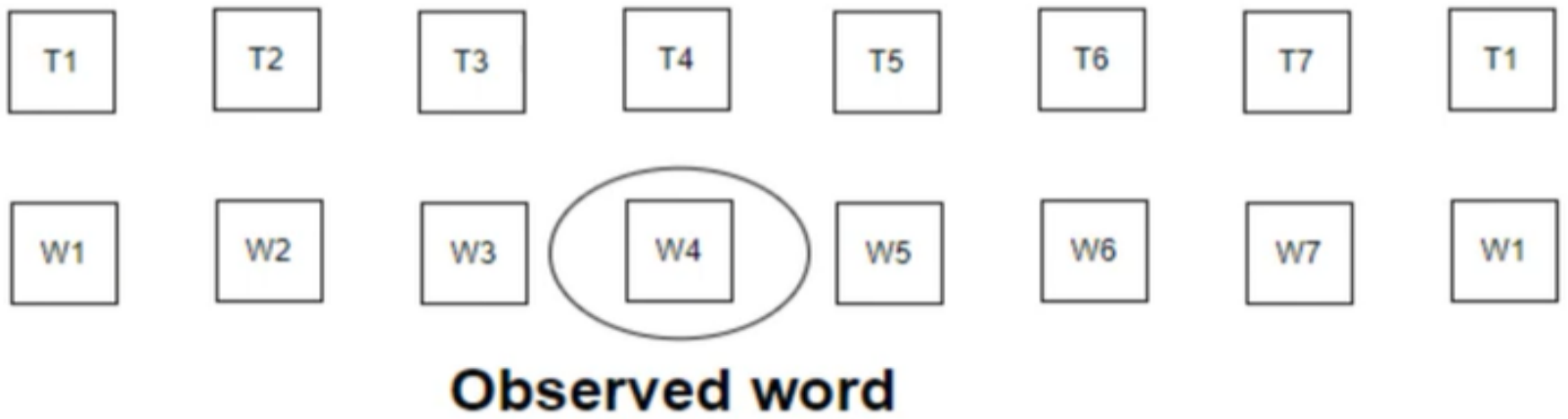
Solution: To determine the correct POS tag, use higher order n-grams to assess the POS of previous words.

3. For big sentences, go slowly.

4. Word features are limited.

Use a feature-rich classifier as a solution.

Maximum Entropy Model (MaxEnt)



Maximum Entropy Model (MEM or MaxEnt)

- For POS disambiguation, HMM POS tagging uses restricted features.
- Both the tag and the observed word in HMM are dependent on the previous tag.
- To establish the POS tag for an observed word, further features must be considered.
- A discriminatory model is the Maximum Entropy Model.
- It does POS tagging for a given sentence by combining several heterogeneous features in a probabilistic framework.

- The MaxEnt is based on the Principle of Maximum Entropy, which states that of all models that match training data, the one with the highest entropy should be chosen.
- When the conditional independence of the features cannot be assumed, the Maximum Entropy classifier is utilised.
- This means that features are interdependent.
- This is especially evident in problems involving text categorization, because the characteristics are words and they are not independent.

- Maximum Entropy Modeling (MaxEnt) is a machine learning framework.
- Classification is the problem of taking a single observation, extracting some useful features describing the observation, and then classifying the observation into one of a set of discrete classes depending on these features.
- Probabilistic classifier: Indicates the likelihood that the observation belongs to that category.
- Non-sequential categorization — When it comes to text classification, we may need to evaluate whether or not an email should be categorised as spam. We must assess whether a statement or document represents a favourable or negative attitude in sentiment analysis. A period character (V) must be classified as a sentence boundary or not.

- MaxEnt belongs to the family of classifiers known as the exponential or log-linear classifiers.
- MaxEnt works by extracting some set of features from the input, combining them linearly (multiply each feature by a weight and then add them up), and then using this sum as an exponent.
- Example: tagging — A feature for tagging might be this word ends in ed or the previous word was adverb

$$p(c|x) = \frac{1}{Z} exp(\sum_i w_i f_i)$$

Applications:

- MaxEnt classification is one of the classical machine learning tasks.
- Sentiment analysis (for example, what does a product reviewer like and dislike about the product?)
- Preferences (for example, who will a person vote for based on their demographics? Would they rather have Superman, Batman, or the Teenage Mutant Ninja Turtles, for example?
- Diagnosis (e.g., what medical disease is a person at risk of getting based on the characteristics of many medical imaging and the patient's history?)

Example of Spam/Not Spam Mail Detection

Some of the features of the emails that might be useful to determine spam or not spam.

Features of Spam Mail:

F1: Look at who the email is addressed to

F2: Check the “from” email address

F3: Check the greeting

F4: Look at the Subject of the Email

F5: Your domain is forwarded to another domain

F6: Bad Grammar and Spelling

F7: No phone number in the signature

Some of these features are more inclined towards spam than the others.

We can assign weight to each features depending on the priority. Weight vary from -1 to 1.

Features	Weight	Weight (Spam)	Weight (Not Spam)
F1: Look at who the email is addressed to	W1	-0.2	0.5
F2: Check the “from” email address	W2	-0.2	0.6
F3: Check the greeting	W3	-0.3	0.7
F4: Look at the Subject of the Email	W4	0.6	-0.3
F5: Email content are money, lottery, win, prizes	W5	0.7	-0.4
F6: Bad Grammar and Spelling	W6	0.5	-0.2
F7: No phone number in the signature	W7	-0.2	0.5

Using the following approach, we can predict decision(d) [Spam/Non Spam] for the given input x. The input x has N binary features that are either 0 or 1.

1. For each feature, determine whether $f_i(x)$ is 1 or 0.
2. Multiply each feature by the weight associated with it, $w_i(d)$, i.e. $f_i(x)*w_i(d)$
3. For N features, add the total of fi(x)*wi(d).

Formula $$\sum_{i=1}^{N} f_i(x) \times w_i(d)$$

4. Take the exponent of summation.

$$\text{Numerator} = \exp\left(\sum_{i=1}^{N} f_i(x) \times w_i(d)\right)$$

If two classes are there we can obtain two numerators one for spam and other for non-spam.

5. Divide the numerator by sum of the numerators for every possible decision d (i.e.for all possible classes):

$$\text{Denominator} = \sum_{d} \exp\left(\sum_{i=1}^{N} f_i(x) \times w_i(d)\right)$$

$$p(c|x) = \frac{\text{Numerator}}{\text{Denominator}}$$

$$p(c|x) = \frac{\exp\left(\sum_{i=1}^{N} f_i(x) \times w_i(d)\right)}{\sum_{d} \exp\left(\sum_{i=1}^{N} f_i(x) \times w_i(d)\right)}$$

$$p(c|x) = \frac{1}{Z} exp\left(\sum_{i} w_i f_i\right)$$

$$p(c|x) = \frac{\exp(\sum_{i=1}^{N} f_i(x) \times w_i(d))}{\sum_d \exp(\sum_{i=1}^{N} f_i(x) \times w_i(d))}$$

$$p(c|x) = \frac{1}{Z} exp(\sum_i w_i f_i)$$

For NLP,c is POS tag and x is observed word

For Email example, c is Spam or Non Spam, x is email input

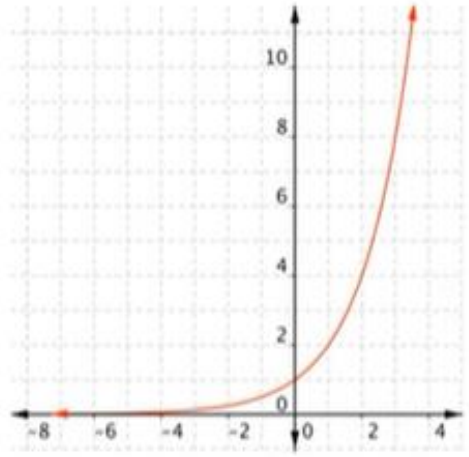
Demonimator (Z) is normalizing constant

Example: Email as input x, d as two classes Spam and Non Spam

Features	Features Present	Weight (Spam)	Weight (Not Spam)
F1: Look at who the email is addressed to	F		
F2: Check the "from" email address	F		
F3: Check the greeting	F		
F4: Look at the Subject of the Email	T	0.6	-0.3
F5: Email content are money, lottery, win, prizes	T	0.7	-0.4
F6: Bad Grammar and Spelling	T	0.5	-0.2
F7: No phone number in the signature	F		

We will calcuate two scores, one for Spam and other for Non Spam

$$p(c|x) = \frac{\exp(\sum_{i=1}^{N} f_i(x) \times w_i(d))}{\sum_d \exp(\sum_{i=1}^{N} f_i(x) \times w_i(d))}$$



Exponential function has a graph that gets very close to the *x*-axis as the graph extends to the left (as *x* becomes more negative), but never really touches the *x*-axis.

$$p(\text{SPAM}|x) = \frac{\exp(0.6 \times 1 + 0.7 \times 1 + 0.5 \times 1)}{\exp(0.6 \times 1 + 0.7 \times 1 + 0.5 \times 1) + \exp(-0.3 \times 1 - 0.4 \times 1 - 0.2 \times 1)}$$

$$p(\text{SPAM}|x) = \frac{\exp(0.18)}{\exp(0.18) + \exp(-0.9)}$$

$$p(\text{SPAM}|x) = \frac{1.20}{1.20 + 0.41} = 0.75$$

$$p(\text{Non}-\text{SPAM}|x)=\frac{\exp(-0.3\times 1-0.4\times 1-0.2\times 1)}{\exp(0.6\times 1+0.7\times 1+0.5\times 1)+\exp(-0.3\times 1-0.4\times 1-0.2\times 1)}$$

$$p(\text{Non}-\text{SPAM}|x)=\frac{\exp(-0.9)}{\exp(0.18)+\exp(-0.9)}$$

$$p(\text{Non}-\text{SPAM}|x)=\frac{1.20}{1.20+0.41}=0.25$$

Email spam score (0.77) is higher than the Non-SPAM score (0.25). Therfore given email is SPAM.

e.g. correctly idetified by HMM (First order Markov]

is passed [Verb, Past Participle]

she passed [Verb, Past Tense]

is **absolutely** passed

she **absolutely** passed

"passed" is observed word

Feature	Weight
F1:T_{i-1}=Adverb	0.7
F2:T_i=Past Tense	0.4
F3:T_i=Past Participle	0.4
F4: T_{i-2}=Auxiliary Verb	0.3
F5:T_{i-2}=Third person singular	0.3
F6:W_{i+1}=<E>	0.1
F7:W_{i+1}=conjunction	0.2
F8:W_{i+1}=preposition	0.5
F9:W_{i+1}=determiner	0.4

Stentence is: “The complex record”

The	complex	record
Det.	Verb	Verb
Noun	Adj.	Noun

Context/Features:

POS Tags	T_{i-1}	T_i	
Word	W_{i-1}	W_i	W_{i+1}

Assumption: All features having same weight which is equal to 1.

Beam Size=2

Features are taken as below. Observed word(W_i). Its previous word (W_{i-1}) and POS Tag (T_i and T_{i-1}). **Stentence is: "The complex record"**

Feature	The (Det.)	The (Noun)	Complex (Verb)	Complex (Adj.)	Record (Verb)	Record (Noun)
F1:T_{i-1}=Det and T_i=Adj				1		
F2:T_{i-1}=Noun and T_i=Verb			1			
F3:T_{i-1}=Adj. And T_i=Noun						1
F4: W_{i-1}=the and T_i=Adj				1		
F5:W_{i-1}=the & W_{i+1}=record and T_i=Adj				1		
F6=:W_{i-1}=complex and T_i=Noun						1
F7:W_{i+1}=complex and T_i=Det	1					
F8:W_{i-1}=NULL and T_i=Noun		1				

The Word

The (Det) F7 feature is present

The(Noun) F8 feature is present

$$p(\text{Det}|\text{The}) = \frac{\exp(1 * 1)}{\exp(1 * 1) + \exp(1 * 1)} = 0.5$$

$$p(\text{Noun}|\text{The}) = \frac{\exp(1 * 1)}{\exp(1 * 1) + \exp(1 * 1)} = 0.5$$

Observed Word is complex Word

For The(Det)

complex(Verb) No feature is present

complex (Adj.) F1,F4,F5 feature is present

For The(Noun)

complex(Verb) F2 feature is present

complex (Adj.) F4,F5 feature is present

For The(Det) : complex(Verb) No feature is present

complex (Adj.) F1, F4, F5 feature is present

$$p(\text{Verb}|\text{complex}) = \frac{\exp(0)}{[\exp(1*1)+\exp(1*1)+\exp(1*1)]+\exp(0)} = 0.11$$

$$= 0.11 \times 0.5 = 0.055$$

$$p(\text{Adj.}|\text{complex}) = \frac{\exp(1*1)+\exp(1*1)+\exp(1*1)}{[\exp(1*1)+\exp(1*1)+\exp(1*1)]+\exp(0)} = 0.89$$

$$= 0.89 \times 0.5 = 0.455$$

For The(Noun)

complex(Verb) F2 feature is present

complex (Adj.) F4,F5 feature is present

$$p(\text{Verb}|\text{complex}) = \frac{\exp(1)}{[\exp(1*1)+\exp(1*1)]+\exp(1)} = 0.33$$

$$= 0.33 \times 0.5 = 0.165$$

$$p(\text{Adj.}|\text{complex}) = \frac{\exp(1*1)+\exp(1*1)}{[\exp(1*1)+\exp(1*1)]+\exp(1)} = 0.66$$

$$= 0.66 \times 0.5 = 0.33$$

First Word: The	Second Word:Complex	Answer
Det	Verb	0.055
Det	Adj.	**0.455**
Noun	Verb	0.165
Noun	Adj.	**0.330**

Beam Size=2

First Word: The	Second Word:Complex	Answer
Det	Adj.	**0.455**
Noun	Adj.	**0.330**

First Word: The	Second Word:Complex	Answer
Det	Adj.	**0.455**

For record : record(Verb) No feature is present

record (Noun) F3, F6 feature is present

$$p(\text{Verb}|\text{record}) = \frac{\exp(0)}{[\exp(1*1)+\exp(1*1)]+\exp(0)} = 0.16$$

$$= 0.16 \times 0.455 = 0.0728$$

$$p(\text{Noun}|\text{record}) = \frac{\exp(1*1)+\exp(1*1)}{[\exp(1*1)+\exp(1*1)]+\exp(0)} = 0.85$$

$$= 0.85 \times 0.455 = 0.387$$

First Word: The	Second Word:Complex	Answer
Noun	Adj.	**0.330**

For record : record(Verb) No feature is present

record (Noun) F3, F6 feature is present

$$p(\text{Verb}|\text{record}) = \frac{\exp(0)}{[\exp(1*1)+\exp(1*1)]+\exp(0)} = 0.16$$

$$= 0.16 \times 0.330 = 0.0528$$

$$p(\text{Noun}|\text{record}) = \frac{\exp(1*1)+\exp(1*1)}{[\exp(1*1)+\exp(1*1)]+\exp(0)} = 0.85$$

$$= 0.85 \times 0.330 = 0.281$$

First Word The	Second Word Complex	Third Word Record	Answer
Det	Adj. (0.455)	Verb	0.0728
Det	Adj. (0.455)	Noun	**0.387**
Noun	Adj. (0.330)	Verb	0.0528
Noun	Adj. (0.330)	Noun	0.281

The	complex	record
Det.	Adj	Noun

Problems with Maximum Entropy Models

Per-state normalization: Bias Label Problem

All the mass that arrives at a state must be distributed among the possible successor states.

First Word The	Second Word Complex
Det	Adj. (0.550)
Noun	Adj. (0.150)

For Third word 'Record', Four features gets mapped.

Two from Det Adj and Two from Noun Adj.

15

First Word The	Second Word Complex	Third Word Record
Det	Adj. (0.550)	Verb -Fa (0.0002)
Det	Adj. (0.550)	Noun -Fb (0.0001)
Noun	Adj. (0.150)	Verb -Fc (0.03)
Noun	Adj. (0.150)	Noun -Fd (0.04)

$$p(\text{Verb}|\text{record}) = \frac{\exp(1 * 0.0002)}{\exp(1 * 0.0002) + \exp(1 * 0.0001)} = 0.50$$

$$p(\text{Verb}|\text{record}) = \frac{\exp(1 * 0.03)}{\exp(1 * 0.03) + \exp(1 * 0.04)} = 0.50$$

$$p(\text{Noun}|\text{record}) = \frac{\exp(1 * 0.0001)}{\exp(1 * 0.0002) + \exp(1 * 0.0001)} = 0.50$$

$$p(\text{Noun}|\text{record}) = \frac{\exp(1 * 0.04)}{\exp(1 * 0.03) + \exp(1 * 0.04)} = 0.50$$

Solution is Conditional Random Field (CRF)

- CRF is a Discriminative Probabilistic Classifiers.
- It is same as Maximum Entropy model only per state normalization is not used.
- Normalization at the sentence level is used

CONDITIONAL RANDOM FIELDS (CRF):

- This is also a sequence modelling algorithm.
- This not only assumes that features are dependent on each other, but also considers the future observations while learning a pattern.
- This combines the best of both HMM and MEMM.
- In terms of performance, it is considered to be the best method for entity recognition problem

Below is the formula for CRF where Y is the hidden state (for example, part of speech) and X is the observed variable (in our example this is the entity or other words around it).

$$p(\mathbf{y}|\mathbf{x}) = \underbrace{\frac{1}{Z(\mathbf{x})}}_{\text{Normalization}} \prod_{t=1}^{T} \exp \left\{ \sum_{k=1}^{K} \underbrace{\theta_k}_{\text{Weight}} \underbrace{f_k(y_t, y_{t-1}, \mathbf{x}_t)}_{\text{Feature}} \right\}$$

Classification of ML Methods

- Generative Methods
  - "Generative" since sampling can generate synthetic data points
  - Popular models
    - Naïve Bayes, Mixtures of multinomials
    - Mixtures of Gaussians, Hidden Markov Models
    - Bayesian networks, Markov random fields
- Discriminative Methods
  - Focus on given task– better performance
  - Popular models
    - Logistic regression, SVMs
    - Traditional neural networks, Nearest neighbor
    - Conditional Random Fields (CRF)

Generative Model: HMM

- X is observed data sequence to be labeled,
  Y is the random variable over the label sequences
- HMM is a distribution that models $p(Y, X)$
- Joint distribution is



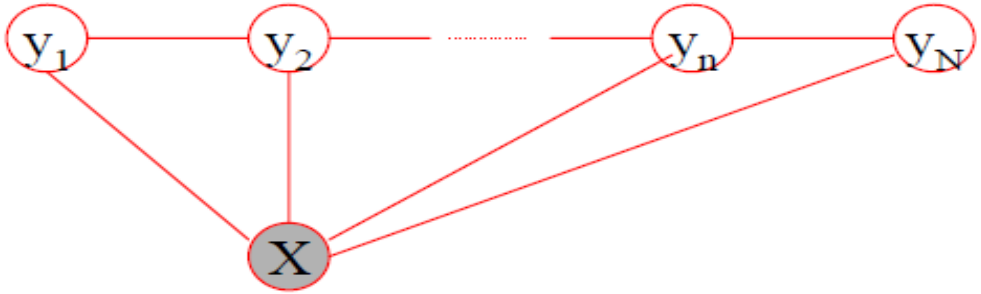
$$p(\mathbf{Y},\mathbf{X}) = \prod_{n=1}^{N} p(y_n \mid y_{n-1})\, p(\mathbf{x}_n \mid y_n)$$

- Highly structured network indicates conditional independences,
  - past states independent of future states
  - Conditional independence of observed given its state.

19

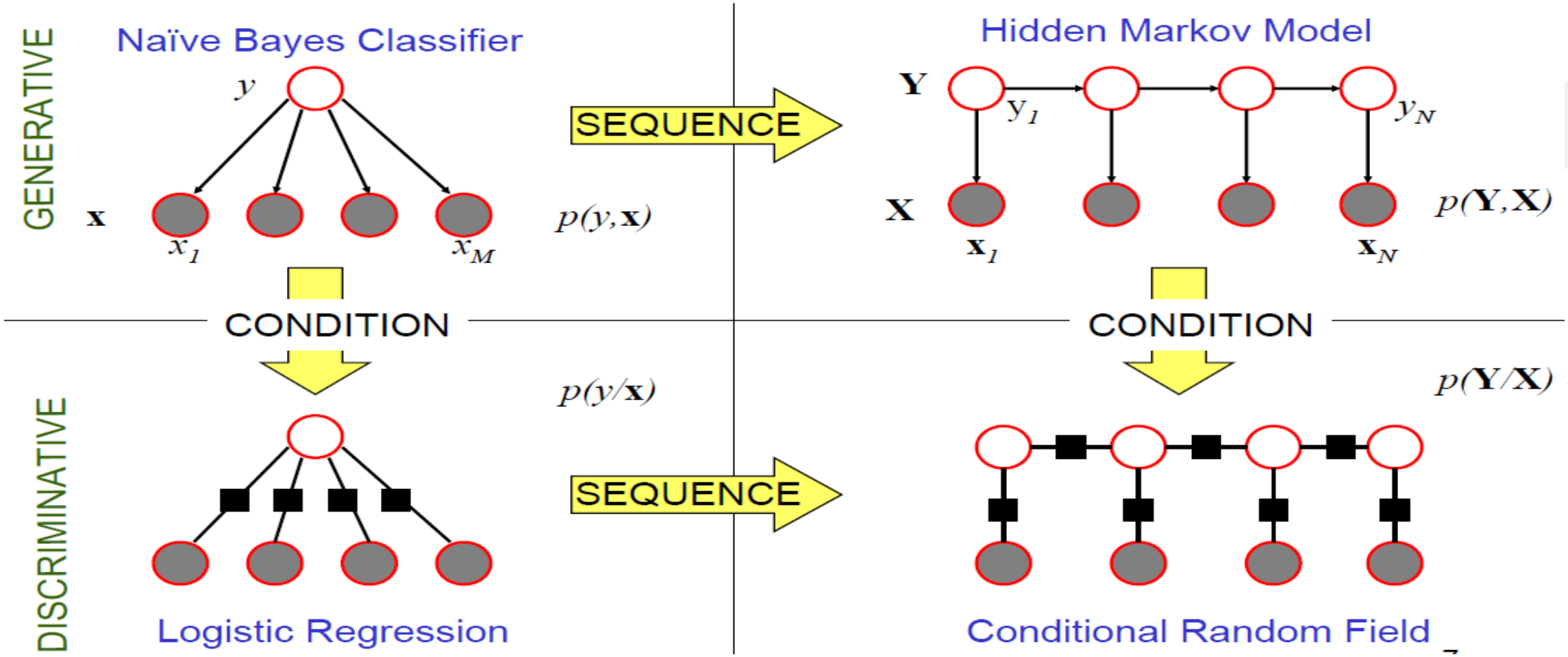
Discriminative Model for Sequential Data

- CRF models the conditional distribution $p(Y/X)$
- CRF is a random field globally conditioned on the observation X
- The conditional distribution $p(Y|X)$ that follows from the joint distribution $p(Y,X)$ can be rewritten as a *Markov Random Field*



20

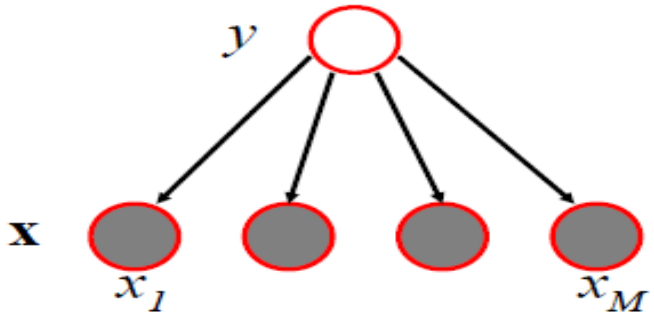
Graphical Model Relationship



Functional Models

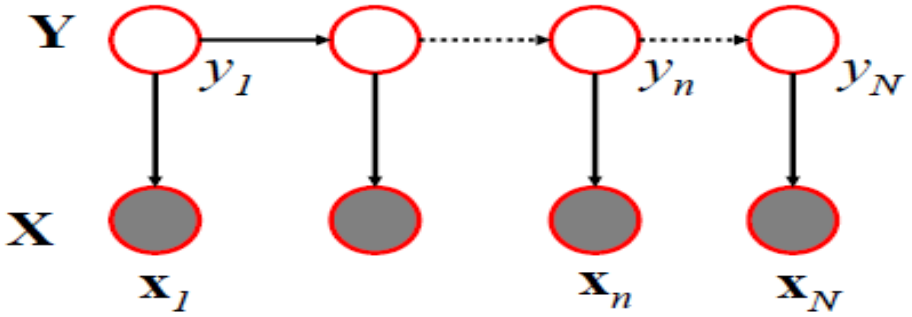
GENERATIVE

Naïve Bayes Classifier



$$p(y,\mathbf{x}) = p(y)\prod_{m=1}^{M} p(x_m \mid y)$$

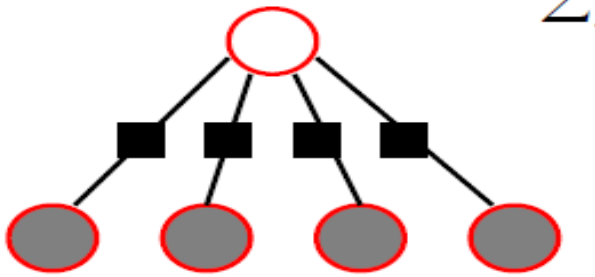
Hidden Markov Model



$$p(\mathbf{Y},\mathbf{X}) = \prod_{n=1}^{N} p(y_n \mid y_{n-1})\, p(\mathbf{x}_n \mid y_n)$$

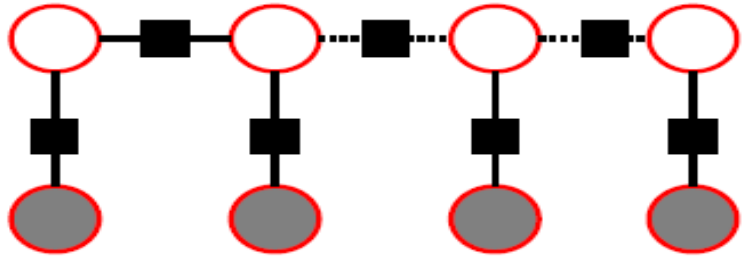
DISCRIMINATIVE

$$p(y \mid \mathbf{x}) = \frac{\exp\left\{\sum_{m=1}^{M} \lambda_m f_m(y,\mathbf{x})\right\}}{\sum_{y'} \exp\left\{\sum_{m=1}^{M} \lambda_m f_m(y',\mathbf{x})\right\}}$$



Logistic Regression

$$p(\mathbf{Y} \mid \mathbf{X}) = \frac{\exp\left\{\sum_{m=1}^{M} \lambda_m f_m(y_n, y_{n-1}, \mathbf{x}_n)\right\}}{\sum_{y'} \exp\left\{\sum_{m=1}^{M} \lambda_m f_m(y_n', y_{n-1}', \mathbf{x}_n)\right\}}$$

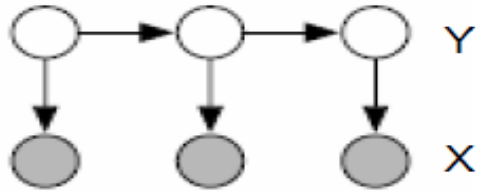


Conditional Random Field

Sequence Models

Hidden Markov Model (HMM)

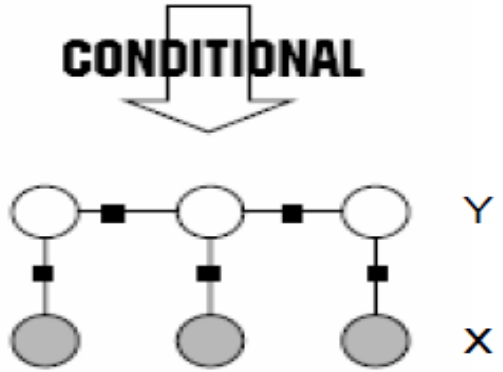
$$p(\mathbf{y}, \mathbf{x}) = \prod_{t=1}^{\mathrm{T}} p(y_t|y_{t-1})p(x_t|y_t)$$



HMMs

Conditional Random Fields (CRF)

$$p(\mathbf{y}|\mathbf{x}) = \frac{1}{Z(\mathbf{x})} \exp\left\{\sum_{k=1}^{K} \lambda_k f_k(y_t, y_{t-1}, \mathbf{x}_t)\right\}$$



Linear-chain CRFs

A key advantage of CRF is their great flexibility to include a wide variety of arbitrary, non-independent features of the observations.

CRF definition

- A *linear chain* CRF is a distribution $p(\mathbf{Y}|\mathbf{X})$ that takes the form

$$p(\mathbf{Y} \mid \mathbf{X}) = \frac{1}{Z(\mathbf{X})} \exp\left\{\sum_{m=1}^{M} \lambda_m f_m(y_n, y_{n-1}, \mathbf{x}_n)\right\}$$

- Where $Z(X)$ is an instance specific normalization function

$$Z(\mathbf{X}) = \sum_{y} \exp\left\{\sum_{m=1}^{M} \lambda_m f_m(y_n, y_{n-1}, \mathbf{x}_n)\right\}$$

NLP: Part Of Speech Tagging

For a sequence of words $w = \{w_1, w_2, .. w_n\}$ find syntactic labels s for each word:

w = The quick brown fox jumped over the lazy dog

s = DET VERB ADJ NOUN-S VERB-P PREP DET ADJ NOUN-S

Model	Error
HMM	5.69%
CRF	5.55%

Baseline is already 90%

- Tag every word with its most frequent tag
- Tag unknown words as nouns

Per-word error rates for POS tagging on the Penn treebank

Table Extraction

To label lines of text document:

Whether part of table and its role in table.

Finding tables and extracting information is necessary component of data mining, question-answering and IR tasks.

HMM	CRF
89.7%	99.9%

Shallow Parsing

- Precursor to full parsing or information extraction
  - Identifies non-recursive cores of various phrase types in text
- Input: words in a sentence annotated automatically with POS tags
- Task: label each word with a label indicating
  - word is outside a chunk (O), starts a chunk (B), continues a chunk (I)

NP chunks

[Rockwell International Corp.] [’s Tulsa unit] said [it] signed [a tentative agreement] extending
[its contract] with [Boeing Co.] to provide [structural parts] for [Boeing] [’s 747 jetliners] .

CRFs beat all reported single-model NP chunking results on standard evaluation dataset

Model	F score
CRF	94.38%
Generalized winnow	93.89%
Voted perceptron	94.09%
MEMM	93.70%

30

Handwritten Word Recognition

Given word image and lexicon, find most probable lexical entry

Algorithm Outline

- Oversegment image
  segment combinations are potential characters
- Given y = a word in lexicon, s = grouping of segments, x = input word image features
- Find word in lexicon and segment grouping that maximizes $P(y,s \mid x)$,

CRF Model

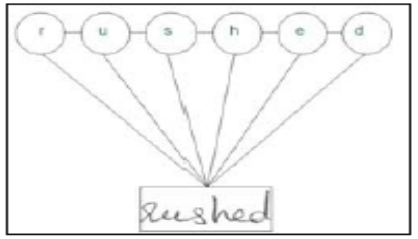
$$P(y \mid \mathrm{x}, \theta) = \frac{e^{\psi(y,\mathrm{x};\theta)}}{\sum_{y'} e^{\psi(y',\mathrm{x};\theta)}}$$

$$\psi(y,\mathrm{x};\theta) = \sum_{j=1}^{m} \left(A(j, y_j, \mathrm{x}; \theta^s + \sum_{(j,k)\in E} I(j,k,y_j,y_k,\mathrm{x},\theta^t) \right)$$

where $y_i \, \varepsilon \, (a\text{-}z, A\text{-}Z, 0\text{-}9\}$, θ : model parameters

Association Potential (state term)

$$A(j, y_j, \mathrm{x}; \theta^s) = \sum_i (f_i^s(j, y_j, \mathrm{x}) \cdot \theta_{ij}^s)$$

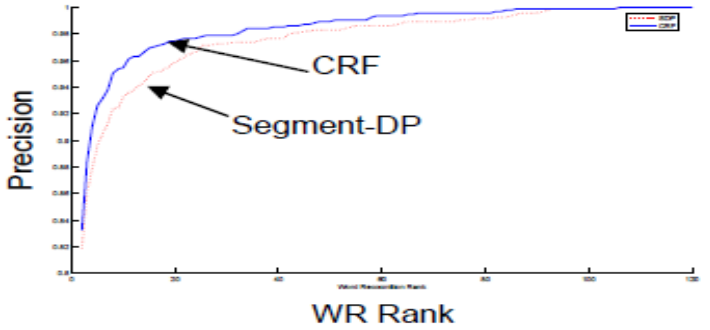


Interaction Potential

$$I(j,k,y_j,y_k,\mathrm{x};\theta^t) = \sum_i (f_i^t(j,k,y_j,y_k,\mathrm{x}) \cdot \theta_{ijk}^t)$$

Feature	Description
Position	Position of character in the lexicon normalized by the length
Place	Whether the character appears in the beginning, middle or at the end.
Height	Height(pixels) of the *candidate character image*
Width	Width(pixels) of the *candidate character image*
Aspect ratio	Ratio of the height of the character to its width
Euclidean Distance	Euclidean Distance of the character to its prototype cluster center
Manhattan Distance	Manhattan Distance of the character to its prototype cluster center
Tanimoto Distance	Tanimoto Distance of the character to its prototype cluster center
Inner Product	Inner Product of the character WMR features and its prototype cluster center features
KNN Distance	Distance of the character from its 5 nearest prototype images
Height Deviation	Deviation of the height of the character from its expected height
Top Deviation	Deviation of the position of the top of the character from its expected top position
Bottom Deviation	Deviation of the position of the top of the character from its expected top position

Feature	Description
Label	Label of the character pair eg. a,b or q,u etc.
Vertical overlap	Vertical overlap(pixels) between the two *candidate character images*
Height difference	Difference in Height(pixels) between the *candidate character images*
Width difference	Difference in Width(pixels) between the *candidate character images*
Aspect ratio difference	Difference in aspect ratio between the *candidate character images*
Bigram width	Sum of individual widths(pixels) of the *candidate character images.*



Document Zoning (labeling regions) error rates

	CRF	Neural Network	Naive Bayes
Machine Printed Text	1.64%	2.35%	11.54%
Handwritten Text	5.19%	20.90%	25.04%
Noise	10.20%	15.00%	12.23%
Total	4.25%	7.04%	12.58%

32

1.Normalization:

You may have observed that there are no probabilities on the right side of the equation where we have the weights and features.

However, the output is expected to be a probability and hence there is a need for normalization.

The normalization constant Z(x) is a sum of all possible state sequences such that the total becomes 1.

2.Weights and Features:

This component can be thought of as the logistic regression formula with weights and the corresponding features.

The weight estimation is performed by maximum likelihood estimation and the features are defined by us.

CONSTITUENCY

- A sequence of words that acts as a single unit
  - Noun phrases
  - Verb phrases
- These units form coherent classes that behave in similar ways
  - For example, we can say that noun phrases can come before verbs

CONSTITUENCY

- For example, following are all ***noun phrases*** in English...

Harry the Horse	a high-class spot such as Mindy's
the Broadway coppers	the reason he comes into the Hot Box
they	three parties from Brooklyn

CONSTITUENTS: HEADS AND DEPENDENTS

- There are different kinds of constituents:
- Noun phrases: the man, a girl with glasses, Illinois
- Prepositional phrases: with glasses, in the garden
- Verb phrases: eat sushi, sleep, sleep soundly
- Every phrase has a head:
- Noun phrases: the man, a girl with glasses, Illinois
- Prepositional phrases: with glasses, in the garden
- Verb phrases: eat sushi, sleep, sleep soundly
- The other parts are its dependents.
- Dependents are either arguments or adjuncts

Constituency structure

- **Phrase structure** organizes words into **nested constituents**
- Starting units: words are given a category: part-of-speech tags

  the, cuddly, cat, by, the, door

  Det, Adj, N, P, Det, N

- Words combine into phrases with categories

  the cuddly cat, by the door

  NP→Det Adj N PP→P NP

- Phrases can combine into bigger phrases recursively

  the cuddly cat by the door

  NP→ NP PP

IS STRING A A CONSTITUENT?

He talks [in class].

Substitution test:

Can α be replaced by a single word?

He talks [there].

Movement test:

Can α be moved around in the sentence?

[In class], he talks.

Answer test:

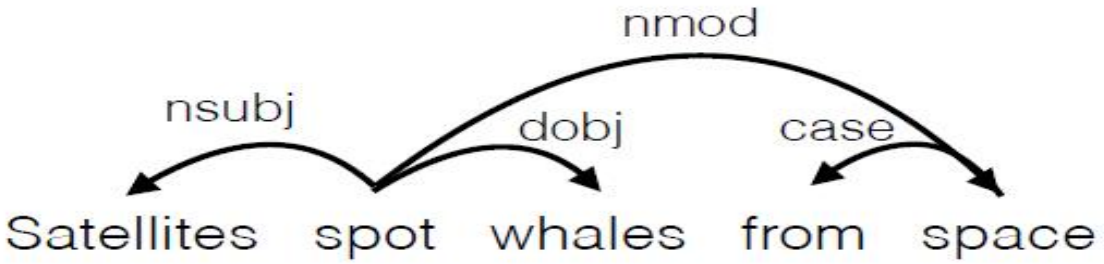
Can α be the answer to a question?

Where does he talk? - [In class].

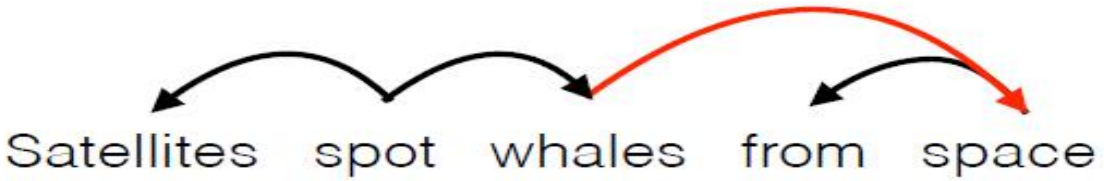
This Thursday

Dependency structure

- Dependency structure shows which words depend on (modify or are arguments of) which other words.









Why do we need sentence structure?

- We need to understand sentence structure in order to be able to interpret language correctly
- Human communicate complex ideas by composing words together into bigger units
- We need to know what is connected to what

- A CFG defines a formal language = the set of all sentences (strings of words) that can be derived by the grammar.
- Sentences in this set said to be grammatical.
- Sentences outside this set said to be ungrammatical

INTRODUCTION TO CFG

$$G = \langle T, N, S, R \rangle$$

- T is set of terminals (lexicon)
- N is set of non-terminals For NLP, we usually distinguish out a set $P \subset N$ of *preterminals* which always rewrite as terminals.
- S is start symbol (one of the nonterminals)
- R is rules/productions of the form $X \rightarrow \gamma$, where X is a nonterminal and γ is a sequence of terminals and nonterminals (may be empty).
- A grammar G generates a language L.

AN EXAMPLE CONTEXT-FREE GRAMMAR

$G = \langle T, N, S, R \rangle$

$T = \{that,\ this,\ a,\ the,\ man,\ book,\ flight,\ meal,\ include,\ read,\ does\}$

$N = \{\text{S, NP, NOM, VP, Det, Noun, Verb, Aux}\}$

$S = S$

$R = \{$

S → NP VP
S → Aux NP VP
S → VP
NP → Det NOM
NOM → Noun
NOM → Noun NOM
VP → Verb
VP → Verb NP

Det → *that* | *this* | *a* | *the*
Noun → *book* | *flight* | *meal* | *man*
Verb → *book* | *include* | *read*
Aux → *does*

$\}$

A Context-Free Grammar for English

$N = \{S, NP, VP, PP, DT, Vi, Vt, NN, IN\}$

$S = S$

$\Sigma = \{sleeps, saw, man, woman, telescope, the, with, in\}$

$R =$

S	→	NP	VP
VP	→	Vi	
VP	→	Vt	NP
VP	→	VP	PP
NP	→	DT	NN
NP	→	NP	PP
PP	→	IN	NP

Grammar

Vi	→	sleeps
Vt	→	saw
NN	→	man
NN	→	woman
NN	→	telescope
NN	→	dog
DT	→	the
IN	→	with
IN	→	in

Lexicon

S:sentence, VP:verb phrase, NP: noun phrase, PP:prepositional phrase, DT:determiner, Vi:intransitive verb, Vt:transitive verb, NN: noun, IN:preposition

APPLICATION OF GRAMMAR REWRITE RULES

S → NP VP	Det → *that* | *this* | *a* | *the*
S → Aux NP VP	Noun → *book* | *flight* | *meal* | *man*
S → VP	Verb → *book* | *include* | *read*
NP → Det NOM	Aux → *does*
NOM → Noun	
NOM → Noun NOM	
VP → Verb	
VP → Verb NP	

S → NP VP
→ Det NOM VP
→ *The* NOM VP
→ *The* Noun VP
→ *The man* VP
→ *The man* Verb NP
→ *The man read* NP
→ *The man read* Det NOM
→ *The man read this* NOM
→ *The man read this* Noun
→ *The man read this book*

L0 GRAMMAR

Grammar Rules			Examples
S	→	*NP VP*	I + want a morning flight
NP	→	*Pronoun*	I
	|	*Proper-Noun*	Los Angeles
	|	*Det Nominal*	a + flight
Nominal	→	*Nominal Noun*	morning + flight
	|	*Noun*	flights
VP	→	*Verb*	do
	|	*Verb NP*	want + a flight
	|	*Verb NP PP*	leave + Boston + in the morning
	|	*Verb PP*	leaving + on Thursday
PP	→	*Preposition NP*	from + Los Angeles

NP STRUCTURE

➔ *All the morning flights from Denver to Tampa leaving before 10*

- Clearly this NP is really about *flights*. That's the central critical noun in this NP. Such word is called as the ***head***.
- We can dissect this kind of NP into the stuff that can come before the head, and the stuff that can come after it.

DETERMINERS

- Noun phrases can start with determiners...
- Determiners can be
  - Simple lexical items: *the, this, a, an*, etc.
    - A car
  - Or simple possessives
    - John’s car
  - Or complex recursive versions of that
    - John’s sister’s husband’s son’s car

NOMINALS

- Contains the head and any pre- and post- modifiers of the head.
  - Pre-
    - Quantifiers, cardinals, ordinals...
      - Three cars
    - Adjectives and Aps
      - large cars
    - Ordering constraints
      - Three large cars
      - ?large three cars

POSTMODIFIERS

- Three kinds
  - **Prepositional phrases**
    - Flights ***from Seattle***
  - **Non-finite clauses**
    - Flights ***arriving before noon***
  - **Relative clauses**
    - Flights ***that serve breakfast***
- Same general (recursive) rule to handle these
  - *Nominal → Nominal PP*
  - *Nominal → Nominal GerundVP*
  - *Nominal → Nominal RelClause*

AGREEMENT

- Constraints that hold among various constituents.
- For example, in English, determiners and the head nouns in NPs have to agree in their number.

- Which of the following cannot be parsed by the rule

NP → Det Nominal **?**

(O) This flight
(O) Those flights

(X) This flights
(X) Those flight

AGREEMENT

- Constraints that hold among various constituents.
- For example, in English, determiners and the head nouns in NPs have to agree in their number.

- Which of the following cannot be parsed by the rule

NP → Det Nominal **?**

➔ This rule does not handle agreement! (The rule does not detect whether the agreement is correct or not.)

(O) This flight
(O) Those flights

(X) This flights
(X) Those flight

PROBLEM

- Our earlier NP rules are clearly deficient since they don't capture the agreement constraint
  - *NP → Det Nominal*
    - Accepts, and assigns correct structures, to grammatical examples (*this flight*)
    - But its also happy with incorrect examples (*these flight)
  - Such a rule is said to ***overgenerate***.
  - We'll come back to this in a bit

VERB PHRASES

- English *VP*s consist of a head verb along with 0 or more following constituents which we'll call ***arguments***.

$VP \rightarrow Verb$ disappear
$VP \rightarrow Verb\ NP$ prefer a morning flight
$VP \rightarrow Verb\ NP\ PP$ leave Boston in the morning
$VP \rightarrow Verb\ PP$ leaving on Thursday

SUBCATEGORIZATION

- But, even though there are many valid VP rules in English, not all verbs are allowed to participate in all those VP rules.
- We can subcategorize the verbs in a language according to the sets of VP rules that they participate in.
- This is a modern take on the traditional notion of transitive/intransitive.
- Modern grammars may have 100s or such classes.

SUBCATEGORIZATION

- Sneeze: John sneezed
- Find: Please find [a flight to NY]$_{NP}$
- Give: Give [me]$_{NP}$[a cheaper fare]$_{NP}$
- Help: Can you help [me]$_{NP}$[with a flight]$_{PP}$
- Prefer: I prefer [to leave earlier]$_{TO\text{-}VP}$
- Told: I was told [United has a flight]$_{S}$
- …

SUBCATEGORIZATION

- ***John sneezed the book**
- ***I prefer United has a flight**
- ***Give with a flight**

- As with agreement phenomena, we need a way to formally express the constraints!

WHY?

- Right now, the various rules for VPs ***overgenerate***.
  - They permit the presence of strings containing verbs and arguments that don't go together
  - For example
  - VP -> V NP therefore

    Sneezed the book is a VP since "sneeze" is a verb and "the book" is a valid NP

Possible CFG Solution

- Possible solution for agreement.
- Can use the same trick for all the verb/VP classes.

- SgS -> SgNP SgVP
- PlS -> PlNp PlVP
- SgNP -> SgDet SgNom
- PlNP -> PlDet PlNom
- PlVP -> PlV NP
- SgVP ->SgV Np
- …

CFG SOLUTION FOR AGREEMENT

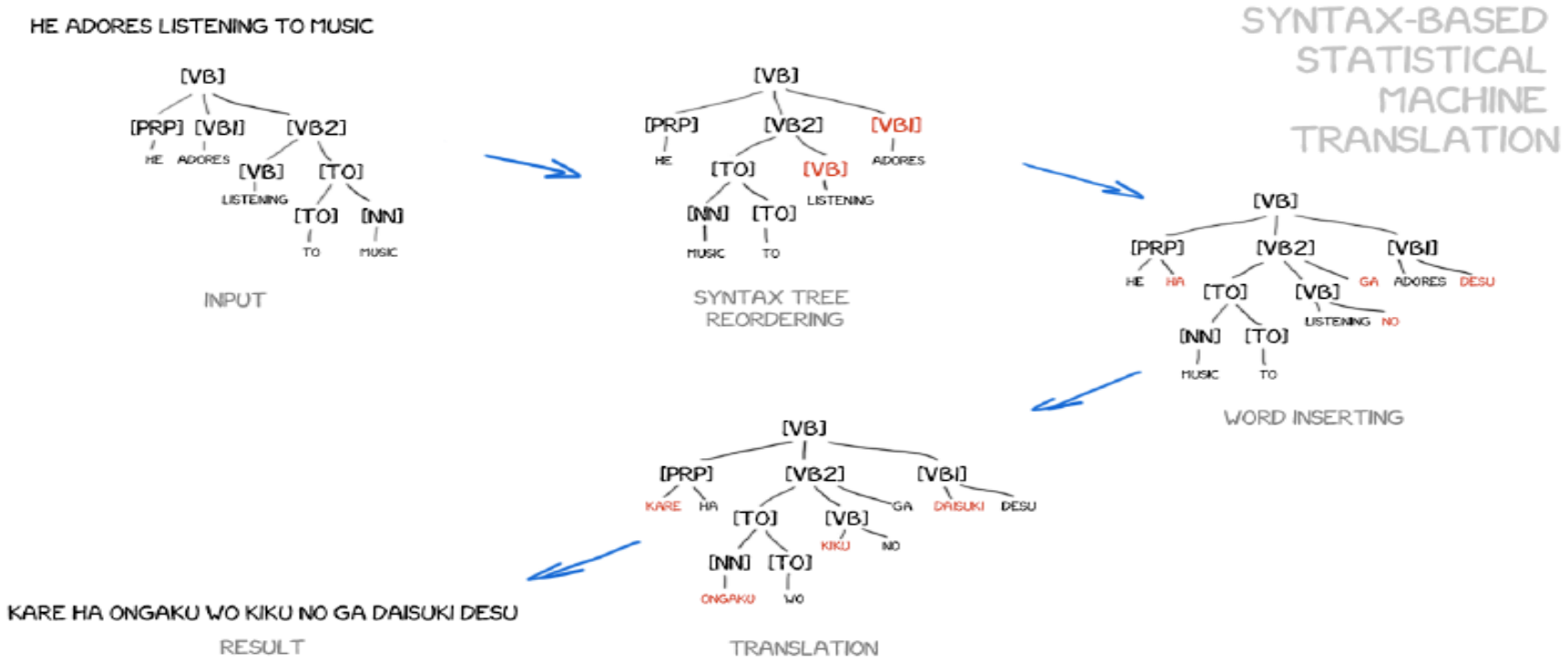
- It works and stays within the power of CFGs
- And it doesn’t scale all that well because of the interaction among the various constraints explodes the number of rules in our grammar.

Syntactic parsing

- Used as intermediate representation for downstream applications

English word order: subject — verb — object

Japanese word order: subject — object — verb

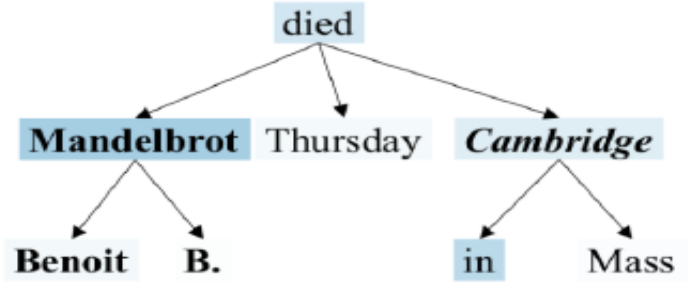


Syntactic parsing

- Used as intermediate representation for downstream applications

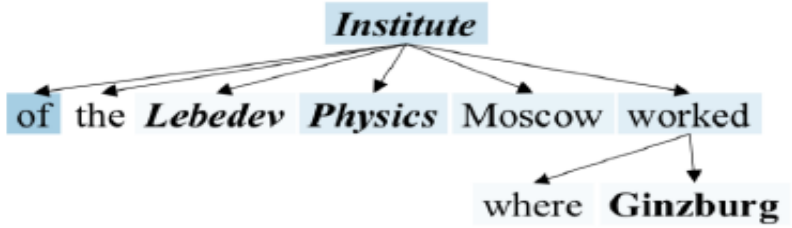
Relation: *per:city_of_death*

Benoit B. Mandelbrot, a maverick mathematician who developed an innovative theory of roughness and applied it to physics, biology, finance and many other fields, died Thursday in ***Cambridge***, Mass.



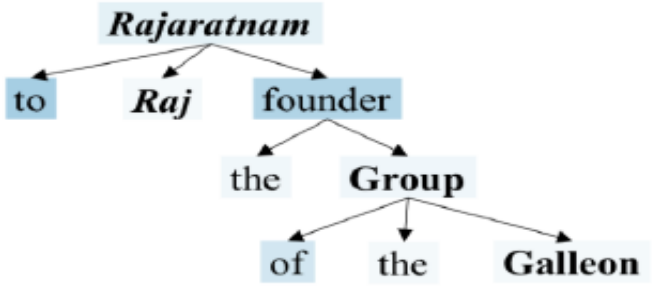
Relation: *per:employee_of*

In a career that spanned seven decades, Ginzburg authored several groundbreaking studies in various fields -- such as quantum theory, astrophysics, radio-astronomy and diffusion of cosmic radiation in the Earth's atmosphere -- that were of "Nobel Prize caliber," said Gennady Mesyats, the director of the ***Lebedev Physics Institute*** in Moscow, where **Ginzburg** worked .



Relation: *org:founded_by*

Anil Kumar, a former director at the consulting firm McKinsey & Co, pleaded guilty on Thursday to providing inside information to ***Raj Rajaratnam***, the founder of the **Galleon Group**, in exchange for payments of at least $ 175 million from 2004 through 2009.



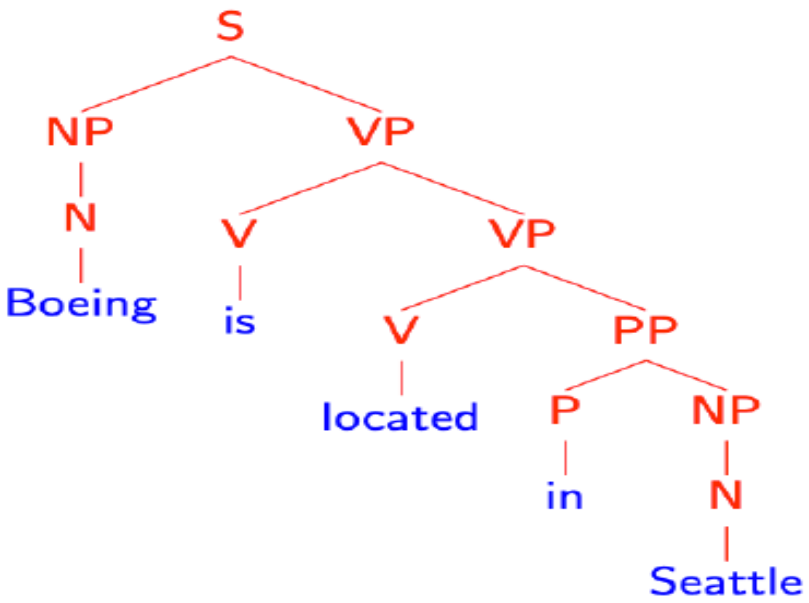
Syntactic parsing

- Syntactic parsing is the task of recognizing a sentence and assigning a structure to it.

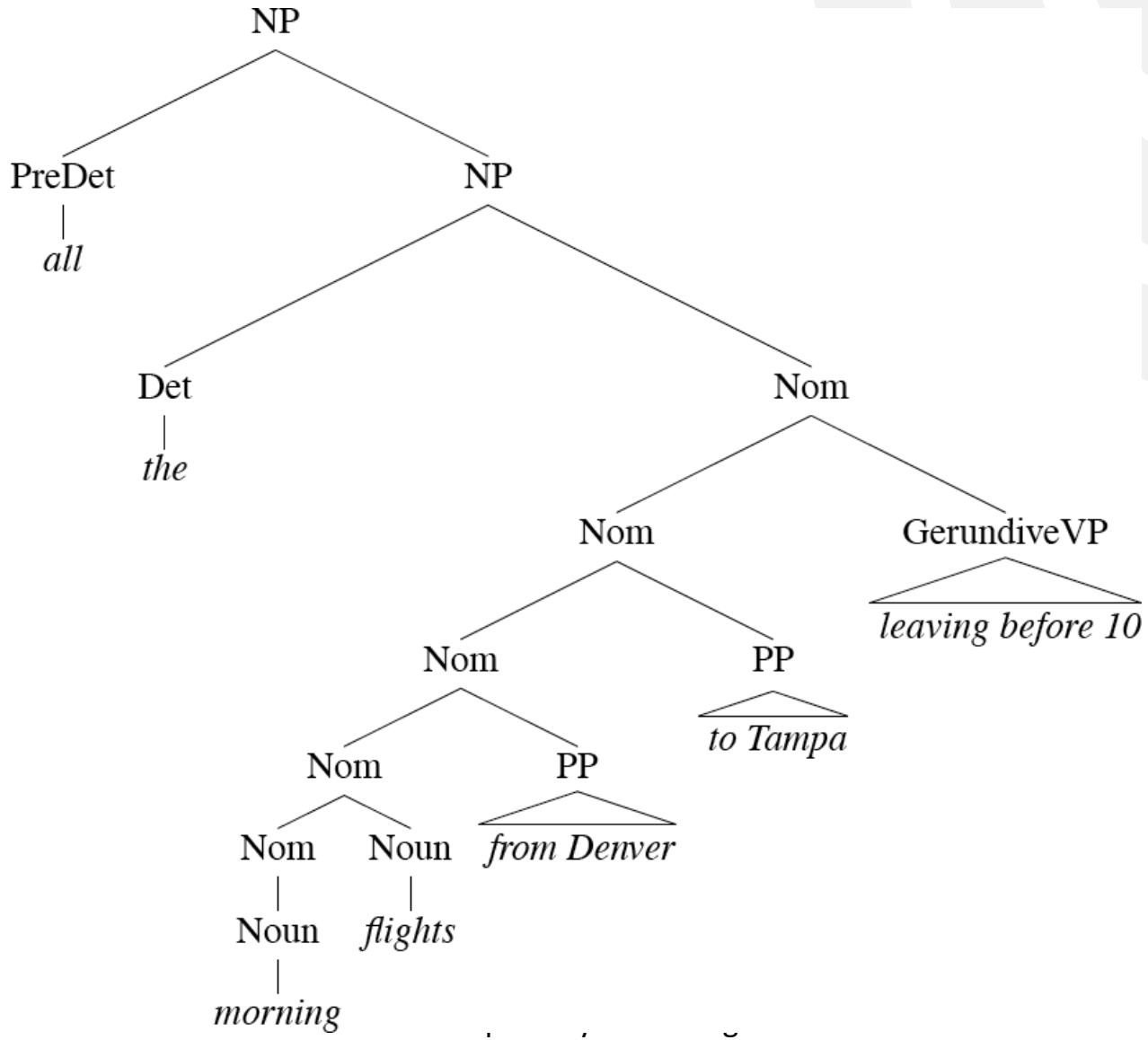
Input:

Beoing is located in Seattle.

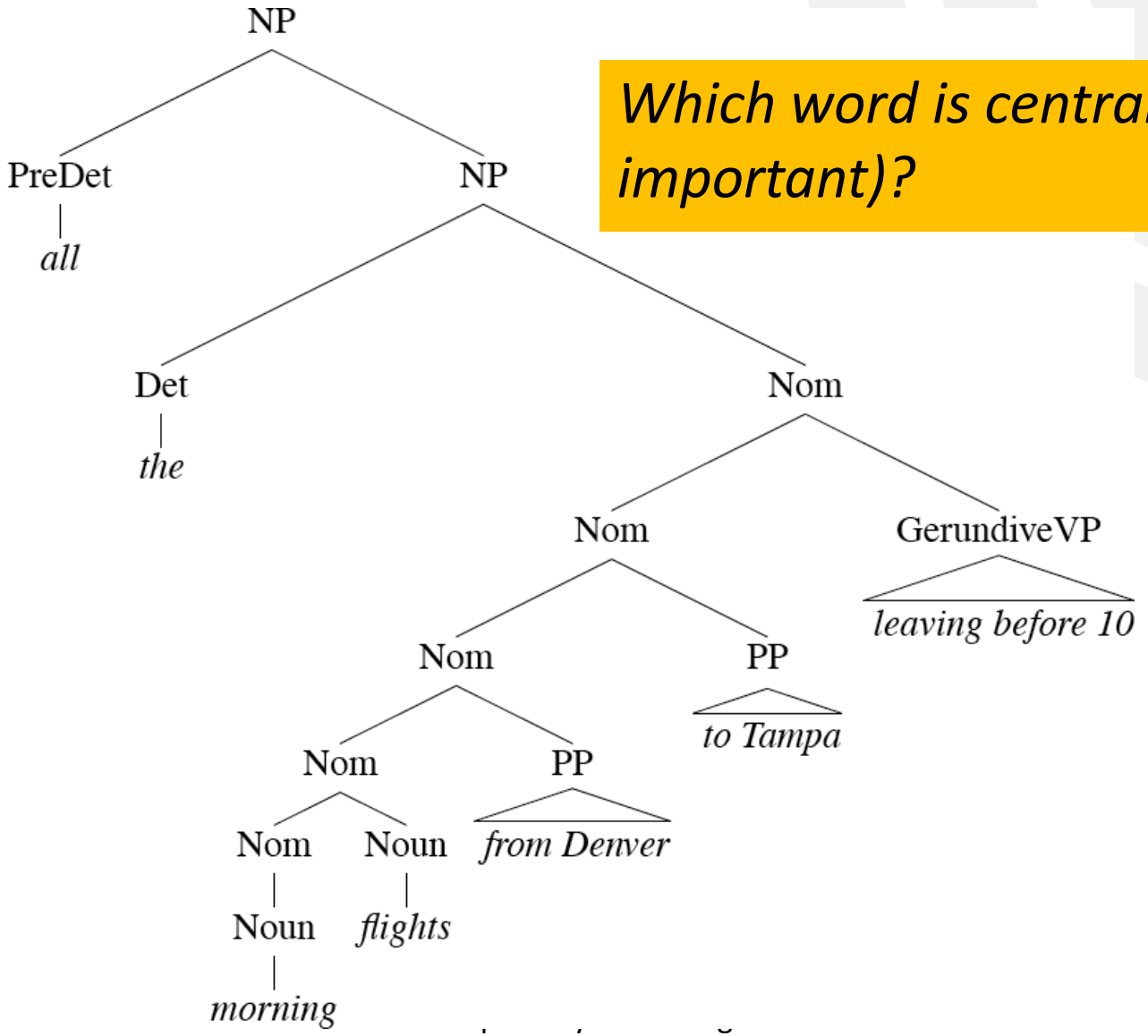
Output:



ALL THE MORNING FLIGHTS FROM DENVER TO TAMPA LEAVING BEFORE 10

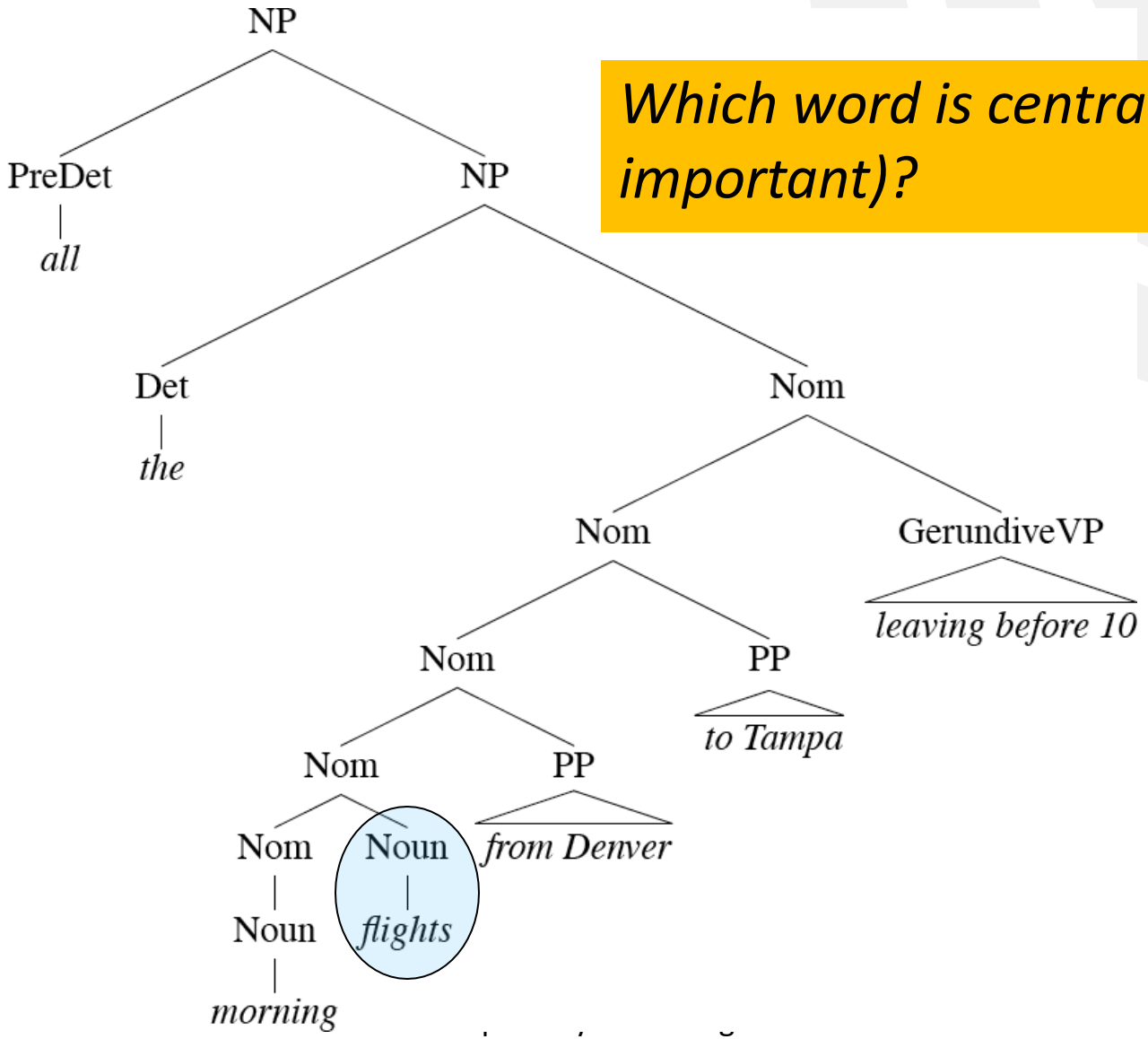


ALL THE MORNING FLIGHTS FROM DENVER TO TAMPA LEAVING BEFORE 10



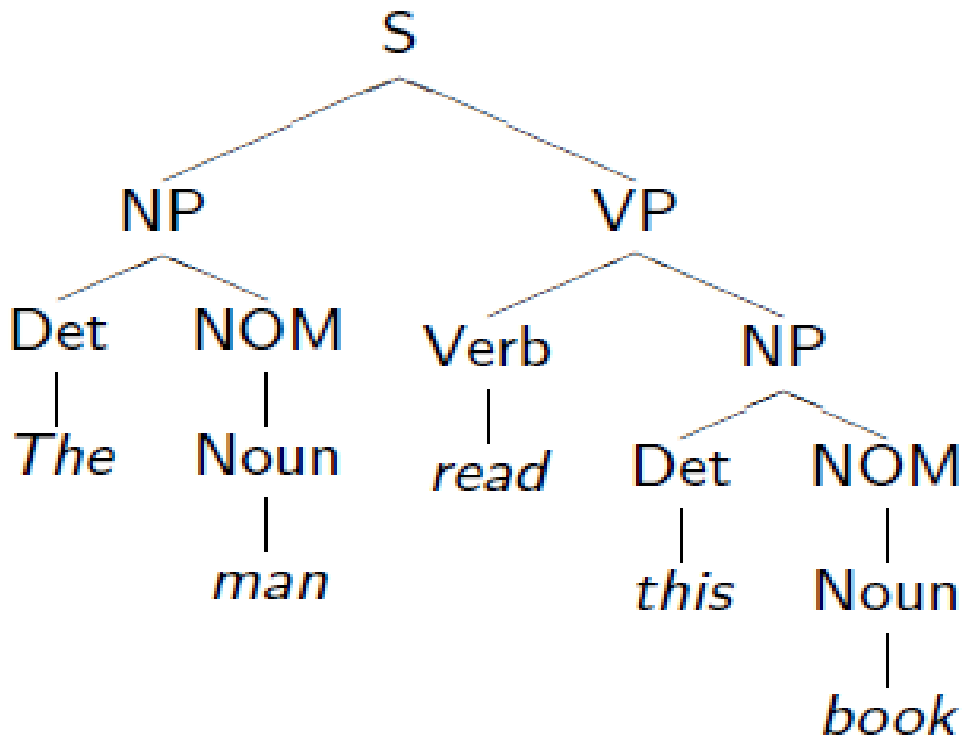
Which word is central (most important)?

ALL THE MORNING FLIGHTS FROM DENVER TO TAMPA LEAVING BEFORE 10



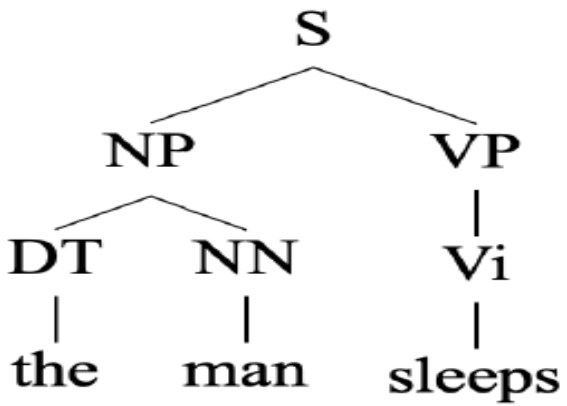
Which word is central (most important)?

PARSE TREE



(Left-most) Derivations

- $s_1 = \text{S}$
- $s_2 = \text{NP VP}$
- $s_3 = \text{DT NN VP}$
- $s_4 = \text{the NN VP}$
- $s_5 = \text{the man VP}$
- $s_6 = \text{the man Vi}$
- $s_7 = \text{the man sleeps}$



$R =$

S	→	NP	VP
VP	→	Vi	
VP	→	Vt	NP
VP	→	VP	PP
NP	→	DT	NN
NP	→	NP	PP
PP	→	IN	NP

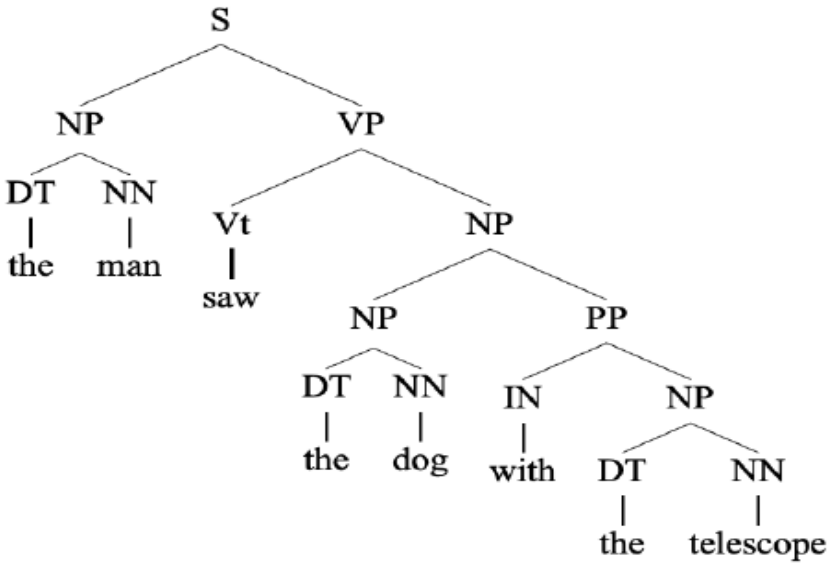
Vi	→	sleeps
Vt	→	saw
NN	→	man
NN	→	woman
NN	→	telescope
NN	→	dog
DT	→	the
IN	→	with
IN	→	in

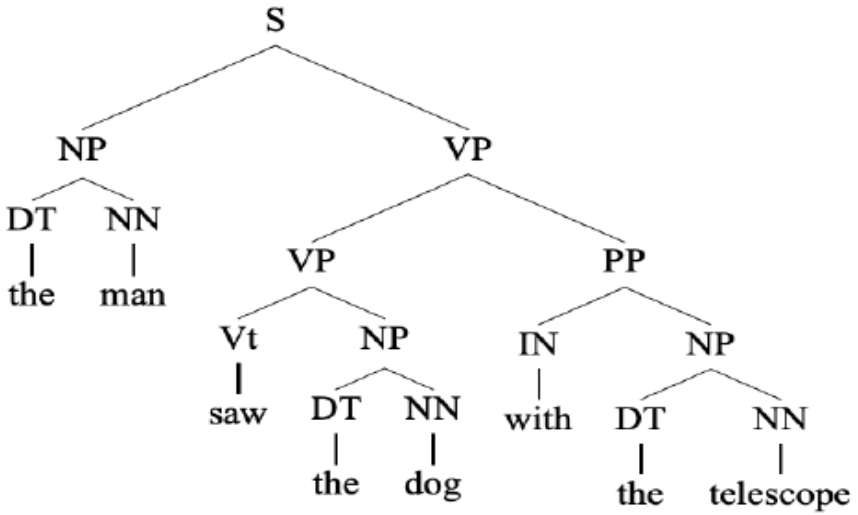
A derivation can be represented as a parse tree!

- A string $s \in \Sigma^*$ is in the language defined by the CFG if there is at least one derivation whose yield is s
- The set of possible derivations may be finite or infinite

Ambiguity

- Some strings may have more than one derivations (i.e. more than one parse trees!).





Statistical parsing

- **Learning from data**: treebanks
- **Adding probabilities to the rules**: probabilistic CFGs (PCFGs)

Treebanks: a collection of sentences paired with their parse trees

```
((S
   (NP-SBJ (DT That)
     (JJ cold) (, ,)
     (JJ empty) (NN sky) )
   (VP (VBD was)
     (ADJP-PRD (JJ full)
       (PP (IN of)
         (NP (NN fire)
           (CC and)
           (NN light) ))))
   (. .) ))
```

(a)

```
((S
   (NP-SBJ The/DT flight/NN )
   (VP should/MD
     (VP arrive/VB
       (PP-TMP at/IN
         (NP eleven/CD a.m/RB ))
       (NP-TMP tomorrow/NN )))))
```

(b)

The Penn Treebank Project (Marcus et al, 1993)

Probabilistic context-free grammars (PCFGs)

S	⇒	NP	VP	1.0
VP	⇒	Vi		0.4
VP	⇒	Vt	NP	0.4
VP	⇒	VP	PP	0.2
NP	⇒	DT	NN	0.3
NP	⇒	NP	PP	0.7
PP	⇒	P	NP	1.0

Vi	⇒	sleeps	1.0
Vt	⇒	saw	1.0
NN	⇒	man	0.7
NN	⇒	woman	0.2
NN	⇒	telescope	0.1
DT	⇒	the	1.0
IN	⇒	with	0.5
IN	⇒	in	0.5

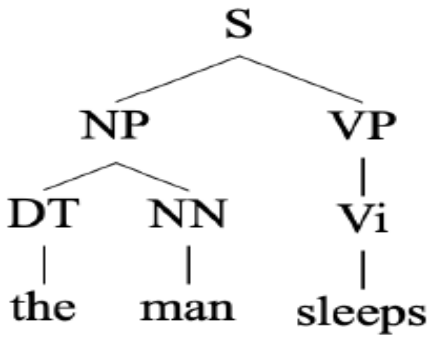
- A probabilistic context-free grammar (PCFG) consists of:
  - A context-free grammar: $G = (N, \Sigma, R, S)$
  - For each rule $\alpha \rightarrow \beta \in R$, there is a parameter $q(\alpha \rightarrow \beta) \geq 0$. For any $X \in N$,

$$\sum_{\alpha \rightarrow \beta : \alpha = X} q(\alpha \rightarrow \beta) = 1$$

Probabilistic context-free grammars (PCFGs)

For any derivation (parse tree) containing rules:
$\alpha_1 \rightarrow \beta_1, \alpha_2 \rightarrow \beta_2, \ldots, \alpha_l \rightarrow \beta_l$, the probability of the parse is:

$$\prod_{i=1}^{l} q(\alpha_i \rightarrow \beta_i)$$



S	⇒	NP	VP	1.0
VP	⇒	Vi		0.4
VP	⇒	Vt	NP	0.4
VP	⇒	VP	PP	0.2
NP	⇒	DT	NN	0.3
NP	⇒	NP	PP	0.7
PP	⇒	P	NP	1.0

Vi	⇒	sleeps	1.0
Vt	⇒	saw	1.0
NN	⇒	man	0.7
NN	⇒	woman	0.2
NN	⇒	telescope	0.1
DT	⇒	the	1.0
IN	⇒	with	0.5
IN	⇒	in	0.5

$$P(t) = q(\text{S} \rightarrow \text{NP VP}) \times q(\text{NP} \rightarrow \text{DT NN}) \times q(\text{DT} \rightarrow \text{the})$$
$$\times q(\text{NN} \rightarrow \text{man}) \times q(\text{VP} \rightarrow \text{Vi}) \times q(\text{Vi} \rightarrow \text{sleeps})$$

$$= 1.0 \times 0.3 \times 1.0 \times 0.7 \times 0.4 \times 1.0 = 0.084$$

Why do we want $\sum_{\alpha \rightarrow \beta : \alpha = X} q(\alpha \rightarrow \beta) = 1$?

Deriving a PCFG from a treebank

- Training data: a set of parse trees $t_1, t_2, \ldots, t_m$
- A PCFG (N, Σ, S, R, q):
  - N is the set of all non-terminals seen in the trees
  - Σ is the set of all words seen in the trees
  - S is taken to be S.
  - R is taken to be the set of all rules $\alpha \rightarrow \beta$ seen in the trees
  - The maximum-likelihood parameter estimates are:

$$q_{ML}(\alpha \rightarrow \beta) = \frac{\text{Count}(\alpha \rightarrow \beta)}{\text{Count}(\alpha)}$$

If we have seen the rule VP $\rightarrow$ Vt NP 105 times, and the the non-terminal VP 1000 times, $q(\text{VP} \rightarrow \text{Vt NP}) = 0.105$

Parsing with PCFGs

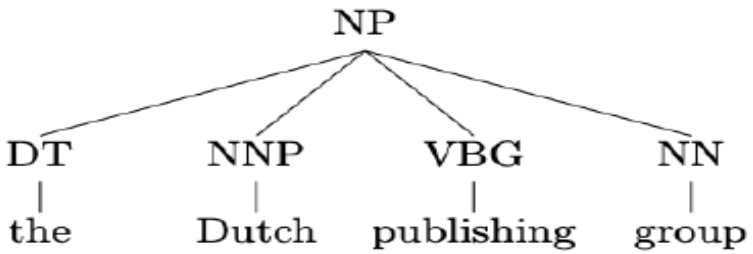
- Given a sentence s and a PCFG, how to find the highest scoring parse tree for s?

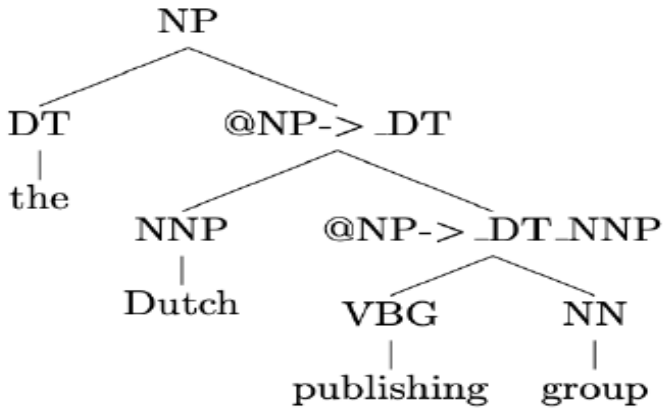
$$argmax_{t \in \mathcal{T}(s)} P(t)$$

- **The CKY algorithm**: applies to a PCFG in Chomsky normal form (CNF)
- **Chomsky Normal Form (CNF)**: all the rules take one of the two following forms:
  - $X \rightarrow Y_1 Y_2$ where $X \in N$, $Y_1 \in N$, $Y_2 \in N$
  - $X \rightarrow Y$ where $X \in N$, $Y \in \Sigma$
- It is possible to convert any PCFG into an equivalent grammar in CNF!
  - However, the trees will look differently; It is possible to do "reverse transformation"

Converting PCFGs into a CNF grammar

- n-ary rules ($n > 2$): NP → DT NNP VBG NN





- Unary rules: VP → Vi, Vi → sleeps
  - Eliminate all the unary rules recursively by adding VP → sleeps
  - We will come back to this later!

THE CYK ALGORITHM

- *The membership problem*:
  - Problem:
    - Given a context-free grammar **G** and a string **w**
      - **G** = (V, ∑ ,P , S) where
        - » V finite set of variables
        - » ∑ (the alphabet) finite set of terminal symbols
        - » P finite set of rules
        - » S start symbol (distinguished element of V)
        - » V and ∑ are assumed to be disjoint
      - **G** is used to generate the string of a language
  - Question:
    - Is **w** in **L**(**G**)?

THE CYK ALGORITHM

- J. Cocke
- D. Younger,
- T. Kasami

  – Independently developed an algorithm to answer this question.

THE CYK ALGORITHM BASICS

- The Structure of the rules in a Chomsky Normal Form grammar
- Uses a “dynamic programming” or “table-filling algorithm”

CHOMSKY NORMAL FORM

- *Normal Form* is described by a set of conditions that each rule in the grammar must satisfy
- Context-free grammar is in CNF if each rule has one of the following forms:
  - $A \rightarrow BC$ at most 2 symbols on right side
  - $A \rightarrow a$, or terminal symbol
  - $S \rightarrow \lambda$ null string

  where $B, C \in V - \{S\}$

CONSTRUCT A TRIANGULAR TABLE

- Each row corresponds to one length of substrings
  - Bottom Row – Strings of length 1
  - Second from Bottom Row – Strings of length 2

    .

    .
  - Top Row – string ‘w’

CONSTRUCT A TRIANGULAR TABLE

- $X_{i,\,i}$ is the set of variables A such that
  $A \rightarrow w_i$ is a production of G

- Compare at most n pairs of previously computed sets:

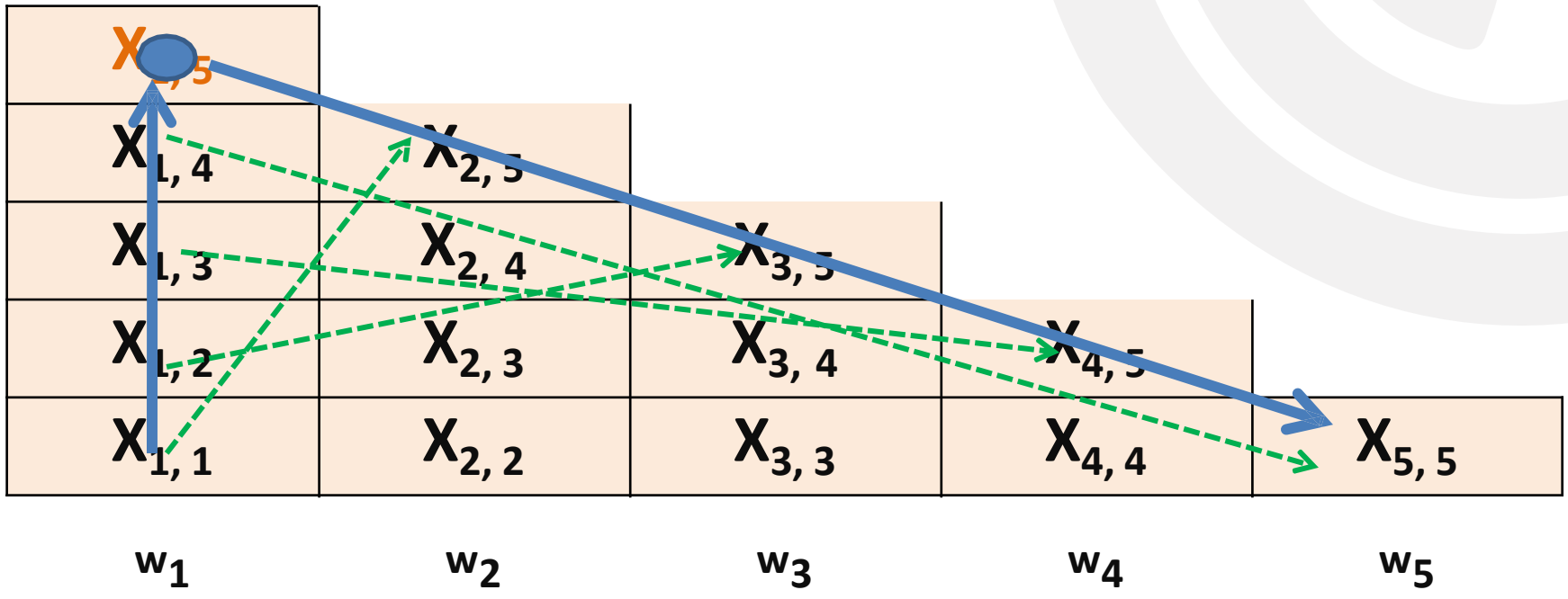
  $(X_{i,\,i}\ ,\ X_{i+1,\,j}),\ (X_{i,\,i+1}\ ,\ X_{i+2,\,j})\ \ldots\ (X_{i,\,j-1}\ ,\ X_{j,\,j})$

CONSTRUCT A TRIANGULAR TABLE

$X_{1,5}$				
$X_{1,4}$	$X_{2,5}$			
$X_{1,3}$	$X_{2,4}$	$X_{3,5}$		
$X_{1,2}$	$X_{2,3}$	$X_{3,4}$	$X_{4,5}$	
$X_{1,1}$	$X_{2,2}$	$X_{3,3}$	$X_{4,4}$	$X_{5,5}$
w_1	w_2	w_3	w_4	w_5

Table for string '**w**' that has length 5

CONSTRUCT A TRIANGULAR TABLE



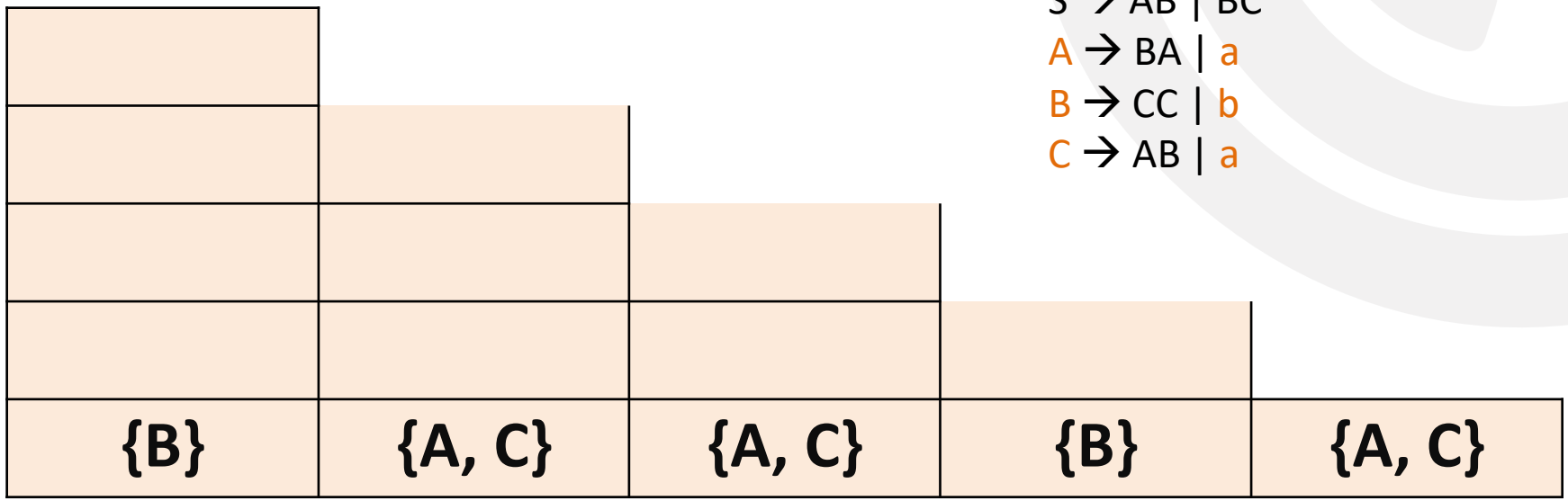
Looking for pairs to compare

EXAMPLE CYK ALGORITHM

- Show the CYK Algorithm with the following example:
  - CNF grammar **G**
    - S → AB | BC
    - A → BA | a
    - B → CC | b
    - C → AB | a
  - **w** is baaba
  - Question Is **baaba** in L(G)?

CONSTRUCTING THE TRIANGULAR TABLE

S → AB | BC
A → BA | a
B → CC | b
C → AB | a



Calculating the Bottom ROW

CONSTRUCTING THE TRIANGULAR TABLE

- $X_{1,2} = (X_{i,i}, X_{i+1,j}) = (X_{1,1}, X_{2,2})$
- ➔ {B}{A,C} = {BA, BC}
- Steps:
  - Look for production rules to generate BA or BC
  - There are two: S and A
  - **$X_{1,2}$ = {S, A}**

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

CONSTRUCTING THE TRIANGULAR TABLE

{S, A}				
{B}	{A, C}	{A, C}	{B}	{A, C}
b	a	a	b	a

CONSTRUCTING THE TRIANGULAR TABLE

- $X_{2,3} = (X_{i,i}, X_{i+1,j}) = (X_{2,2}, X_{3,3})$
- ➔ {A, C}{A,C} = {AA, AC, CA, CC} = Y
- Steps:
  - Look for production rules to generate Y
  - There is one: B
  - $X_{2,3} = \{B\}$

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

CONSTRUCTING THE TRIANGULAR TABLE

{S, A}	{B}			
{B}	{A, C}	{A, C}	{B}	{A, C}
b	a	a	b	a

CONSTRUCTING THE TRIANGULAR TABLE

- $X_{3,4} = (X_{i,i}, X_{i+1,j}) = (X_{3,3}, X_{4,4})$
- ➔ {A, C}{B} = {AB, CB} = Y
- Steps:
  - Look for production rules to generate Y
  - There are two: S and C
  - **$X_{3,4}$ = {S, C}**

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

CONSTRUCTING THE TRIANGULAR TABLE

{S, A}	{B}	{S, C}		
{B}	{A, C}	{A, C}	{B}	{A, C}
b	a	a	b	a

CONSTRUCTING THE TRIANGULAR TABLE

- $X_{4,5} = (X_{i,i}, X_{i+1,j}) = (X_{4,4}, X_{5,5})$
- ➔ {B}{A, C} = {BA, BC} = Y
- Steps:
  - Look for production rules to generate Y
  - There are two: S and A
  - **$X_{4,5}$ = {S, A}**

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

CONSTRUCTING THE TRIANGULAR TABLE

{S, A}	{B}	{S, C}	{S, A}	
{B}	{A, C}	{A, C}	{B}	{A, C}
b	a	a	b	a

CONSTRUCTING THE TRIANGULAR TABLE

- $X_{1,3} = (X_{i,i}, X_{i+1,j})\ (X_{i,i+1}, X_{i+2,j})$
  $= (X_{1,1}, X_{2,3})\ , (X_{1,2}, X_{3,3})$
- ➔ {B}{B} **U** {S, A}{A, C}= {BB, SA, SC, AA, AC} = Y
- Steps:
  - Look for production rules to generate Y
  - There are NONE: S and A
  - $X_{1,3} = \emptyset$
  - **no elements in this set (empty set)**

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

CONSTRUCTING THE TRIANGULAR TABLE

Ø				
{S, A}	{B}	{S, C}	{S, A}	
{B}	{A, C}	{A, C}	{B}	{A, C}
b	a	a	b	a

CONSTRUCTING THE TRIANGULAR TABLE

- $X_{2,4} = (X_{i,i}, X_{i+1,j})\ (X_{i,i+1}, X_{i+2,j})$
  $= (X_{2,2}, X_{3,4}), (X_{2,3}, X_{4,4})$
- ➔ {A, C}{S, C} **U** {B}{B}= {AS, AC, CS, CC, BB} = Y
- Steps:
  - Look for production rules to generate Y
  - There is one: B
  - **$X_{2,4}$ = {B}**

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

CONSTRUCTING THE TRIANGULAR TABLE

Ø	{B}			
{S, A}	{B}	{S, C}	{S, A}	
{B}	{A, C}	{A, C}	{B}	{A, C}

b a a b a

CONSTRUCTING THE TRIANGULAR TABLE

- $X_{3,5} = (X_{i,i}, X_{i+1,j})\ (X_{i,i+1}, X_{i+2,j})$
  $= (X_{3,3}, X_{4,5}), (X_{3,4}, X_{5,5})$
- ➔ {A,C}{S,A} **U** {S,C}{A,C}
  = {AS, AA, CS, CA, SA, SC, CA, CC} = Y
- Steps:
  - Look for production rules to generate Y
  - There is one: B
  - **$X_{3,5}$ = {B}**

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

CONSTRUCTING THE TRIANGULAR TABLE

Ø	{B}	{B}		
{S, A}	{B}	{S, C}	{S, A}	
{B}	{A, C}	{A, C}	{B}	{A, C}
b	a	a	b	a

FINAL TRIANGULAR TABLE

{S, A, C}	← $X_{1,5}$			
Ø	{S, A, C}			
Ø	{B}	{B}		
{S, A}	{B}	{S, C}	{S, A}	
{B}	{A, C}	{A, C}	{B}	{A, C}
b	a	a	b	a

- Table for string '**w**' that has length 5
- The algorithm populates the triangular table

EXAMPLE (RESULT)

- Is baaba in L(G)?

Yes

We can see the S in the set X_{1n} where ‘n’ = 5

We can see the table

the cell X_{15} = (S, A, C) then

if S ϵ X_{15} then baaba ϵ L(G)

THEOREM

- The CYK Algorithm correctly computes X_{ij} for all i and j; thus w is in L(G) if and only if S is in X_{1n}.
- The running time of the algorithm is $O(n^3)$.